



UNIVERSITÀ DEGLI STUDI DI ROMA "TOR VERGATA"

FACOLTÀ DI INGEGNERIA

CORSO DI LAUREA MAGISTRALE IN INGEGNERIA INFORMATICA

Performance Modeling of Computer Systems and Networks

Anno Accademico 2025/2026

Relazione del Progetto:

Modellazione e Valutazione delle Prestazioni di un'Infrastruttura

Cloud con Autoscaling Gerarchico

FRANCESCO MASCI

Matricola 0365258

Indice

1	Introduzione	1
2	Descrizione del Sistema	2
2.1	Componenti Logici dell'Infrastruttura	2
2.2	Dinamiche di Routing	3
2.3	Logiche di Scaling	4
3	Obiettivi della Simulazione	7
3.1	Dimensionamento Infrastrutturale	7
3.2	Analisi delle Prestazioni	8
3.3	Mitigazione dell'Instabilità	9
4	Modellazione Concettuale del Sistema	10
4.1	Variabili di Input	10
4.2	Caratterizzazione dei Centri di Servizio	11
4.3	Definizione dello Stato del Sistema	12
4.4	Analisi degli Eventi Discreti	13
5	Modello delle Specifiche	14
5.1	Tempi di Interarrivo	14
5.2	Tempi di Servizio e Processor Sharing	15
5.3	Probabilità di Routing	16
6	Modello Computazionale	18
6.1	Oggetti di Dominio e Rappresentazione dello Stato	18
6.2	Controller e Motore Next-Event	20
6.3	Routing, Load Manager e Meccanismi di Scaling	21
6.4	Builder, Facade e Modalità di Esecuzione	22
6.5	Accumulatori Statistici e Intervalli di Confidenza	23
6.6	Autocorrelazione, Cutoff e Scelta dei Batch	25
6.7	Stima della Convergenza e Scelta del Warm-Up	26
6.8	Generatori Pseudo-Casuali e Variate Aleatorie	27
7	Verifica del Simulatore	29
7.1	Verifica Analitica e Consistenza Statistica	29
7.2	Verifica delle Logiche di Routing	31
7.3	Verifica dei Meccanismi di Scaling	36
8	Validazione del Sistema	40
8.1	Validazione della Stazione Singola Iperesponenziale	40
8.2	Validazione dell'Architettura Web Server e Spike Server	42
9	Risultati e Analisi	44
9.1	Dimensionamento	44
9.2	Analisi	52

10 Obiettivo di Miglioramento	66
10.1 Modifica al Modello Concettuale	66
10.2 Modello delle Specifiche	66
10.3 Modello Computazionale	67
10.4 Verifica e Validazione	67
10.5 Risultati e Considerazioni	67
11 Conclusioni	70

1. Introduzione

L'attuale paradigma del *Cloud Computing* ha rivoluzionato le modalità di erogazione dei servizi software, offrendo alle organizzazioni la capacità di attingere a risorse computazionali virtualmente illimitate con un modello di consumo *pay-per-use*. In questo scenario, l'efficienza operativa ed economica dipende strettamente dalla capacità di gestire dinamicamente le risorse in risposta a carichi di lavoro intrinsecamente volatili. La garanzia della continuità del servizio e il rispetto dei *Service Level Agreements* rappresentano requisiti fondamentali per la competitività e la reputazione aziendale nel mercato digitale globale.

Le fluttuazioni del carico sui server dei datacenter, sia privati che pubblici, sono dovute agli effetti combinati di due diversi aspetti stocastici: la variabilità della frequenza del traffico in ingresso e la variabilità del tempo di calcolo richiesto per il soddisfacimento delle singole richieste di servizio. Tali fluttuazioni presentano intensità, durate e scale di tempo marcatamente eterogenee, che possono essere classificate in due macro-categorie:

- **Long-term fluctuations:** oscillazioni a bassa frequenza e intensità medio/bassa, tipicamente guidate dall'evoluzione dei pattern di utilizzo del server e dai trend di consumo sul lungo periodo.
- **Short-term fluctuations:** caratterizzate da alta frequenza, intensità elevata e breve durata; tali picchi transitori possono manifestarsi a istanti di tempo imprevedibili, saturando repentinamente le risorse computazionali.

A fronte di questa dinamicità, emerge il problema del corretto dimensionamento, volto a determinare il corretto numero di risorse necessarie a raggiungere gli obiettivi di performance prestabiliti minimizzando i costi operativi. Un dimensionamento statico fallisce inevitabilmente esponendo il sistema a due scenari critici: l'*over-provisioning*, che si traduce in uno spreco inefficiente di risorse e capitale economico, e l'*under-provisioning*, responsabile della saturazione delle code, dell'innalzamento asintotico dei tempi di risposta e della conseguente violazione delle metriche di Quality of Service imposte dagli SLA.

Sebbene le tecniche tradizionali di *horizontal autoscaling* offrano ottimi risultati nel contrastare le fluttuazioni a lungo termine, la loro applicazione a carichi soggetti a *short-term fluctuations* si rivela ampiamente insoddisfacente. La natura reattiva degli algoritmi orizzontali e i ritardi intrinseci legati ai tempi di boot delle istanze provocano una congestione dinamica delle risorse, innescando tempi di risposta estremamente elevati. Inoltre, la rapida transitorietà dei picchi può indurre l'ottimizzatore a decisioni di scaling contraddittorie in intervalli temporali ridotti, generando pericolose oscillazioni e instabilità nel numero di risorse fornite.

Per ovviare a tali inconvenienti, il presente studio si propone di analizzare l'impatto prestazionale di un'architettura di *autoscaling gerarchico* cooperativa. L'obiettivo principale è valutare quantitativamente, mediante simulazione a eventi discreti, l'efficacia del dirottamento dinamico dei carichi di picco verso un'unità computazionale ausiliaria, studiando la sensitività del sistema rispetto alla reattività dei parametri di controllo e verificando la robustezza del meccanismo di protezione a fronte di brusche variazioni nell'intensità del traffico in ingresso.

2. Descrizione del Sistema

L'infrastruttura oggetto di studio è un datacenter orientato all'erogazione di servizi web, governato da un modulo di controllo intelligente delle risorse distribuite: il Load controller. L'architettura è concepita per rispondere in modo coordinato a carichi di lavoro complessi, caratterizzati dalla sovrapposizione di dinamiche transitorie e strutturali che mettono a rischio la stabilità delle prestazioni e il rispetto degli accordi sui livelli di servizio.

Il funzionamento del sistema si fonda sulla separazione tra la gestione dei flussi operativi ordinari (Layer 1) e l'attivazione dei canali d'emergenza (Layer 2). Questa orchestrazione è interamente affidata al Load controller, il quale monitora costantemente lo stato computazionale dell'infrastruttura per gestire in modo integrato sia l'instradamento dinamico del traffico in ingresso, sia lo scaling delle risorse su scale temporali differenti.

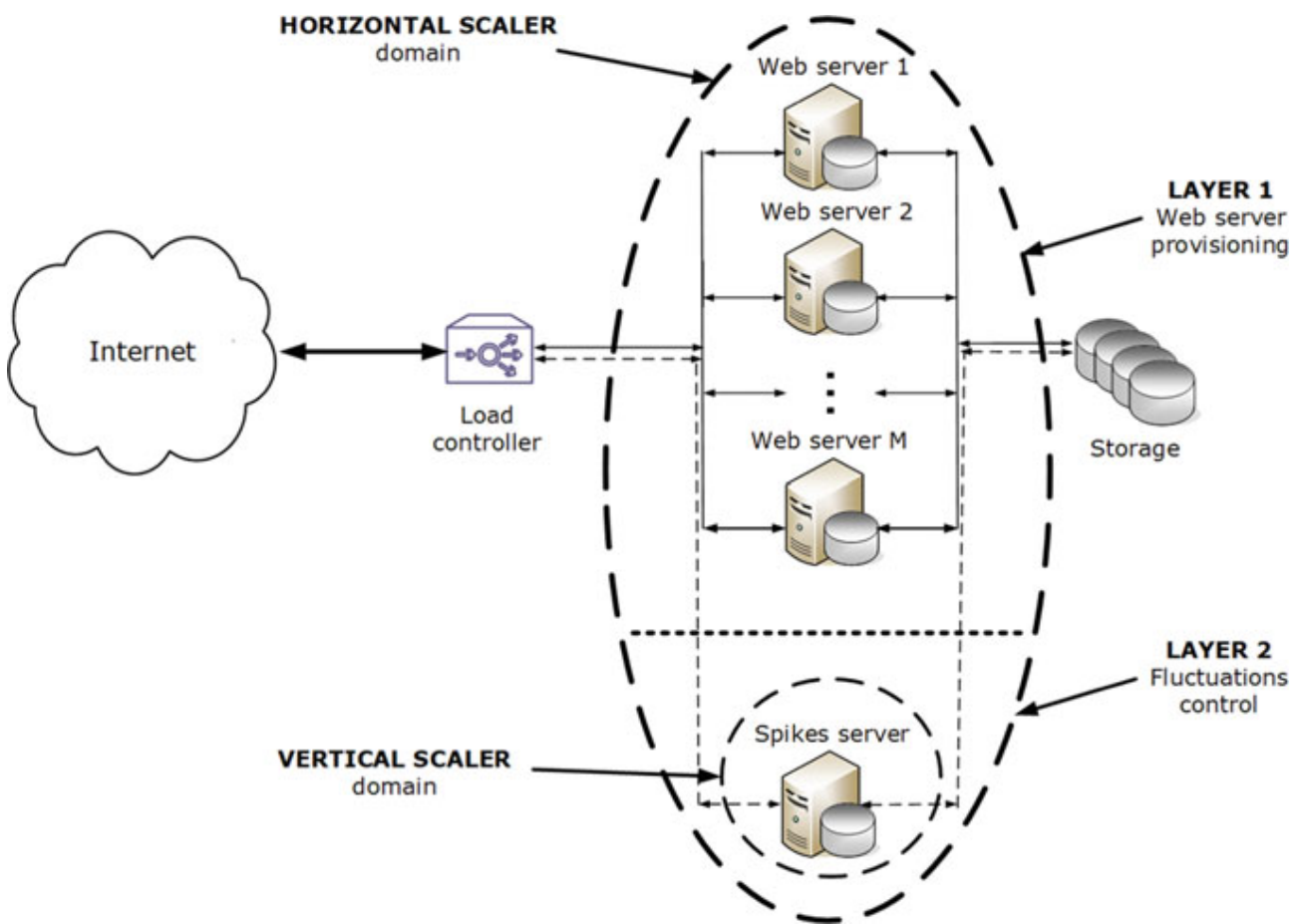


Figura 1: Rappresentazione schematica dell'architettura ad alto livello dell'autoscaler gerarchico [2, Fig. 6.8].

2.1 Componenti Logici dell'Infrastruttura

Da una prospettiva puramente funzionale ed astratta, il sistema è composto da tre macro-moduli interconnessi che cooperano per elaborare il flusso di richieste in ingresso:

- **Load Controller:** Rappresenta il punto di ingresso unico del datacenter. Questo modulo riceve l'intero flusso del traffico esterno e applica algoritmi di instradamento dinamico

basati sullo stato istantaneo dei nodi di calcolo, agendo come decisore di prima istanza per l'allocazione del carico.

- **Cluster di Web Server:** È costituito da un pool di server standard destinati all'elaborazione del carico di lavoro convenzionale. Ciascun server assegna le proprie risorse hardware secondo logiche *Processor Sharing*, tipiche dei server e della architetture di calcolo moderne, in cui la potenza di calcolo della CPU viene costantemente ripartita tra tutte le transazioni concorrenti in carico, evitando che singoli task monopolizzino il nodo.
- **Spike Server:** È una risorsa computazionale ad alta capacità mantenuta in uno stato di disponibilità elastica. Questo server non partecipa al bilanciamento ordinario del traffico, ma interviene esclusivamente come camera di compensazione quando i nodi del cluster principale si trovano in una condizione di congestione o stress critico.

2.2 Dinamiche di Routing

Il meccanismo che governa la deviazione dei flussi applicativi si basa sul concetto di **metrica reattiva istantanea**. Anziché affidarsi a complessi e computazionalmente onerosi algoritmi di analisi predittiva delle serie storiche del traffico, l'orchestratore gestisce l'instradamento attraverso una sequenza logica a due stadi indipendenti, comune a qualsiasi stato di carico del sistema:

1. **Selezione del Web Server:** All'arrivo di una nuova richiesta, il Load Controller applica preventivamente la politica di distribuzione principale per individuare un Web Server target all'interno del cluster.
2. **Valutazione dello Spike Indicator:** Una volta determinato il server di destinazione, l'orchestratore ne interroga lo stato istantaneo estraendo lo *Spike Indicator SI*, identificato nel numero esatto di richieste concorrenti che quel singolo nodo sta elaborando in quell'istante. Questa metrica viene confrontata istante per istante con la soglia critica di allarme SI^{max} .

Il comportamento del router e la destinazione finale della richiesta seguono quindi una logica di controllo puramente reattiva e *memoryless*: la decisione di instradamento viene presa per ogni singolo compito computazionale in tempo reale, basandosi esclusivamente sul valore puntuale di SI osservato all'istante di campionamento. A seconda del risultato del confronto con la soglia d'allarme, si determinano due scenari di instradamento alternativi:

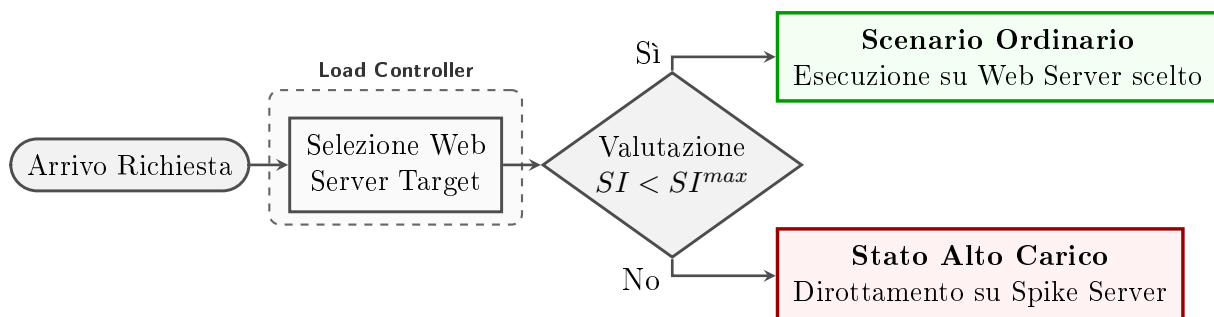


Figura 2: Diagramma di flusso sequenziale del processo decisionale e di routing.

Scenario Ordinario - $SI < SI^{max}$: Questo scenario si verifica nel momento in cui lo *Spike Indicator* calcolato sul Web Server selezionato soddisfa la condizione di stabilità, indicando che l'intensità istantanea del traffico e le relative domande di servizio si mantengono entro i margini di capacità del nodo computazionale di linea. In questo caso, il Load Controller convalida la scelta del server target e vi immette direttamente la richiesta in ingresso per l'ordinaria esecuzione; poichè la richiesta viene interamente presa in carico ed elaborata dal cluster di linea, lo *Spike Server* non viene coinvolta nel flusso.

Scenario di Alto Carico - $SI \geq SI^{max}$: Questo scenario si attiva nel momento esatto in cui il controllo rileva che l'occupazione istantanea del server target ha raggiunto o superato il limite di tolleranza, indicando l'insorgenza di un picco transitorio causato da un improvviso addensamento del tasso di arrivo o da un incremento imprevisto della complessità intrinseca dei task in esecuzione. Per prevenire un collasso prestazionale del *Web Server* selezionato e il drastico innalzamento dei tempi di risposta, il modulo attiva immediatamente una deviazione d'emergenza, instradando la richiesta corrente verso lo *Spike Server*. Di conseguenza, il *Web Server* target sperimenta un'interruzione mirata dei nuovi arrivi che gli permette di operare in una modalità di svuotamento controllato, concentrando la potenza di calcolo esclusivamente sullo smaltimento dei processi precedentemente presi in carico; in questo modo, lo *Spike Indicator* decresce progressivamente man mano che i task completano l'esecuzione e abbandonano la stazione. Il dirottamento applicato alle nuove richieste decade poi istantaneamente non appena un successivo arrivo campiona un valore dello *Spike Indicator* ritornato strettamente al di sotto della soglia di sicurezza SI^{max} , ripristinando la normale allocazione sul cluster di linea.

2.3 Logiche di Scaling

L'efficienza complessiva dell'architettura gerarchica risiede nella cooperazione strategica e coordinata tra le politiche di **routing reattivo** e le logiche di **autoscaling delle risorse**: questi due sistemi non operano come entità isolate, ma si integrano sinergicamente su piani strutturali e scale temporali marcatamente differenziate per proteggere l'infrastruttura. Se da un lato il routing interviene istante per istante a livello microscopico, dall'altro lato i meccanismi di scaling adattano dinamicamente la topologia e la capacità hardware del sistema per assorbire i carichi sul lungo e sul breve termine.

Scaler Orizzontale: Questa logica agisce sulla dimensionalità quantitativa del cluster principale e opera su una scala temporale macroscopica a bassa frequenza, erigendosi a presidio delle variazioni strutturali e a lungo termine del traffico. Il meccanismo valuta la necessità di ridimensionare il pool di linea monitorando costantemente il tempo medio di risposta delle richieste, calcolato all'interno di una finestra temporale mobile; l'adozione di tale media mobile si rivela fondamentale per filtrare il rumore stocastico e prevenire reazioni premature a fluttuazioni transitorie. Quando il tempo medio di risposta supera la soglia di sbarramento superiore R^{max} e il periodo di vincolo temporale dall'ultima riconfigurazione è completamente esaurito, l'orchestratore dispone l'allocazione e l'integrazione immediata di un nuovo server nel cluster per ripristinare i livelli di servizio attesi. Al contrario, qualora la metrica basata sulla finestra mobile si attesti stabilmente al di sotto della soglia inferiore R^{min} , il sistema pianifica la dismissione delle risorse in esubero per minimizzare i costi operativi. Al fine di preservare la continuità del servizio e non penalizzare le transazioni in corso, la contrazione della capacità applica una rigida politica di svuotamento controllato: il nodo designato alla rimozione viene immediatamente rimosso dalle rotazioni del Load Controller per bloccare l'ingresso di

nuovi flussi, ma viene mantenuto alimentato ed attivo il tempo necessario a consentire il totale completamento di tutti i processi precedentemente presi in carico e rimasti pendenti nella sua coda.

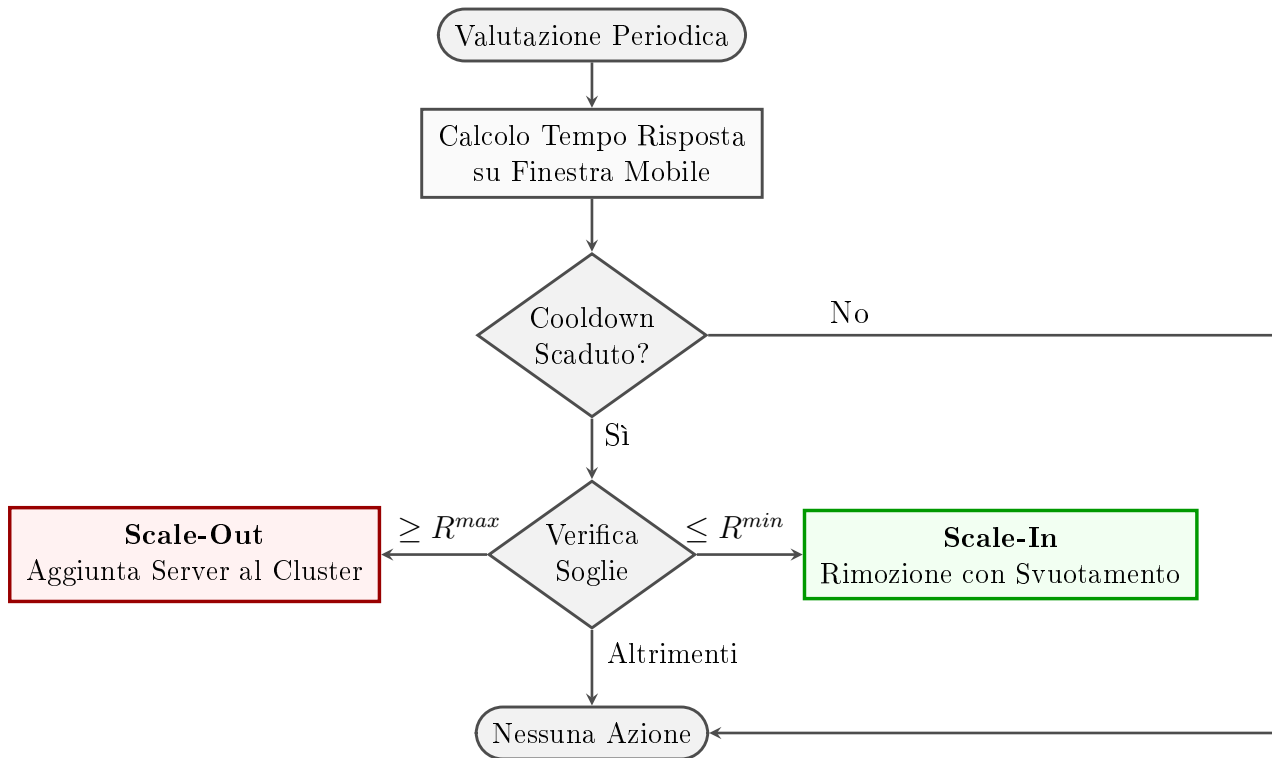


Figura 3: Logica di monitoraggio e processo decisionale dell'autoscaling orizzontale.

Scaler Verticale: Questa logica interviene sulla capacità computazionale dell'unità di riserva e opera su una scala temporale ad attivazione immediata, rispondendo alle repentine e violente variazioni indotte dai picchi di traffico ad alta frequenza. Il meccanismo di controllo agisce in modo dinamico modulando la quota di capacità hardware dedicata allo Spike Server (il fattore di capacità elaborativa della CPU) in base al carico istantaneo, valutato direttamente in termini di numero assoluto di richieste concorrenti simultaneamente attive all'interno della sua stazione. Nel momento esatto in cui il volume di task devianti e presi in carico dall'unità ausiliaria eguaglia o supera una determinata soglia limite superiore, e nel rispetto del tempo di sosta imposto dal relativo *cooldown*, l'ottimizzatore verticale interviene istantaneamente incrementando la potenza di calcolo allocata per accelerare i tempi di soddisfacimento dei processi d'emergenza. Questa accelerazione funge da scudo immediato e temporaneo, assorbendo il picco transitorio senza la necessità di attendere i lunghi tempi di boot tipici del provisioning orizzontale. Non appena la pressione stocastica si attenua e lo svuotamento delle code fa flettere il numero di richieste attive al di sotto della soglia di rilascio inferiore, il sistema revoca l'assegnazione straordinaria di risorse ripristinando il fattore di capacità elaborativa alla sua configurazione di base, riducendo istantaneamente l'impronta economica e salvaguardando l'efficienza energetica del datacenter.

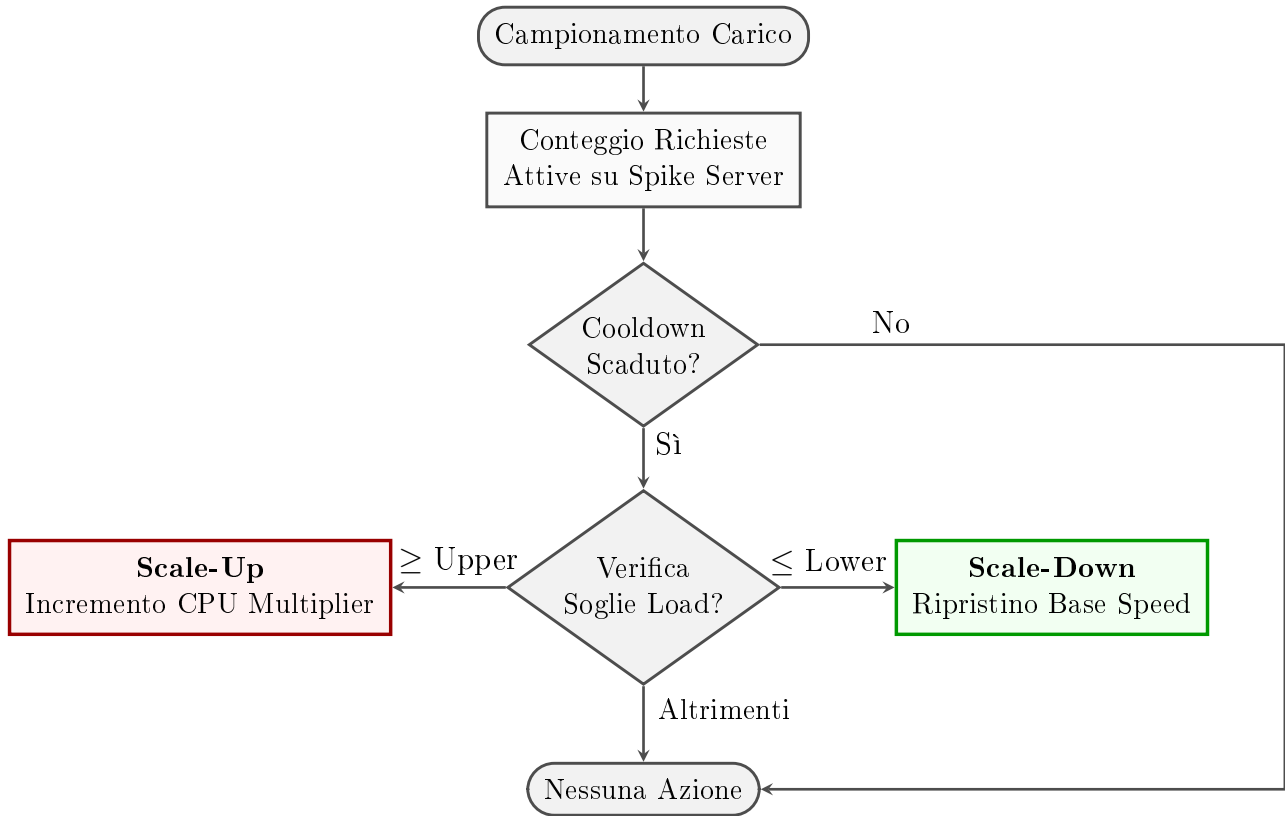


Figura 4: Processo di modulazione della capacità computazionale gestito dall'autoscaling verticale.

3. Obiettivi della Simulazione

La progettazione e lo sviluppo del simulatore a eventi discreti oggetto del presente studio sono orientati alla definizione, quantificazione e validazione empirica delle metriche analitiche e prestazionali che governano l'infrastruttura gerarchica proposta. L'obiettivo fondamentale della ricerca risiede nella dimostrazione, condotta attraverso un rigoroso approccio sperimentale, dell'efficacia derivante dalla stretta cooperazione e sinergia tra le decisioni di **routing reattivo istantaneo** (operate al Layer 2 dal Load Controller) e i meccanismi di **adattamento dinamico delle risorse** (attuati tramite le logiche di scaling orizzontale e verticale).

Attraverso la riproduzione di scenari sintetici caratterizzati da differenti intensità stocastiche e coefficienti di variabilità del traffico, la campagna di simulazione mira a mappare le interdipendenze tra l'allocazione hardware e la degradazione dei tempi di risposta. Questo percorso metodologico si articola in tre macro-aree di indagine sequenziali e strettamente interconnesse, orientate rispettivamente al dimensionamento ottimale dei parametri operativi, all'analisi di sensibilità economica e computazionale del sistema in contesti sia dinamici che stazionari, e infine all'introduzione di meccanismi ottimizzati per l'abbattimento dei fenomeni di instabilità computazionale nei casi limite.

3.1 Dimensionamento Infrastrutturale

La fase di dimensionamento si occupa di identificare la configurazione strutturale ottimale e i parametri operativi di base necessari a garantire che l'infrastruttura sia in grado di sostenere i livelli di traffico attesi, massimizzando l'efficienza hardware. In particolare, l'intero processo di calibrazione parametrica è guidato dal vincolo di progetto di mantenere un valore target massimo del tempo di risposta globale del sistema impostato rigorosamente a 6 secondi. Tale soglia critica rappresenta il limite di tolleranza oltre il quale il sistema viene considerato in uno stato di degradazione prestazionale, e funge da benchmark fondamentale per valutare l'efficacia delle decisioni di routing e delle soglie di intervento dei moduli di autoscaling.

Politiche di Instradamento: L'obiettivo primario sul piano della ripartizione del traffico risiede nella valutazione comparativa e quantitativa di diverse discipline stocastiche e deterministiche di bilanciamento del carico, assumendo come principale metrica di prestazione il tempo di risposta medio del sistema. Attraverso la campagna di simulazione, si intende analizzare in che misura l'adozione di differenti algoritmi influenzi l'efficienza del cluster ordinario, indipendentemente dallo *Spike Server*. L'indagine mira a identificare la politica di routing ottimale capace di minimizzare i tempi medi di attesa e di elaborazione dei job, riducendo l'insorgenza di colli di bottiglia locali e concorrendo attivamente a limitare la necessità di ricorrere al meccanismo di offloading verso lo *Spike Server*.

Calibrazione dello Scaling Verticale: In questo contesto, la ricerca si prefigge di validare l'efficacia e la stabilità della strategia di adattamento verticale applicata all'unità ausiliaria di gestione dei picchi. L'obiettivo si focalizza sulla calibrazione precisa dei parametri operativi che governano la capacità di questa risorsa di variare istantaneamente la propria potenza computazionale, modulando il fattore di capacità elaborativa della CPU in risposta al superamento delle soglie di carico stimate. L'analisi punta a verificare la capacità di tale scudo ad alta frequenza nel contenere le derive della latenza e della coda nei transitori critici in cui il cluster principale sperimenta una saturazione temporanea.

Dimensionamento dello Scaling Orizzontale: L'obiettivo è l'identificazione e il dimensionamento accurato dei parametri macroscopici che regolano la variazione della numerosità dei server attivi nel cluster principale. Sfruttando l'approccio empirico della simulazione, si intendono determinare le soglie critiche ottimali di aggiunta e di rimozione delle istanze computazionali, basate sulle metriche temporali aggregate ed elaborate tramite la finestra mobile. La ricerca è volta a definire una configurazione topologica bilanciata, capace di rispondere con agilità e precisione a variazioni di carico strutturali e sostenute nel tempo, minimizzando sia i ritardi di provisioning che potrebbero esporre il sistema a congestioni prolungate incompatibili con il target prestazionale, sia i fenomeni di instabilità o oscillazioni della capacità hardware.

3.2 Analisi delle Prestazioni

In questa fase si procede a una valutazione approfondita e sistematica delle prestazioni computazionali e dei costi operativi dell'infrastruttura, esaminandone il comportamento sia in contesti dinamici fortemente perturbati sia in condizioni stazionarie di regime. L'obiettivo centrale di questa indagine è verificare se, applicando i parametri operativi calibrati nella precedente fase di dimensionamento, l'infrastruttura sia effettivamente in grado di garantire e mantenere i Quality of Service attesi sul tempo di risposta globale. Contestualmente, l'analisi si propone di quantificare l'efficienza della soluzione proposta attraverso uno studio comparativo dell'impatto economico, misurando i benefici finanziari associati ai differenti profili di reattività del sistema.

Analisi dei Costi e Trade-off Economico: Viene condotta un'analisi economica orientata alla quantificazione del *trade-off* vincolante tra i costi operativi infrastrutturali e il guadagno prestazionale ottenuto. L'obiettivo finale consiste nell'individuare un punto di equilibrio ottimale che permetta di minimizzare la spesa operativa totale del datacenter, dimostrando la sostenibilità economica dello scaling gerarchico rispetto ai costi ridondanti legati a un provisioning fisso, pur garantendo il rispetto del vincolo del tempo di risposta entro il target stabilito per l'utente finale.

Analisi Transitoria e Dinamica dei Sistemi: L'indagine focalizzata sulle dinamiche transitorie del sistema viene condotta avviando deliberatamente l'ambiente di simulazione in uno stato completamente vuoto e inattivo. Questo approccio metodologico, basato sull'inizializzazione a zero delle code e delle risorse computazionali, si rivela indispensabile per mappare l'evoluzione temporale del sistema fuori dal regime e determinare con precisione l'istante di convergenza in cui l'infrastruttura raggiunge la stabilità operativa. Lo studio del transitorio iniziale permette così di quantificare la durata della fase di *warm-up*, necessaria affinché le metriche prestazionali si liberino dal bias delle condizioni iniziali, e di misurare empiricamente il tempo di reazione e i ritardi di adattamento dei moduli hardware a seguito dell'immissione del traffico stocastico. L'analisi consente infine di sottoporre a stress test il coordinamento microscopico tra i meccanismi di routing e le soglie di attivazione degli scaler proprio nelle fasi più volatili e critiche di avvio, verificando la tempestività dell'intero impianto gerarchico nel preservare i QoS legati alla latenza prima del consolidamento del regime stazionario.

Analisi in Stato Stazionario: Una volta esauriti completamente gli effetti dei transitori di avvio e superata la fase di *warm-up*, l'obiettivo si sposta sulla caratterizzazione e sulla verifica empirica del comportamento del sistema in stato stazionario. Questa indagine è finalizzata a osservare e validare le risposte operative e le prestazioni dell'infrastruttura a regime sotto l'effetto dei parametri strutturali precedentemente deliberati. Attraverso il monitoraggio puntuale

di indicatori chiave, come il livello di utilizzazione medio dello Spike Server e il tempo medio di risposta stabilizzato, lo studio si propone di confermare che la configurazione architetturale selezionata sia in grado di sostenere i tassi di arrivo costanti nel lungo periodo. L'analisi mira a certificare la robustezza intrinseca del sistema e la convergenza asintotica dei Quality of Service ai valori nominali di progetto, garantendo la stabilità dei flussi anche in presenza di sorgenti di traffico affette da un'elevata variabilità stocastica.

3.3 Mitigazione dell'Instabilità

L'ultima macro-area di indagine si concentra su una proposta evolutiva del sistema attraverso l'introduzione di logiche algoritmiche ottimizzate specificamente per la stabilizzazione dei controlli nei casi limite e negli scenari di transizione critica.

Meccanismo di Controllo Migliorativo: L'obiettivo conclusivo dello studio risiede nella valutazione empirica di una soluzione migliorativa volta a ottimizzare la gestione dei fenomeni di saturazione sui singoli nodi computazionali. Nello specifico, si analizza lo scenario critico in cui un Web Server del cluster opera in prossimità della soglia di allarme dello *Spike Indicator* SI^{max} : in assenza di contromisure, la natura stocastica del carico può produrre oscillazioni frequenti nelle decisioni di routing. Lo studio mira quindi a verificare se una logica di controllo più stabile, introdotta solo nella fase finale del progetto, consenta di ridurre tali oscillazioni senza compromettere il rispetto dello SLA e senza trasferire in modo eccessivo il costo operativo sullo Spike Server.

4. Modellazione Concettuale del Sistema

Il sistema viene formalizzato come una rete aperta di code operante in parallelo, all'interno della quale i flussi applicativi complessivi, caratterizzati da un tasso di arrivo globale λ , vengono intercettati ed orchestrati da un'unità centrale di controllo e indirizzati ai vari centri di servizio.

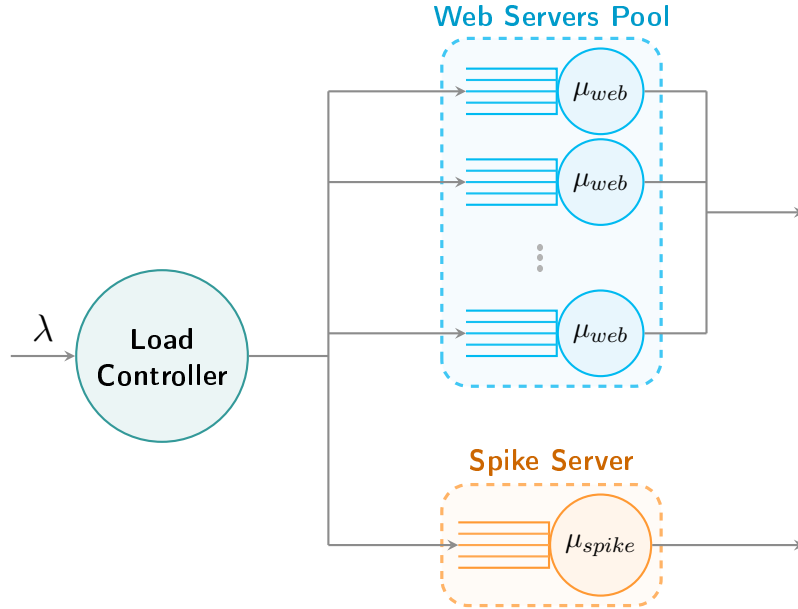


Figura 5: Modello concettuale a reti di code parallele.

4.1 Variabili di Input

La modellazione concettuale non si limita a descrivere la topologia logica del sistema, ma identifica anche l'insieme di parametri che verranno successivamente sottoposti a calibrazione sperimentale. Tali grandezze rappresentano le leve attraverso cui il controllore modifica il comportamento dell'infrastruttura e, di conseguenza, determinano il compromesso tra prestazioni, stabilità e costo operativo.

I parametri di input si distinguono in tre famiglie principali. La prima riguarda il **workload**, cioè il processo stocastico che alimenta il sistema: tasso medio di arrivo, variabilità degli interarrivi, domanda media di servizio e variabilità del servizio. La seconda comprende i **parametri strutturali** dell'architettura, come il numero minimo e massimo di Web Server e la presenza dello Spike Server. La terza raccoglie i **parametri di controllo**, cioè le soglie e i tempi di attesa che governano routing, scaling orizzontale e scaling verticale.

Questa separazione è utile anche dal punto di vista metodologico: i parametri di workload definiscono gli scenari sperimentali, i limiti strutturali delimitano le configurazioni ammissibili, mentre i parametri di controllo costituiscono l'oggetto principale del tuning. In particolare, la campagna di risultati procederà fissando progressivamente politica di routing, soglia SI_{max} , incremento verticale, soglie e finestra dello scaling orizzontale e *cooldown*; solo dopo tale calibrazione il sistema completo verrà valutato rispetto a costo, transitorio, burstiness e stabilità con il meccanismo di miglioramento.

Parametro	Categoria	Ruolo nel modello
λ	Workload	Tasso medio di ingresso delle richieste nel sistema; controlla l'intensità del carico applicato.
C_V^A	Workload	Coefficiente di variazione degli interarrivi; permette di passare da traffico regolare a traffico fortemente bursty.
$E[S], C_V^S$	Workload	Domanda media di servizio e relativa variabilità; determinano la complessità computazionale dei job.
π_R	Routing	Politica usata per selezionare il Web Server candidato all'arrivo di un nuovo job, scelta tra <i>Round Robin</i> , <i>Least Loaded</i> , <i>Random</i> e <i>Power of Two Choices</i> .
SI_{max}	Routing	Soglia di attivazione del dirottamento verso lo Spike Server nel modello base.
R^{max}, R^{min}	Scaling orizzontale	Soglie sul tempo medio di risposta osservato nella finestra mobile, usate per decidere <i>Scale Out</i> e <i>Scale In</i> .
Δv_s	Scaling verticale	Incremento additivo del fattore di capacità elaborativa dello Spike Server.
T_c	Controllo	Tempo di <i>cooldown</i> tra due valutazioni effettive di scaling, usato per evitare oscillazioni troppo ravvicinate.

Tabella 1: Parametri di input del modello concettuale sottoposti a calibrazione o analisi di sensitività.

4.2 Caratterizzazione dei Centri di Servizio

Ciascun componente strutturale viene astratto come un preciso centro di servizio, definito univocamente dalle proprie variabili di stato, capacità di memorizzazione e algebra di ripartizione della risorsa computazionale.

- **Load Controller:** Viene modellato concettualmente come un centro a servizio istantaneo e capacità di accumulo nulla, privo di code di attesa o servitori reali. Esso agisce come l'organo centrale di governo dell'infrastruttura, responsabile sia della scomposizione e instradamento del flusso poissoniano globale in ingresso in base alle politiche di routing, sia dell'attivazione delle logiche di scaling orizzontale e verticale per variare dinamicamente la topologia delle risorse computazionali.
- **Pool dei Web Server:** Costituito da un insieme di $N(t)$ stazioni di servizio parallele, disaccoppiate e indipendenti, dove la numerosità dei nodi attivi è regolata nel lungo termine dall'ottimizzatore orizzontale. Dal punto di vista analitico, ciascun nodo del pool è formalizzato come una stazione a capacità di accodamento infinita e a singolo servente con disciplina di servizio *Processor Sharing*.
- **Spike Server:** Viene astratto come una stazione di servizio singola a capacità illimitata, isolata topologicamente e operante anch'essa sotto disciplina *Processor Sharing* per garantire l'omogeneità analitica dei tempi di sosta.

4.3 Definizione dello Stato del Sistema

Lo stato del sistema al tempo t è formalizzato attraverso un vettore esteso di variabili aleatorie e deterministiche, in grado di mappare compiutamente sia l'occupazione fisica delle risorse computazionali, sia lo stato interno dei moduli di controllo e monitoraggio:

$$S(t) = \left\{ N(t), \mathbf{n}(t), n_s(t), v_s(t), \hat{W}_{mw}(t), \tau_h(t), \tau_v(t) \right\} \quad (1)$$

La componente legata all'infrastruttura hardware e all'allocazione del carico è descritta dalle seguenti variabili:

- $N(t) \in \{N_{min}, \dots, N_{max}\}$ rappresenta la numerosità istantanea dei Web Server attivi nel cluster di linea, governata dallo scaling orizzontale.
- $\mathbf{n}(t) = (n_1(t), n_2(t), \dots, n_{N(t)}(t))$ indica il vettore del carico corrente del pool, in cui ciascuna componente $n_i(t) \in \mathbb{N}$ esprime il numero di richieste concorrenti in fase di elaborazione all'interno del server i -esimo.
- $n_s(t) \in \mathbb{N}$ rappresenta la quantità di task simultaneamente presi in carico e in fase di processamento presso lo Spike Server.
- $v_s(t) \in \{v_{base}, v_{base} + \Delta v_s\}$ definisce il fattore di capacità elaborativa della stazione di picco, modulato in forma additiva dalle decisioni dello scaling verticale.

Le variabili di controllo, necessarie per tracciare la memoria storica dell'algoritmo e determinare l'evoluzione dei controllori, sono invece definite come:

- $\hat{W}_{mw}(t) \in \mathbb{R}^+$ indica la stima del tempo medio di risposta globale del sistema, calcolata dinamicamente sull'orizzonte temporale della finestra mobile attiva al tempo t , che funge da metrica di input per la logica di scaling orizzontale.
- $\tau_h(t) \in \mathbb{R}^+$ esprime il tempo di *cooldown* residuo per l'autoscaling orizzontale; se $\tau_h(t) > 0$, qualsiasi azione di *Scale-Out* o *Scale-In* viene temporaneamente inibita, e il timer decade linearmente con il progredire del tempo.
- $\tau_v(t) \in \mathbb{R}^+$ definisce il tempo di *cooldown* residuo associato al modulo di scaling verticale dell'unità ausiliaria, agendo come vincolo temporale per impedire oscillazioni ad alta frequenza sul fattore di capacità elaborativa dello Spike Server.

Transizioni nei Moduli di Scaling: Le variazioni delle variabili di controllo dello stato avvengono su scale temporali disaccoppiate e secondo logiche distinte:

- **Transizioni Orizzontali:** La mutazione della topologia del cluster, ovvero $\Delta N(t) = \pm 1$, avviene su eventi discreti. Essa è subordinata alla doppia condizione di azzeramento del timer residuo, che si ottiene con $\tau_h(t) = 0$, e di superamento delle soglie da parte della variabile $\hat{W}_{mw}(t)$.
- **Transizioni Verticali:** La variabile $v_s(t)$ scala tra i livelli discreti delle velocità in funzione del variare della componente di stato $n_s(t)$ (sempre secondo delle soglie impostate), a patto che il rispettivo timer di blocco verifichi la condizione $\tau_v(t) = 0$.

4.4 Analisi degli Eventi Discreti

L'evoluzione temporale del modello è orchestrata da un motore di simulazione a eventi discreti, in cui lo stato del sistema muta esclusivamente in corrispondenza di tali istanti temporali isolati.

Il ciclo di vita della simulazione è governato da tre tipologie fondamentali di eventi operativi:

1. **Evento di Arrivo:** Identifica l'immissione di una nuova richiesta nel sistema secondo il tasso stocastico λ . Il Load Controller intercetta l'evento ed esegue istantaneamente l'algoritmo di routing e il controllo a soglie differenziate, assegnando il job al Web Server candidato i o all'unità di riserva. Questa transizione incrementa la variabile di carico corrispondente $n_i(t) \leftarrow n_i(t) + 1$ oppure $n_s(t) \leftarrow n_s(t) + 1$, provocando la ridefinizione immediata della quota computazionale spettante ai task concorrenti e l'aggiornamento dei rispettivi tempi di completamento stimati.
2. **Evento di Completamento:** Segnala la fine del processamento di un job all'interno di un centro di servizio. L'evento determina il decremento unitario della variabile di occupazione del rispettivo nodo, $n_i(t) \leftarrow n_i(t) - 1$ oppure $n_s(t) \leftarrow n_s(t) - 1$, e la contestuale rimozione del job dal sistema. In questa esatta transizione, il simulatore effettua il campionamento della metrica sul tempo di risposta individuale, ne invia il valore al modulo della finestra mobile e rimodula al rialzo la velocità di elaborazione per i task rimasti all'interno della stazione.
3. **Evento Periodico di Monitoraggio:** Rappresenta un evento di clock deterministico e ricorrente, totalmente disaccoppiato dai flussi di traffico. Al verificarsi di questo trigger, gli agenti di controllo analizzano i valori assunti dalle metriche aggregate memorizzate nello stato, specificamente il tempo medio di risposta globale sulla finestra mobile $\hat{W}_{mw}(t)$ e l'occupazione della risorsa di picco $n_s(t)$, per valutare l'opportunità di variare la topologia del cluster $N(t)$ o il fattore di capacità elaborativa $v_s(t)$, aggiornando i relativi vettori dei cooldown qualora un'azione di scaling venga effettivamente validata e rischedulando un evento di monitoraggio ad una distanza pari proprio al cooldown.

5. Modello delle Specifiche

Il modello delle specifiche traduce il modello concettuale in vincoli quantitativi e leggi operative. In questa sezione vengono quindi fissati i processi di ingresso, la disciplina di servizio e le probabilità effettive di routing.

Le assunzioni comuni a tutte le specifiche sono: code a capacità illimitata, assenza di perdita di job, overhead nullo per routing e scaling, indipendenza tra interarrivi e domande di servizio e conservazione del lavoro nei centri *Processor Sharing*.

5.1 Tempi di Interarrivo

Il flusso esterno è rappresentato da una sequenza di job indicizzati da $j = 1, 2, \dots$, ciascuno associato a un istante di arrivo A_j . Gli interarrivi sono definiti come

$$T_j = A_j - A_{j-1}, \quad (2)$$

con valore medio $E[T] = 1/\lambda$.

Per rappresentare regimi operativi differenti sono stati costruiti scenari a basso, medio e alto carico, variando il tasso medio di arrivo e il coefficiente di variazione degli interarrivi. La Tabella 2 riporta i valori usati come specifica del workload nelle simulazioni a orizzonte finito e infinito.

Scenario	λ [job/s]	$E[T]$ [s]	C_V^A	Obiettivo modellistico
Basso carico	2.5	0.400	2.0, 4.0	Osservare il comportamento con risorse ordinariamente scariche e basso ricorso allo Spike Server.
Medio carico	5.0	0.200	4.0, 6.0	Rappresentare il regime nominale del progetto, vicino al carico usato per il tuning principale.
Alto carico	8.0	0.125	4.0, 6.0	Valutare la reazione del sistema in condizioni più stressanti, con maggiore probabilità di congestioni locali.
Frontiera di capacità	2.0–14.0	0.500–0.071	4.0	Ricostruire la regione di stabilità al variare del numero di Web Server attivi.

Tabella 2: Scenari di interarrivo utilizzati per simulare regimi di carico differenti.

Per gli esperimenti viene usata una distribuzione iperesponenziale a due fasi, indicata come H_2 , in modo da riprodurre interarrivi con coefficiente di variazione maggiore di uno. Questa scelta consente di generare sequenze in cui periodi di arrivi ravvicinati si alternano a intervalli più lunghi di quiete, comportamento più vicino ai carichi cloud reali. In forma generale, la variabile di interarrivo è descritta da:

$$T \sim \begin{cases} Exp(m_1) & \text{con probabilità } p, \\ Exp(m_2) & \text{con probabilità } 1 - p, \end{cases} \quad (3)$$

dove i parametri dei due rami sono scelti con il metodo dei *balanced means*. Indicando con m la media desiderata e con c il coefficiente di variazione, il simulatore calcola:

$$p = \frac{1}{2} \left(1 - \sqrt{\frac{c^2 - 1}{c^2 + 1}} \right), \quad m_1 = \frac{m}{2p}, \quad m_2 = \frac{m}{2(1-p)}. \quad (4)$$

Nel caso nominale degli interarrivi, $m = 0.20$ e $c = 4.0$, da cui si ottiene $p \simeq 0.0303$, $m_1 \simeq 3.297$ s e $m_2 \simeq 0.103$ s. La componente breve, molto più frequente, genera sequenze ravvicinate di arrivi; la componente lunga introduce periodi di quiete. Nel modello, l'aumento di C_V^A non modifica il tasso medio λ , ma aumenta la dispersione temporale degli arrivi e quindi la probabilità di congestioni locali temporanee.

5.2 Tempi di Servizio e Processor Sharing

Ogni job è caratterizzato da una domanda di servizio S_j , espressa in secondi di CPU normalizzata. Anche per il servizio sono considerate distribuzioni iperesponenziali; il valore medio $E[S]$ determina la capacità nominale richiesta da ogni job, mentre C_V^S modella l'eterogeneità della complessità computazionale delle richieste. Nel caso nominale si usa $E[S] = 0.25$ s e $C_V^S = 4.0$, quindi $\mu_{web} = 4.0$ job/s. Applicando gli stessi calcoli della distribuzione H_2 si ottengono $p \simeq 0.0303$, $m_1 \simeq 4.121$ s e $m_2 \simeq 0.129$ s per la domanda di servizio.

Evoluzione del Tasso di Servizio: All'interno di ciascuna stazione attiva, sia essa un Web Server i o lo Spike Server, l'effettivo tasso di completamento unitario non è costante, ma evolve in modo asincrono a ogni transizione del carico locale. Definita la capacità nominale del centro, il tasso di servizio istantaneo erogato al singolo job è condizionato inversamente dallo stato di occupazione del centro stesso.

Le stazioni di servizio adottano disciplina *Processor Sharing*. Un job accettato da un Web Server o dallo Spike Server entra immediatamente nell'insieme dei job attivi, senza attendere in una coda FCFS. Se al tempo t nel server i sono presenti $n_i(t) > 0$ job, ciascun job riceve una frazione della capacità disponibile pari a:

$$\mu_i^{job}(t) = \frac{\mu_i(t)}{n_i(t)}. \quad (5)$$

Per un Web Server ordinario si ha $\mu_i(t) = \mu_{web}$. Lo Spike Server presenta invece una capacità di servizio tempo-variante, determinata dal fattore di capacità elaborativa $v_s(t)$:

$$\mu_s(t) = v_s(t)\mu_{base}, \quad v_s(t) = \begin{cases} v_{base}, & \text{nello stato nominale,} \\ v_{base} + \Delta v_s, & \text{nello stato scalato.} \end{cases} \quad (6)$$

La variazione applicata dallo scaling verticale è quindi additiva rispetto al fattore di base: il termine Δv_s viene sommato a v_{base} (impostato pari a 1.0 negli esperimenti) quando il numero di job attivi supera la soglia superiore, e il fattore torna al valore nominale al raggiungimento della soglia inferiore. Sotto disciplina *Processor Sharing*, la capacità istantanea assegnata a ciascuno degli $n_s(t)$ job presenti sullo Spike Server risulta pertanto:

$$\mu_s^{job}(t) = \frac{v_s(t)\mu_{base}}{n_s(t)}. \quad (7)$$

Ad esempio, fissando $\mu_{web} = 4.0$ job/s e assumendo per semplicità una domanda media $E[S] = 0.25$ s, la capacità assegnata al singolo job decresce al crescere dei job concorrenti. Con

$n_i(t) = 4$ job attivi, ogni job riceve un tasso istantaneo pari a $\mu_i^{job} = 4/4 = 1$ job/s; in un intervallo di 1 s ciascun job consuma quindi una unità di lavoro normalizzata. Con $n_i(t) = 8$, invece, il tasso per job scende a 0.5 job/s e nello stesso intervallo ogni richiesta consuma solo metà del lavoro che avrebbe ricevuto nel caso precedente. La Tabella 3 mostra alcuni valori rappresentativi.

Job attivi $n_i(t)$	Tasso totale μ_{web}	Tasso per job μ_i^{job}	Lavoro ricevuto in 1 s
1	4.0	4.00	4.00
2	4.0	2.00	2.00
4	4.0	1.00	1.00
8	4.0	0.50	0.50

Tabella 3: Esempio numerico di ripartizione della capacità sotto disciplina Processor Sharing.

Questa scelta ha una conseguenza importante sulla gestione del tempo: il completamento schedato di un job non è definitivo fino a quando il numero di job residenti nella stazione rimane invariato. Ogni arrivo, completamento o variazione di velocità modifica infatti la quota di CPU assegnata a tutti i job attivi e richiede la rischedulazione dei futuri eventi di completamento. Nel modello non esiste quindi un tempo di attesa separato in senso FCFS; il tempo di risposta coincide con il tempo trascorso dal job all'interno della stazione, durante il quale esso riceve servizio in modo continuo ma condiviso.

5.3 Probabilità di Routing

Il Load Controller implementa due decisioni consecutive. La prima seleziona il Web Server candidato in base alla politica di routing ordinaria; la seconda valuta se il job debba essere effettivamente inviato al server candidato o deviato allo Spike Server.

Indicando con p_i la probabilità operativa che un arrivo venga inizialmente assegnato al Web Server i , il flusso nominale verso il server è:

$$\lambda_i^{cand} = p_i \lambda, \quad \sum_{i=1}^{N(t)} p_i = 1. \quad (8)$$

Il valore di p_i non è necessariamente costante: in *Round Robin* tende a distribuirsi uniformemente, in *Random* è uniforme in media, mentre in *Least Loaded* e *Power of Two Choices* dipende dallo stato istantaneo del cluster.

La seconda decisione introduce la probabilità effettiva di dirottamento. Detto q_i il valore osservato, a regime, della probabilità che un job candidato per il server i venga inviato allo Spike Server, i flussi effettivi possono essere espressi tramite:

$$\lambda_i = (1 - q_i) p_i \lambda, \quad \lambda_s = \sum_{i=1}^{N(t)} q_i p_i \lambda. \quad (9)$$

Il tasso di arrivo dello Spike Server è quindi intermittente e dipende dallo stato del cluster ordinario. In assenza di Web Server oltre soglia si ha $q_i = 0$ per ogni i e, di conseguenza, $\lambda_s = 0$; il flusso verso la risorsa ausiliaria si attiva esclusivamente quando almeno un job candidato viene dirottato in seguito al superamento della soglia critica del relativo Web Server. Il valore di q_i non è imposto a priori, ma emerge dalla politica a soglia sullo *Spike Indicator*. Nel

modello base, adottato per la calibrazione iniziale del sistema, il dirottamento avviene quando il Web Server candidato soddisfa:

$$SI_i(t) \geq SI_{max}. \quad (10)$$

La decisione sul singolo job può essere rappresentata dalla funzione indicatrice:

$$d_i(t) = \begin{cases} 1 & \text{se } SI_i(t) \geq SI_{max}, \\ 0 & \text{se } SI_i(t) < SI_{max}. \end{cases} \quad (11)$$

Questa percentuale non indica che tutto il traffico globale venga deviato, ma che il job candidato per quel server viene inviato integralmente allo Spike Server oppure integralmente al Web Server ordinario. La percentuale aggregata di traffico deviato a regime viene invece stimata tramite throughput:

$$D_{spike} = \frac{X_{spike}}{X_0} \simeq \frac{\lambda_s}{\lambda}, \quad (12)$$

dove X_{spike} è il throughput dei job completati dallo Spike Server e X_0 è il throughput complessivo del sistema. In questo modo il dirottamento viene letto come quota percentuale effettiva del lavoro servito dalla risorsa ausiliaria. La formulazione rende esplicito che il routing verso lo Spike Server non modifica il carico esterno complessivo, ma ne ridistribuisce una frazione quando il singolo nodo ordinario entra in una regione di congestione locale.

6. Modello Computazionale

Il modello computazionale traduce il modello concettuale e delle specifiche in un simulatore Java a eventi discreti. Il codice è organizzato nel repository del progetto, disponibile all'indirizzo [7], e adotta una separazione esplicita fra configurazione, oggetti di dominio, motore next-event, componenti di controllo, raccolta dati e output analysis. Questa organizzazione consente di mantenere separati i due livelli del lavoro: da una parte la logica del sistema simulato, dall'altra la logica sperimentale necessaria per verifica, validazione e tuning.

La struttura principale del progetto è suddivisa per responsabilità:

- **configs**: definizione dei parametri di carico, cluster, routing, scaling e metodo di esecuzione;
- **model**: rappresentazione di job, eventi, sorgenti di workload, server, cluster, routing e scaler;
- **controller**: interfaccia comune del simulatore e implementazione del motore a eventi discreti;
- **builder**: costruzione degli scenari a partire dalla configurazione;
- **facade**: API sperimentale ad alto livello per Batch Means, repliche indipendenti e aggregazione dei risultati;
- **controller.decorator**: raccolta di serie temporali, checkpoint di convergenza e salvataggio dei dati;
- **lib.rng** e **lib.statistics**: generatori pseudo-casuali, variate aleatorie, accumulatori statistici, autocorrelazione e intervalli di confidenza;
- **objective**: campagne sperimentali usate per stimare parametri e confrontare le alternative progettuali.

6.1 Oggetti di Dominio e Rappresentazione dello Stato

Il modello di simulazione è costruito a partire da un insieme ristretto di oggetti di dominio. La classe **Job** rappresenta una richiesta del sistema e contiene le informazioni necessarie per ricostruirne la storia: identificativo, istante di arrivo, domanda di servizio iniziale, domanda residua, istante di inizio ed istante di completamento. Il tempo di risposta R_j viene quindi ottenuto direttamente come:

$$R_j = C_j - A_j, \quad (13)$$

dove A_j è il tempo di arrivo e C_j il tempo di completamento del job. La presenza della domanda residua è fondamentale per implementare correttamente il Processor Sharing, poiché il servizio non viene prelevato in un'unica soluzione ma consumato progressivamente tra due eventi consecutivi.

Gli eventi sono rappresentati dalla classe **Event**, ordinata temporalmente all'interno della *Future Event List*. Ogni evento contiene il tempo di occorrenza, il tipo e, quando necessario, il job e il server coinvolti. I tipi di evento gestiti dal simulatore sono: arrivo, completamento, controllo di scaling orizzontale e controllo di scaling verticale. Questa scelta mantiene esplicite le uniche transizioni che possono modificare lo stato del sistema.

Astrazione comune dei server. La classe astratta **Server** definisce il comportamento comune dei nodi di servizio. Essa contiene l'identificativo del server, il fattore di capacità elaborativa, la lista dei job attivi, gli accumulatori statistici e le operazioni di accettazione, completamento e avanzamento del lavoro. La coda tradizionale è mantenuta solo per coerenza di interfaccia: nel modello PS i job accedono immediatamente al servizio e quindi non esiste tempo di attesa separato dal tempo di risposta.

Il nucleo computazionale del Processor Sharing è il metodo **processJobs**. Se durante un intervallo Δt sono presenti $n(t)$ job attivi e il fattore di capacità elaborativa del server è s , ciascun job riceve:

$$\Delta w_j = \frac{s\Delta t}{n(t)}. \quad (14)$$

Nel codice questa regola è implementata in modo diretto:

```

1 public void processJobs(double elapsed) {
2     if (elapsed <= 0.0 || activeJobs.isEmpty()) {
3         return;
4     }
5     double workPerJob = (elapsed * speedMultiplier)
6                         / activeJobs.size();
7     for (Job job : activeJobs) {
8         job.decreaseRemainingDemand(workPerJob);
9     }
10 }
```

I tempi di uscita dal sistema non dipendono unicamente dalla domanda di servizio originaria del job. Ogni volta che un job arriva o completa, il numero di job attivi cambia e quindi cambiano anche le date future di completamento degli altri job; lo stesso accade quando lo scaling verticale modifica il fattore di capacità elaborativa dello Spike Server. Per questo motivo il controller elimina e rischedula gli eventi di completamento relativi al server interessato dopo ciascuna di queste transizioni. Il tempo di completamento previsto per un job con domanda residua d_j è:

$$t_j^{comp} = t + \frac{d_j n(t)}{s}. \quad (15)$$

Questa formula è coerente con il fatto che, a parità di domanda residua, il completamento si allontana al crescere del numero di job concorrenti e si avvicina al crescere della velocità del server.

WebServer: specializza **Server** senza introdurre code aggiuntive o meccanismi di servizio differenti. Il suo ruolo è modellare un nodo applicativo standard del primo livello. L'unica grandezza specifica esposta dalla classe è lo *Spike Indicator*, definito come numero di job attualmente in esecuzione:

$$SI_i(t) = N_i(t). \quad (16)$$

```

1 public int getSpikeIndicator() {
2     return activeJobs.size();
3 }
```

SpikeServer: rappresenta il nodo speciale usato per assorbire richieste di picco. Anch'esso eredita la disciplina Processor Sharing da **Server**, ma aggiunge un accumulatore time-weighted per il fattore di capacità elaborativa. In questo modo il simulatore può stimare non solo l'utilizzazione dello Spike Server, ma anche la capacità elaborativa media impiegata durante la run. Questa informazione è usata negli obiettivi di costo, nei quali il costo computazionale dipende sia dal tempo di permanenza nello stato scalato sia dal livello medio di capacità allocata.

WebServerCluster Il cluster dei Web Server è gestito dalla classe **WebServerCluster**. Essa mantiene tre insiemi distinti: server attivi, server in *draining* e server complessivamente creati. La distinzione è necessaria per modellare correttamente lo scale-in: quando un server viene rimosso dal pool attivo non riceve più nuovi job, ma i job già presenti continuano ad essere serviti fino al completamento. Solo quando la lista dei job attivi diventa vuota il server può essere eliminato dalla parte operativa del sistema. In questo modo lo scaling non introduce perdita di lavoro e non altera artificialmente i tempi di risposta dei job già assegnati.

Il cluster aggiorna inoltre il numero medio di server attivi tramite un accumulatore time-weighted e mantiene i contatori delle azioni di scale-out e scale-in. Tali metriche sono poi raccolte dalla Facade e diventano parte degli output sperimentali.

6.2 Controller e Motore Next-Event

Il simulatore adotta un avanzamento temporale *next-event*: l'orologio non procede con passo fisso, ma salta direttamente al tempo del prossimo evento presente nella *Future Event List*.

L'interfaccia **Simulator** definisce il contratto comune del motore, mentre **SimulationController** ne implementa la logica concreta. La coda degli eventi futuri è una **PriorityQueue<Event>**; il ciclo di esecuzione prosegue fino al raggiungimento della condizione di arresto richiesta, che può essere un numero di completamenti, un orizzonte temporale o l'esaurimento della coda eventi:

```
1 while (getTotalJobsCompleted() < targetJobs
2     && getClock() < targetTime
3     && processNextEvent()) {
4     // optional progress reporting
5 }
```

L'elaborazione di un singolo evento segue sempre la stessa sequenza: estrazione dell'evento più vicino, avanzamento del lavoro accumulato nell'intervallo trascorso, aggiornamento dell'orologio e dispatch verso il gestore specifico:

```
1 Event event = eventQueue.poll();
2 double nextTime = event.getTime();
3
4 processActiveJobs(nextTime - clock, nextTime);
5 clock = nextTime;
6 processEvent(event);
```

Nel caso di un arrivo, il controller genera il job successivo, invoca il **LoadManager** per la scelta del server, accetta il job nella stazione selezionata, rischedula i completamenti della stazione e aggiorna le statistiche globali. Nel caso di un completamento, il job viene rimosso dal server, il tempo di risposta viene sommato alle statistiche globali, il completamento viene passato allo

scaler orizzontale e gli eventi di completamento del server vengono ricalcolati. Nei controlli periodici di scaling, invece, il controller interroga gli scaler e, se la capacità del sistema cambia, rischedula tutti i completamenti perché le velocità effettive o il numero di server disponibili sono stati modificati.

Il controller mantiene anche le grandezze globali del sistema: numero totale di arrivi, completamenti, job dirottati, tempo di risposta cumulato, throughput, numero medio di job nel sistema e utilizzazione media. Le ultime due metriche sono processi a tempo continuo e vengono aggiornate tramite `TimedWelford`; il numero di job nel sistema è la somma dei job attivi sui Web Server attivi, sui server in draining e, quando abilitato, sullo Spike Server.

6.3 Routing, Load Manager e Meccanismi di Scaling

La logica di controllo del carico è raccolta nella classe `LoadManager`. Il suo compito non è implementare direttamente una politica specifica, ma coordinare tre componenti sostituibili: il router, lo scaler orizzontale e lo scaler verticale. Questa scelta rende possibile confrontare politiche differenti senza modificare il motore di simulazione.

Routing. Il routing è diviso in due passaggi. Prima viene selezionato un Web Server candidato tramite una strategia fra quelle implementate nel package di routing: Round Robin, Least Loaded, Random, Power of Two Choices e varianti deterministiche. Successivamente la strategia di routing verso lo Spike Server decide se mantenere il job sul Web Server candidato o deviarlo. La classe `Router` rende esplicita questa composizione:

```

1 public final Server route(Job job, WebServerCluster cluster,
2                           SpikeServer spikeServer) {
3     WebServer targetServer =
4         webServerRoutingStrategy.selectWebServer(job, cluster);
5     if (spikeServerRoutingStrategy.shouldRouteToSpike(targetServer, siMax)) {
6         return spikeServer;
7     }
8     return targetServer;
9 }

```

Nel modello base la strategia di dirottamento usa una soglia superiore sullo Spike Indicator. Con la configurazione senza bande, un job viene deviato quando il server candidato soddisfa:

$$SI_i(t) \geq SI_{max}. \quad (17)$$

Il meccanismo a bande viene introdotto solo nella parte di miglioramento, dove diventa un parametro oggetto di tuning; in questa sezione è sufficiente osservare che il router espone già l'astrazione necessaria per sostituire la regola di instradamento senza cambiare il controller.

Scaling orizzontale. Lo scaling orizzontale è implementato dalla classe `MovingWindowHorizontalScaler`. Ogni completamento produce una coppia formata dal tempo di completamento e dal tempo di risposta osservato. Lo scaler conserva gli ultimi `WINDOW_SIZE` completamenti e calcola la media mobile:

$$\bar{R}_{win}(t) = \frac{1}{|\mathcal{W}(t)|} \sum_{j \in \mathcal{W}(t)} R_j. \quad (18)$$

Se tale valore supera la soglia di *Scale Out*, il cluster prova ad aggiungere un Web Server; se scende sotto la soglia di *Scale In*, il cluster prova a rimuoverne uno. Entrambe le decisioni rispettano il cooldown, così da evitare oscillazioni troppo frequenti:

```

1  double avgResponse = getCurrentMetric(clock);
2
3  if (avgResponse >= scaleOutThreshold) {
4      boolean scaled = cluster.scaleOut(clock);
5      if (scaled) {
6          lastScalingTime = clock;
7          return true;
8      }
9  } else if (avgResponse <= scaleInThreshold) {
10     boolean scaled = cluster.scaleIn(clock);
11     if (scaled) {
12         lastScalingTime = clock;
13         return true;
14     }
15 }

```

La finestra è espressa in numero di completamenti, non in secondi. Questa scelta rende la regola più legata all'evidenza statistica raccolta dal sistema: a basso carico la finestra copre un intervallo temporale più lungo, mentre ad alto carico si aggiorna più rapidamente.

Scaling verticale. Lo scaling verticale è implementato da `LoadThresholdVerticalScaler`, che specializza `ThresholdVerticalScaler`. Il componente osserva il numero di job attivi sullo Spike Server e modifica il fattore di capacità elaborativa quando la metrica supera la soglia alta o scende sotto la soglia bassa. In particolare, lo stato scalato è ottenuto sommando l'incremento configurato al valore di base:

$$s_{spike}(t) = \begin{cases} s_{base} + \Delta v_s, & N_{spike}(t) \geq \theta_{up}, \\ s_{base}, & N_{spike}(t) \leq \theta_{down}. \end{cases} \quad (19)$$

Anche in questo caso la decisione è vincolata dal cooldown. Dopo una variazione del fattore di capacità, il controller rischedula gli eventi di completamento perché la capacità complessiva del server è cambiata.

6.4 Builder, Facade e Modalità di Esecuzione

La classe `SimulationBuilder` centralizza la costruzione del simulatore. A partire da `ApplicationConfig`, essa crea la sorgente del workload, il cluster di Web Server, lo Spike Server, le strategie di routing, gli scaler e il controller. L'estratto seguente mostra il punto in cui la configurazione viene trasformata in componenti eseguibili:

```

1  EventSource eventSource = switch (config.load().workloadType()) {
2      case TRACE -> new TraceEventSource(config.load().tracePath());
3      case HYPEREXPONENTIAL -> new HyperexponentialEventSource(
4          seed,
5          config.load().meanInterarrival(),
6          config.load().cvInterarrival(),

```

```

7      config.load().meanService(),
8      config.load().cvService()
9  );
10  case EXPONENTIAL -> new ExponentialEventSource(
11      seed,
12      config.load().meanInterarrival(),
13      config.load().meanService()
14  );
15  };
16
17  WebServerCluster cluster = new WebServerCluster(
18      config.cluster().minServers(),
19      config.cluster().maxServers()
20  );

```

La `SimulationFacade` rappresenta invece il punto di accesso per le campagne sperimentali. Essa nasconde i dettagli di costruzione del controller e offre tre modalità principali: singola simulazione, simulazione a Batch Means e simulazione a repliche indipendenti. Nel caso Batch Means, la Facade esegue prima un warm-up, azzerà le statistiche e poi raccoglie un campione per ciascun batch:

```

1  controller.run(StopCondition.untilJobsCompleted(warmUpJobs));
2  controller.resetStatistics();
3
4  for (int i = 0; i < numBatches; i++) {
5      controller.run(StopCondition.untilJobsCompleted(batchSize));
6      updateAggregators(controller);
7      controller.resetStatistics();
8  }

```

Nel caso delle repliche indipendenti, invece, la Facade ricostruisce un nuovo controller per ogni replica, inizializzando il generatore con il seed previsto per quella run. Le metriche prodotte da ogni replica vengono poi aggregate tramite accumulatori `Welford`. L'oggetto finale, `AggregatedResults`, contiene per ciascuna metrica media, deviazione standard, Half-Width e intervallo di confidenza. Le metriche aggregate includono: tempo di risposta, job nel sistema, utilizzazione globale, throughput, job deviati, numero medio di server attivi, azioni di scale-out, scale-in, scale-up, scale-down, fattore medio di capacità dello Spike Server e utilizzazione dello Spike Server.

I decorator completano questa architettura permettendo di aggiungere raccolte dati senza modificare il controller. Ad esempio, `TimeSerieCollector` raccoglie la serie dei tempi di risposta per la stima dell'autocorrelazione, `ConvergenceCheckpointCollector` registra metriche a checkpoint comuni fra repliche indipendenti e i decorator di storage salvano i risultati su CSV o JSON.

6.5 Accumulatori Statistici e Intervalli di Confidenza

Le statistiche del simulatore sono divise in due famiglie. Le metriche osservate per evento, come il tempo di risposta o il numero di azioni di scaling per run, sono campioni discreti e vengono aggregate con `Welford`. Le metriche che descrivono un processo nel tempo, come il numero di

job nel sistema, l'utilizzazione e il fattore di capacità elaborativa dello Spike Server, sono invece grandezze time-weighted e vengono aggregate con **TimedWelford**.

Welford. L'algoritmo di Welford aggiorna media e varianza in modo incrementale, evitando sia la memorizzazione di tutti i campioni sia il calcolo instabile basato sulla differenza fra somma dei quadrati e quadrato della somma. Se x_n è la nuova osservazione, l'aggiornamento è:

$$\delta_n = x_n - \bar{x}_{n-1}, \quad (20)$$

$$\bar{x}_n = \bar{x}_{n-1} + \frac{\delta_n}{n}, \quad (21)$$

$$M_{2,n} = M_{2,n-1} + \delta_n(x_n - \bar{x}_n). \quad (22)$$

Nel codice:

```
1 count++;
2 double delta = x - mean;
3 mean += delta / count;
4 double delta2 = x - mean;
5 m2 += delta * delta2;
```

La varianza campionaria viene poi stimata come $s^2 = M_2/(n - 1)$. Questo approccio è particolarmente adatto alle simulazioni lunghe, perché consente di elaborare milioni di osservazioni mantenendo memoria costante.

TimedWelford. Per le grandezze a tempo continuo non è corretto calcolare la media aritmetica dei valori osservati agli eventi, perché uno stato che dura molto deve pesare più di uno stato che dura pochissimo. La media corretta è:

$$\bar{x}_T = \frac{1}{T} \int_0^T x(t) dt \simeq \frac{\sum_r x_r \Delta t_r}{\sum_r \Delta t_r}. \quad (23)$$

TimedWelford implementa proprio questa integrazione a tratti costanti: quando il valore cambia al tempo corrente, il valore precedente viene accumulato con peso pari alla durata per cui è rimasto valido.

```
1 double duration = currentTime - lastTime;
2 if (duration > 0.0) {
3     update(currentValue, duration);
4 }
5 currentValue = newValue;
6 lastTime = currentTime;
```

Questa distinzione è essenziale per non distorcere utilizzazione e popolazione media del sistema, che sono grandezze definite rispetto al tempo e non rispetto al numero di eventi.

Intervalli di confidenza. Gli intervalli di confidenza sono stimati con la distribuzione t di Student, coerentemente con la procedura di output analysis per campioni indipendenti con varianza ignota [6, pp. 3–8]. La Half-Width al livello $1 - \alpha$ è:

$$HW = t_{n-1, 1-\alpha/2} \frac{s}{\sqrt{n}}, \quad (24)$$

e l'intervallo riportato nelle tabelle è:

$$[\bar{x} - HW, \bar{x} + HW]. \quad (25)$$

La classe `IntervalEstimator` riceve il numero di campioni, la media e la deviazione standard e restituisce direttamente media, deviazione standard, Half-Width ed estremi dell'intervallo.

6.6 Autocorrelazione, Cutoff e Scelta dei Batch

Nelle simulazioni a orizzonte infinito le osservazioni consecutive prodotte da una singola run lunga non possono essere considerate indipendenti. In particolare, nei sistemi di coda il tempo di risposta di un job è influenzato dallo stato lasciato dai job precedenti; questa dipendenza genera autocorrelazione seriale e rende eccessivamente ottimistici gli intervalli calcolati trattando le osservazioni grezze come indipendenti. Il problema e la necessità di individuare un taglio della correlazione sono discussi nel testo nella parte dedicata alla correlazione seriale e all'output analysis delle simulazioni stazionarie [1, pp. 180–183].

La funzione di autocorrelazione campionaria usata dal simulatore è:

$$\hat{\rho}(h) = \frac{\sum_{i=1}^{n-h} (x_i - \bar{x})(x_{i+h} - \bar{x})}{\sum_{i=1}^n (x_i - \bar{x})^2}, \quad (26)$$

dove h è il lag. Il cutoff L è scelto come il primo lag dal quale l'autocorrelazione rimane stabilmente al di sotto della soglia:

$$|\hat{\rho}(h)| < \frac{2}{\sqrt{n}}. \quad (27)$$

Per evitare di selezionare un lag isolato dovuto a fluttuazioni casuali, l'implementazione richiede che la condizione sia verificata per più lag consecutivi:

```

1 final int requiredConsecutiveLags =
2     (int) Math.max(5, Math.round(2.5 * Math.log10(n)));
3
4 if (acf < threshold) {
5     if (consecutiveBelow == 0) {
6         candidateLag = lag;
7     }
8     consecutiveBelow++;
9     if (consecutiveBelow >= requiredConsecutiveLags) {
10         return candidateLag;
11     }
12 } else {
13     consecutiveBelow = 0;
14 }
```

Il `BatchMeanEstimator` usa il cutoff per costruire batch sufficientemente distanziati rispetto alla memoria della serie. Nel metodo analitico viene eseguita una run pilota lunga, vengono raccolti i tempi di risposta tramite `TimeSerieCollector`, si calcola L e si imposta:

$$b = 10L, \quad k = \left\lfloor \frac{n}{b} \right\rfloor, \quad (28)$$

dove b è la dimensione del batch e k il numero di batch. La media del batch j è:

$$Y_j = \frac{1}{b} \sum_{i=(j-1)b+1}^{jb} X_i. \quad (29)$$

Le medie Y_j vengono poi trattate come campioni dell'output stazionario. A valle della costruzione dei batch viene calcolata la lag-1 ACF delle medie: valori prossimi a zero indicano che la struttura scelta ha ridotto in modo adeguato la dipendenza seriale.

Accanto al metodo basato su una run pilota fissa è stata implementata una versione dinamica. Questa procedura parte da una dimensione di batch iniziale, estende progressivamente la simulazione a blocchi e verifica due condizioni: precisione relativa dell'intervallo e autocorrelazione lag-1 delle medie di batch. Se la precisione è insufficiente vengono raccolti altri campioni; se l'autocorrelazione rimane elevata la dimensione del batch viene raddoppiata:

```

1 boolean independent = Math.abs(acf1) < 0.05;
2 boolean precise = relativePrecision < TARGET_PRECISION;
3
4 if (independent && precise) {
5     converged = true;
6 } else if (!independent) {
7     b *= 2;
8 }
```

Questo approccio è stato utile negli obiettivi più onerosi, dove una run pilota molto lunga avrebbe richiesto tempi di calcolo e memoria non convenienti. In tali casi la procedura dinamica fornisce una scelta operativa di b e k , poi arrotondata in modo conservativo nei parametri effettivi della campagna sperimentale.

6.7 Stima della Convergenza e Scelta del Warm-Up

La stima del warm-up è separata dalla stima dei batch. Il Batch Mean Estimator cerca parametri adeguati per l'analisi stazionaria, mentre **ConvergenceEstimator** valuta se e quando una specifica configurazione raggiunge una regione stabile. La procedura usa simulazioni a orizzonte finito con repliche indipendenti: per ogni replica viene eseguita una run lunga e un decorator registra, a checkpoint comuni espressi in numero di job completati, il tempo medio di risposta cumulato, il throughput e il tempo simulato.

I checkpoint non crescono con passo costante per tutta la run. Si parte da un incremento iniziale e, dopo un numero prefissato di finestre valutate, l'incremento viene moltiplicato. In questo modo la parte iniziale della traiettoria, dove il transiente è più importante, viene osservata con granularità maggiore, mentre la coda della simulazione viene esplorata con passi più ampi e costo computazionale minore.

Per ogni checkpoint la classe costruisce un intervallo di confidenza sulle repliche indipendenti. La configurazione viene considerata convergente quando il tempo medio di risposta mostra una variazione relativa inferiore alla tolleranza per un numero consecutivo di finestre e, nello stesso tempo, la Half-Width relativa rimane sotto la soglia di precisione. Viceversa, viene classificata come divergente quando la media cresce in modo statisticamente significativo per più finestre consecutive, con intervalli di confidenza che peggiorano senza sovrapporsi. Il punto di convergenza individuato viene poi usato come stima del numero di job di warm-up per le successive simulazioni a Batch Means.

6.8 Generatori Pseudo-Casuali e Variate Aleatorie

La libreria `lib.rng` deriva dalle classi `Rngs`, `Rvgs` e `Rvms` di Park, Miller e collaboratori, adottate nei testi di simulazione per la generazione di numeri pseudo-casuali e variate aleatorie [5, 1]. Il generatore di base è un Lehmer multiplicative congruential generator:

$$X_{n+1} = 48271X_n \bmod (2^{31} - 1), \quad U_n = \frac{X_n}{2^{31} - 1}. \quad (30)$$

Il modulo $m = 2147483647$ è primo e il periodo del generatore è $m - 1$. L'implementazione usa la forma di Schrage per evitare overflow nel prodotto fra moltiplicatore e stato:

```

1 long Q = MODULUS / MULTIPLIER;
2 long R = MODULUS % MULTIPLIER;
3 long t = MULTIPLIER * (seed[stream] % Q)
4       - R * (seed[stream] / Q);
5 seed[stream] = (t > 0) ? t : t + MODULUS;
6 return ((double) seed[stream] / MODULUS);

```

La classe `Rngs` espone 256 stream indipendenti. Questa caratteristica è importante per la riproducibilità degli esperimenti: arrivi, servizi e selezioni interne delle distribuzioni possono usare stream separati, così da evitare che una modifica locale, ad esempio l'aggiunta di una scelta casuale nel routing, alteri l'intera sequenza di servizi o arrivi. Il metodo `plantSeeds` inizializza tutti gli stream a partire da un unico seed, mantenendo quindi sia riproducibilità sia separazione delle sorgenti casuali.

Le variate esponenziali sono generate dalla classe `Rvgs` tramite trasformata inversa. Se U è uniforme in $(0, 1)$ e m è la media desiderata, allora:

$$X = -m \ln(1 - U). \quad (31)$$

Il codice è:

```

1 public double exponential(double m) {
2     return (-m * Math.log(1.0 - rngs.random()));
3 }

```

La distribuzione iperesponenziale a due fasi viene generata per composizione. Un primo stream decide il ramo, mentre due stream distinti generano la variata esponenziale del ramo selezionato:

```

1 rngs.selectStream(streamSelect);
2 double u = rngs.random();
3
4 if (u < p) {
5     rngs.selectStream(streamExp1);
6     return exponential(m1);
7 } else {
8     rngs.selectStream(streamExp2);
9     return exponential(m2);
10 }

```

I parametri p , m_1 e m_2 sono calcolati con la parametrizzazione bilanciata già introdotta nel modello delle specifiche, in modo da ottenere la media e il coefficiente di variazione richiesti. Nel simulatore, arrivi e servizi usano stream differenti per selezione del ramo e generazione della fase esponenziale; questo rende gli scenari replicabili e riduce il rischio di dipendenze spurie tra processi diversi.

7. Verifica del Simulatore

La fase di verifica del simulatore rappresenta il passaggio metodologico fondamentale per certificare la correttezza intrinseca del software sviluppato, garantendo che la traduzione del modello a reti di code all'interno dell'architettura a oggetti dell'applicativo Java sia rigorosa e priva di difetti di programmazione o di interpretazione logica. Il processo di verifica si concentra sull'analisi di coerenza strutturale, accertando che il motore di simulazione a eventi discreti esegua le transizioni del vettore di stato $S(t)$ e la rischedulazione della *Event List* in esatta conformità con le specifiche matematiche e algoritmiche formalizzate nei capitoli precedenti.

A tale scopo, l'intera codebase è stata dotata di una suite sistematica di test automatizzati, ingegnerizzata per operare in regime di integrazione continua, tramite l'utilizzo di strumenti di continuous integration, in modo da isolare precocemente eventuali regressioni funzionali. Il piano di test adotta un approccio modulare e rigidamente strutturato, sottoponendo i singoli componenti core a *corner cases* e scenari limite deterministici per verificarne la stabilità e l'accuratezza numerica.

7.1 Verifica Analitica e Consistenza Statistica

Il primo pilastro della fase di verifica risiede nell'accertare la correttezza intrinseca del motore di simulazione sviluppato, isolando il comportamento computazionale delle singole stazioni per escludere l'interferenza delle retroazioni algoritmiche dei moduli di autoscaling. Questa analisi è orientata a verificare la precisione numerica del generatore di numeri casuali, l'esatta rischedulazione della *Event List* ad ogni transizione asincrona e il rispetto delle leggi di conservazione operative.

Ogni esperimento di verifica è stato costruito partendo da uno scenario con comportamento atteso noto: nel primo caso il riferimento è dato dalle formule chiuse del modello M/M/1, mentre nel secondo caso il controllo avviene tramite una legge di conservazione che deve valere indipendentemente dalla distribuzione dei tempi. L'esito atteso è quindi che le medie campionarie del simulatore cadano all'interno degli intervalli di confidenza dei valori teorici o, nel caso della legge di Little, che lo scarto residuo sia compatibile con la variabilità statistica della run.

Verifica rispetto al Modello M/M/1: Una prima sottomissione di controllo è stata eseguita configurando il simulatore in uno scenario a servitore singolo ad orizzonte infinito con flussi Markoviani, caratterizzati da un tasso medio di ingresso $\lambda = 2.5$ job/s e da un tasso nominale di elaborazione $\mu = 4.0$ job/s. Sotto tali ipotesi deterministiche, le metriche prestazionali a regime stazionario sono ricavabili in forma chiusa dalle equazioni:

$$\text{Utilizzazione } \rho = \frac{\lambda}{\mu} = \frac{2.5}{4.0} = 0.6250 \quad (62.5\%) \quad (32)$$

$$\text{Tempo medio di Risposta } E[T_s] = \frac{1}{\mu - \lambda} = \frac{1}{4.0 - 2.5} \approx 0.6667 \text{ s} \quad (33)$$

$$\text{Numero medio di Job } E[N_s] = \frac{\rho}{1 - \rho} = \frac{0.6250}{1 - 0.6250} \approx 1.6667 \quad (34)$$

Prima di procedere alla run di verifica, le proprietà e l'autocorrelazione di questo specifico carico esponenziale sono state analizzate tramite l'algoritmo del *Batch Means Estimator* formalizzato nei capitoli precedenti. Gli output di questa fase di stima e convergenza sono riassunti nella Tabella 4.

Metodo	Dim. Batch	Num. Batch	Lag-1 ACF	Media Stimata	Half-Width
ACF	1080	4629	0.0063	0.6657 s	0.0025
Dinamico	128	234	0.0274	0.6584 s	0.0292

Tabella 4: Output di calibrazione preliminare del Batch Means Estimator sul tempo di risposta del modello M/M/1.

I risultati della calibrazione hanno confermato la stabilità delle metriche e l'abbattimento della correlazione. Sulla scorta di queste informazioni, al fine di azzerare il transitorio di avvio e restringere ulteriormente l'ampiezza degli intervalli per isolare microscopici bug del software, il test di verifica finale del simulatore è stato configurato applicando un'architettura sovradimensionata ad orizzonte infinito strutturata su 8192 batch da 2048 job ciascuno. Il confronto sistematico tra i valori attesi dalla teoria delle code e le metriche stazionarie campionate empiricamente dal codice Java è raccolto nella Tabella 5.

Metrica	Valore Teorico	Media Campionaria	Half-Width	Intervallo di Conf. 95%
ρ	0.6250	0.6251	0.0004	[0.6247, 0.6256]
$E[T_s]$	0.6667 s	0.6656 s	0.0014	[0.6642, 0.6670]
$E[N_s]$	1.6667	1.6662	0.0039	[1.6622, 1.6701]
X	2.5000 job/s	2.5010 job/s	0.0012	[2.4998, 2.5022]

Tabella 5: Quadro comparativo di verifica tra metriche analitiche M/M/1 ed output statistici del simulatore.

Dato che i parametri analitici esatti ricadono rigorosamente all'interno degli intervalli di confidenza determinati dal simulatore, la logica di avanzamento del tempo e di accodamento del codice Java è da ritenersi verificata. Il comportamento ottenuto è quindi pienamente atteso: in assenza di autoscaling, routing dinamico e soglie di dirottamento, il simulatore deve ridursi a un modello M/M/1 classico e riprodurre gli indici stazionari entro l'errore statistico.

Verifica tramite Legge di Little su $H_2/H_2/1$: Al fine di convalidare la robustezza del motore software in presenza di flussi caratterizzati da un'elevata varianza stocastica, la suite di test ha previsto una sottomissione di controllo operante sotto distribuzioni iperesponziali H_2 sia per gli interarrivi che per i tempi di servizio. Lo scenario, isolato dalle logiche dinamiche di autoscaling, è stato eseguito impostando un coefficiente di variazione $C_V = 4.0$ e strutturando la run di produzione su 512 batch da 16384 job ciascuno (risultati in Tabella 6), garantendo un campionamento statistico ad alta precisione.

Metodo	Dim. Batch	Num. Batch	Lag-1 ACF	Media Stimata	Half-Width
ACF	11970	417	0.0656	1.1970 s	0.0197
Dinamico	65536	32	0.0248	1.1939 s	0.0419

Tabella 6: Output di calibrazione preliminare del Batch Means Estimator sul tempo di risposta del modello $H_2/H_2/1$.

La verifica della correttezza logica del codice si è concentrata sulla validazione della *Legge di Little*, un'identità operativa universale che deve tassativamente sussistere a regime per

qualunque sistema conservativo. Tale controllo funge da stress-test formale per accertare che l'infrastruttura software non introduca derive computazionali latenti, quali la perdita di record o eventi all'interno della *Event List*, anomalie incrementali nei contatori dei job residenti o distorsioni nell'integrazione temporale delle aree necessarie al calcolo delle metriche.

Sfruttando le medie campionarie registrate in modo indipendente dalle rispettive routine del simulatore, l'uguaglianza analitica è stata sottoposta a verifica secondo il bilancio strutturato nella Tabella 7.

Metrica	Valore Campionato	Half-Width	Intervallo di Conf. 95%
Throughput Medio: $\hat{X}$	2.4991 job/s	0.0071	[2.4921, 2.5062]
Tempo di Risposta: $\hat{R}$	1.1975 s	0.0149	[1.1826, 1.2123]
Job nel Sistema: $\hat{N}$	2.9979 job	0.0419	[2.9561, 3.0398]

Tabella 7: Verifica di consistenza operativa tramite bilanciamento asincrono della legge di Little sotto carico iperesponziale $H_2/H_2/1$.

Lo scostamento relativo risultante tra l'aspettativa teorica di conservazione, dato da $\hat{X} \times \hat{R}$, e il valore reale dell'occupazione media rilevato dai contatori fisici del simulatore in $\hat{N}$ si attesta ad un valore del tutto marginale dello 0.17%. Tale discrepanza infinitesima, ampiamente inferiore alle fluttuazioni stocastiche nominali definite dall'half-width dell'intervallo di confidenza al 95% (± 0.0419 per il numero di job nel sistema), convalida formalmente la perfetta coerenza logica della pipeline di accodamento e l'assenza di bug strutturali nell'architettura software del codice Java. Anche in questo caso l'esito è quello atteso: l'elevata variabilità dell'input aumenta la dispersione delle misure, ma non deve alterare le identità operative del sistema.

7.2 Verifica delle Logiche di Routing

La correttezza del modulo di *Load Balancing* e della commutazione dinamica delle rotte è stata sottoposta a un processo di verifica operato tramite simulazioni *Trace-Driven*. Questo approccio consente di analizzare puntualmente le transizioni di stato del router in corrispondenza di sequenze d'ingresso note, confrontando i vettori di carico istantanei con i flussi attesi e accertando la totale assenza di derive logiche nelle politiche di ripartizione.

L'obiettivo di questa verifica non è misurare una prestazione media, ma controllare che ciascuna decisione locale del router coincida con la regola algoritmica implementata. Per questo motivo sono state usate tracce deterministiche, nelle quali istanti di arrivo e tempi di servizio sono noti a priori: ci si attende una rotazione esatta per il Round Robin, una scelta del nodo meno occupato per il Least Loaded, una dispersione non deterministica per il Random e un comportamento intermedio per il Power of Two Choices.

Verifica Funzionale delle Politiche di Distribuzione dei Job: La verifica della correttezza del modulo di routing si è concentrata esclusivamente sull'analisi del comportamento atteso nella ripartizione spaziale e temporale dei job all'interno del pool dei quattro server. A tale scopo, il simulatore è stato alimentato con una traccia d'ingresso deterministica composta da 10 job complessivi, esplicitati dalle rispettive coppie di valori (istante di arrivo, tempo di servizio):

Questo preciso pattern di input consente di verificare la reattività della logica di instradamento sia di fronte a richieste isolate (come nel caso del Job 4), sia durante burst ravvicinati con cadenza di un secondo (Job 1-3, Job 5-7 e Job 8-10). Il primo controllo macroscopico ha

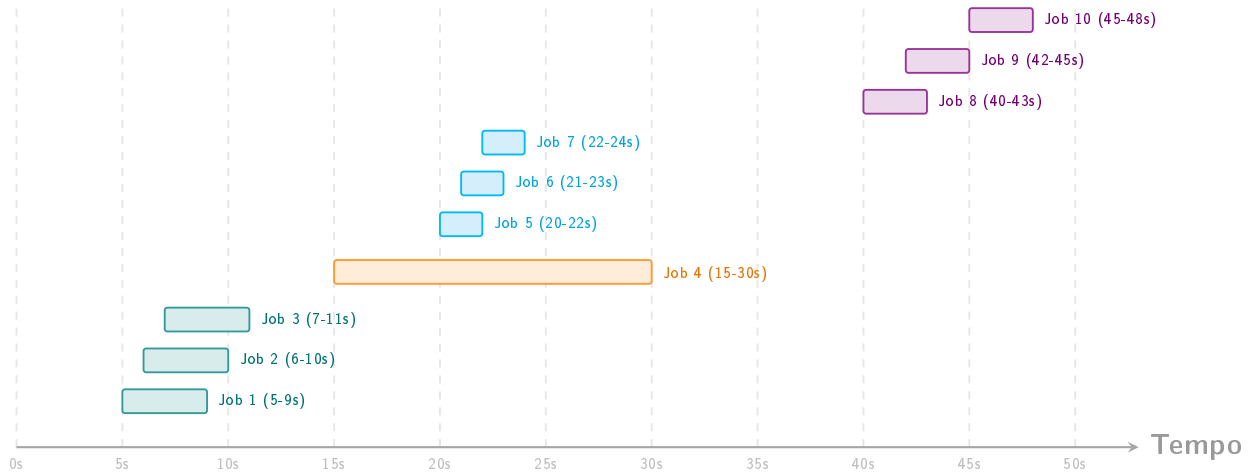


Figura 6: Rappresentazione temporale della traccia deterministica dei job in ingresso.

riscontrato l'esatta conservazione del flusso: per tutti e quattro gli algoritmi esaminati, il numero totale di task elaborati e completati a fine simulazione è pari a 10, confermando l'assenza di perdite di eventi o fallimenti strutturali lungo le rotte del Load Controller.

L'ispezione microscopica dei grafici temporali conferma che il pattern di assegnazione e accodamento dei job risponde fedelmente alle specifiche logiche di ciascuna politica:

- **Round Robin:** Il diagramma in Figura 7 mostra una distribuzione dei job perfettamente ciclica e simmetrica. Ogni nuovo arrivo viene indirizzato sequenzialmente ruotando l'indice dei nodi, verificando l'esatta implementazione del contatore circolare interno del router.
- **Least Loaded:** Al contrario del Round Robin, il grafico in Figura 8 mostra come questa politica non utilizzi l'intero pool in modo circolare, lasciando il Server 4 completamente scarico. Questo comportamento evidenzia l'esatta esecuzione della logica di stato istantanea del controllore: al secondo 15, all'arrivo del Job 4 (caratterizzato da una sosta prolungata di 15 secondi), l'intero pool si trova in stato di quiete e il meccanismo di risoluzione dei pareggi seleziona il primo nodo disponibile a parità di carico minimo, ovvero il Server 1. Nei passi successivi (secondi 20-21), l'algoritmo rileva la saturazione del Server 1 e dirotta i Job 5 e 6 sui Server 2 e 3. Al secondo 22, l'immissione del Job 7 coincide esattamente con il completamento del Job 5 sul Server 2; trovando quest'ultimo nuovamente libero, il Load Controller lo seleziona per via della gerarchia degli indici, escludendo di fatto l'attivazione del Server 4. Ciò convalida la corretta e tempestiva lettura dei contatori di occupazione prima di ogni decisione di routing.
- **Random:** Il grafico in Figura 9 evidenzia la natura totalmente cieca della politica, caratterizzata da un'allocazione asimmetrica e disomogenea. La mancanza di memoria sullo stato delle code provoca collisioni strutturali in cui più job vengono indirizzati contemporaneamente sullo stesso nodo, generando sovrapposizioni temporali con un livello di concorrenza pari a 2 job, mentre altre stazioni del pool permangono temporaneamente inutilizzate.
- **Power of Two Choices:** Il pattern osservato in Figura 10 mostra un comportamento intermedio coerente con le specifiche della tecnica. Pur mantenendo una componente stocastica nell'assegnazione dei job, la parziale conoscenza dello stato, ottenuta campionando

due server casuali, attenua la gravità delle collisioni rispetto al Random puro, confermando che il codice esegue correttamente la comparazione algebrica delle code della coppia selezionata prima di procedere al dirottamento del task.

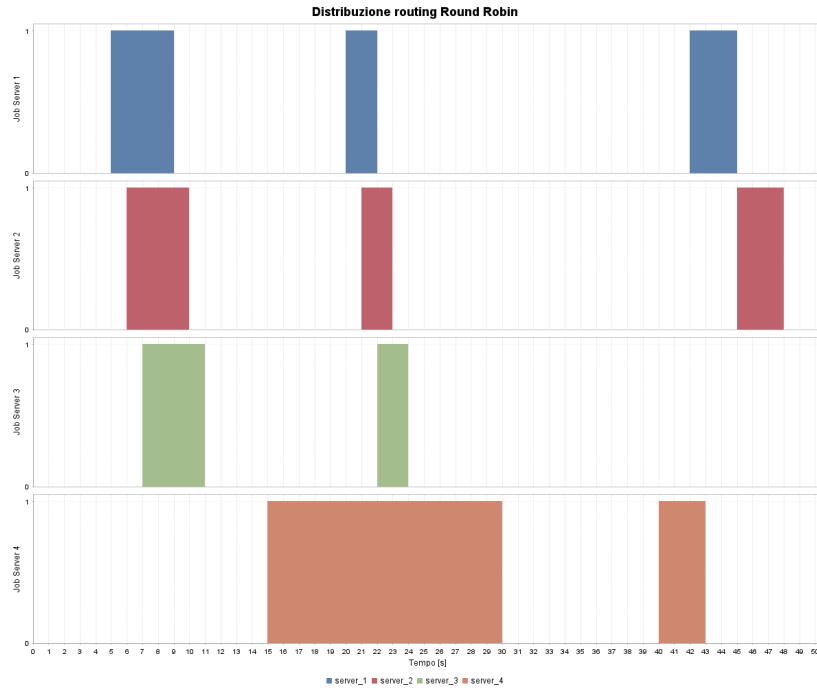


Figura 7: Distribuzione tempore dei job sotto politica Round Robin.

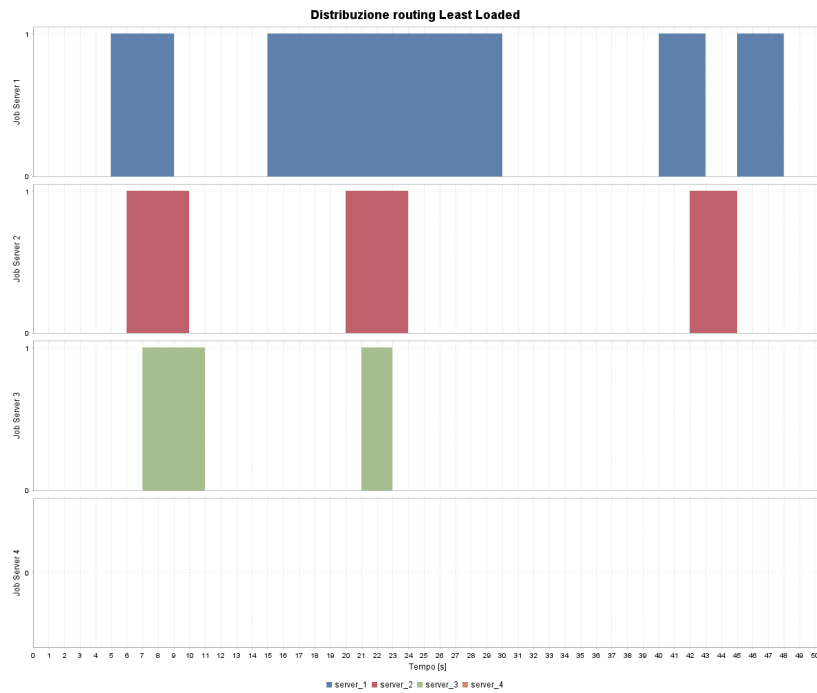


Figura 8: Saturazione dei server sotto politica Least Loaded.

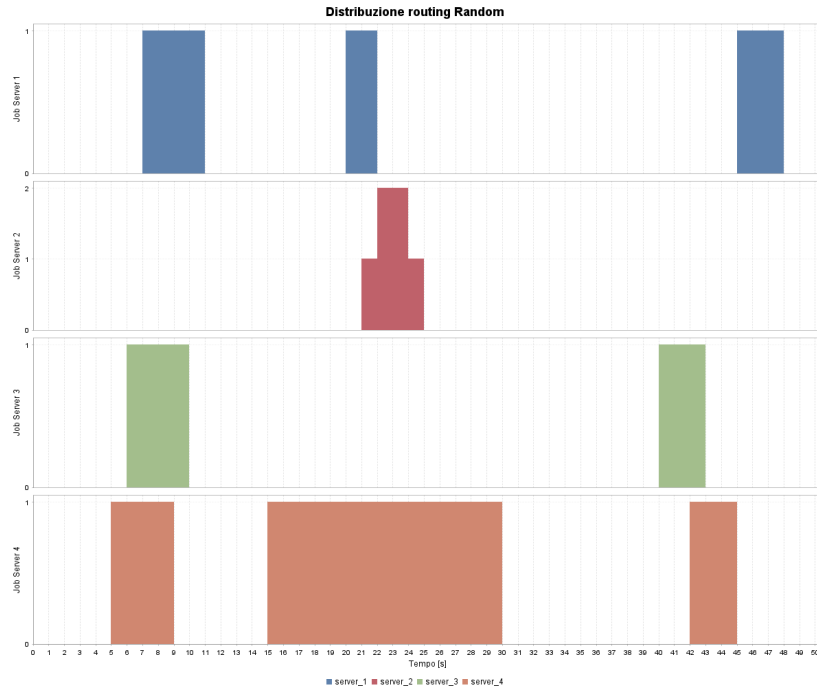


Figura 9: Assegnazione stocastica dei task tramite politica Random.

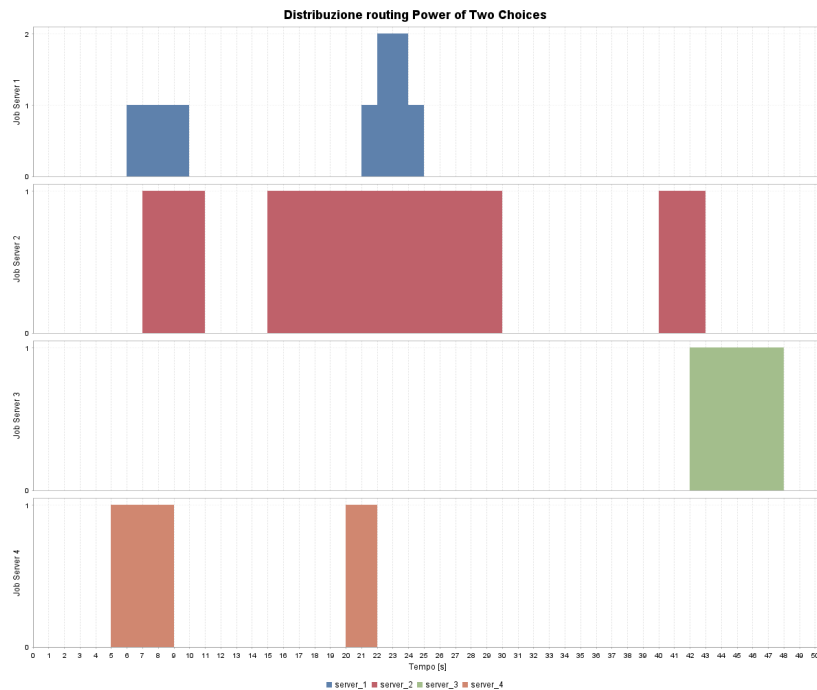


Figura 10: Bilanciamento del carico con tecnica Power of Two Choices.

Verifica del Meccanismo di Dirottamento verso lo Spike Server: Per convalidare la logica di commutazione d'emergenza del router, il modulo è stato sottoposto a uno stress-test deterministico basato su una traccia d'ingresso ad andamento alternato. L'ambiente di verifica viene isolato configurando un cluster minimale composto da un singolo Web Server di linea assistito dallo Spike Server. I parametri operativi del controllore di instradamento sono stati configurati impostando rigidamente la soglia critica a $SI_{max} = 5$ job concorrenti.

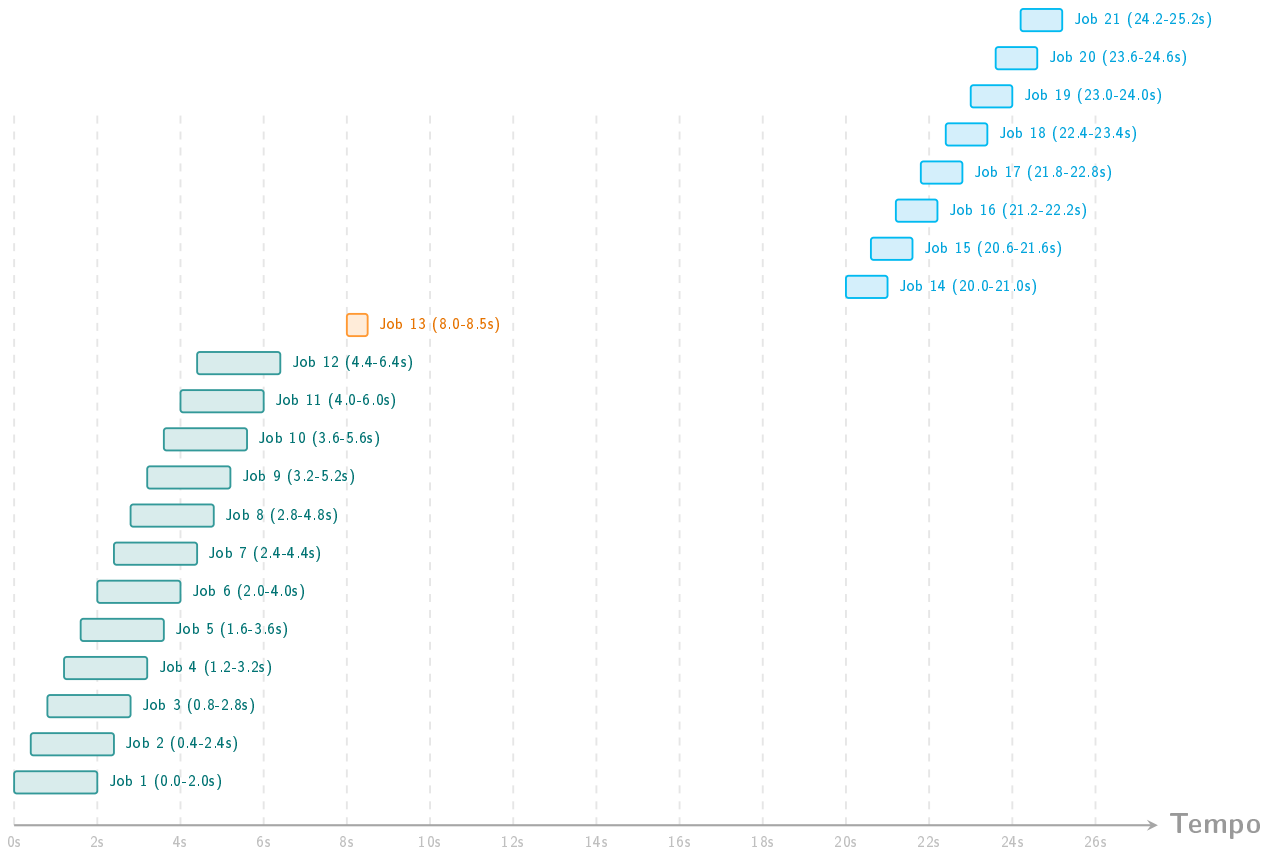


Figura 11: Rappresentazione temporale della traccia stocastica a dente di sega per la verifica del dirottamento sullo Spike Server.

L'andamento della dinamica dei flussi, catturato dettagliatamente nella Figura 12, evidenzia la correttezza matematica delle transizioni di stato del controllore:

- **Fase Ordinaria:** Nella finestra temporale iniziale con $t \in [0, 1.6]$ s, l'intera massa dei job in ingresso viene assorbita dal Web Server di linea, il cui contatore interno cresce linearmente ad ogni arrivo.
- **Innesco dello Spike Mode:** All'istante esatto in cui il carico locale tocca il valore limite di 5 job concorrenti, il router scatta istantaneamente in modalità di dirottamento. Come visibile dall'emergere dell'area rosa nel grafico, tutti i successivi job del burst vengono veicolati sulla coda dello Spike Server, congelando l'accumulo sul server principale e forzando l'elaborazione distribuita.
- **Rilascio e Svuotamento:** Al cessare del transitorio di picco, lo Spike Server smaltisce i job dirottati operando al massimo delle proprie capacità stazionarie.

I report analitici estratti a completamento della run documentano che, a fronte di un tempo di risposta medio globale di 6.9368 secondi e un carico di sistema pari a 5.3953 job, il modulo ha deviato con precisione chirurgica un totale di esattamente 8.0000 job verso l'unità ausiliaria. Questo sversamento controllato ha determinato un'utilizzazione finale dello Spike Server pari esattamente a 0.5000, confermando la perfetta implementazione del ciclo di dirottamento e l'immunità del codice da blocchi o perdite di task durante le transizioni di instradamento. Il risultato è conforme al comportamento atteso: il percorso ausiliario resta inattivo finché il Web Server non raggiunge la soglia, entra in funzione solo per il traffico eccedente e si svuota senza interrompere i job già assegnati.

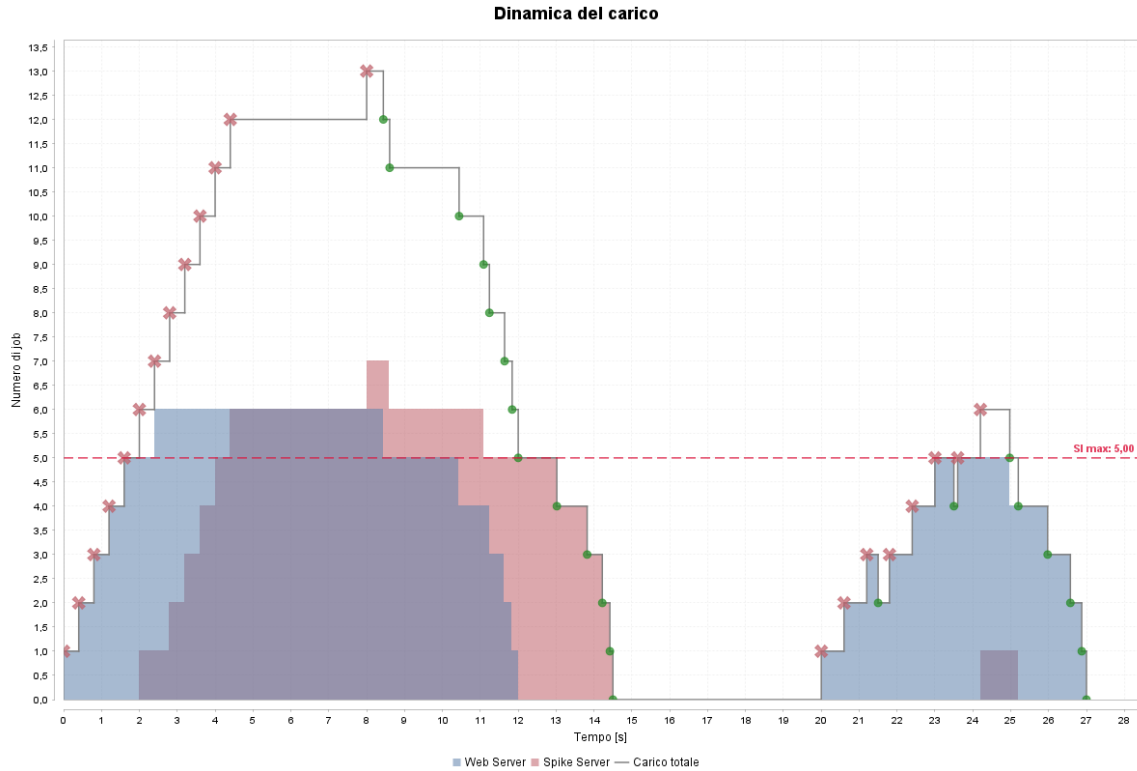


Figura 12: Evoluzione dinamica del carico e attivazione della Dedicated Spike Unit al superamento della soglia SI Max.

7.3 Verifica dei Meccanismi di Scaling

L'ultima fase del processo di verifica ha riguardato la verifica funzionale delle logiche di auto-scaling cooperativo, accertando la capacità del sistema di espandere o contrarre dinamicamente la propria capacità elaborativa al variare del carico di lavoro imposto.

Anche in questo caso l'esperimento è stato costruito in forma deterministica: l'obiettivo è verificare che le azioni di *Scale Out*, *Scale In*, *Speed Up* e *Speed Down* vengano generate solo quando le metriche finestrate attraversano le rispettive soglie e nel rispetto del *Cooldown*. Il risultato atteso è quindi una corrispondenza puntuale tra istanti di superamento soglia, azioni registrate nei log e andamento dei grafici temporali.

Scaling Orizzontale: Per verificare l'adattamento del numero di server attivi guidato dal tempo di risposta, il simulatore è stato configurato con una capacità minima di 1 nodo, un tetto massimo di 4 nodi e un tempo di *Cooldown* di 4.0 secondi. La suite ha eseguito una sottomissione alimentata dalla seguente traccia deterministica programmata di 14 job complessivi (Figura 13).

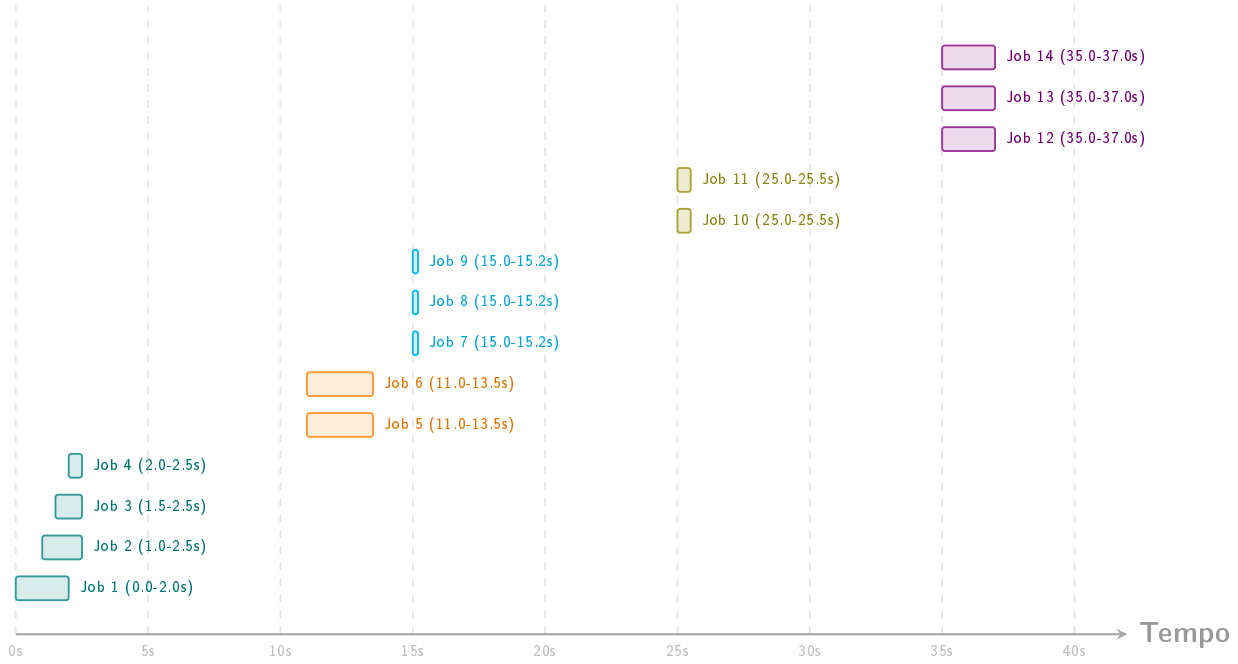


Figura 13: Rappresentazione temporale della traccia deterministica dei job in ingresso per la verifica del modulo di Scaling Orizzontale.

Le soglie di attuazione dell'algoritmo di controllo sono state impostate a $T_{out} = 2.0$ s per l'innescio dello *Scale Out* e a $T_{in} = 0.5$ s per lo *Scale In*.

L'analisi dell'andamento temporale documentato nella Figura 14 convalida formalmente l'esattezza algoritmica del modulo:

- Dinamica di Scale Out:** Al secondo $t = 4.0$, l'accumulo di job porta il tempo medio di risposta nella Moving Window a superare la soglia critica di 2.0 s (toccando quota 2.15 s). Il sistema reagisce istantaneamente incrementando i server attivi da 1 a 2 e avviando il timer di *Cooldown* di 4.0 s. Poiché il carico permane elevato, alla scadenza di ciascun ciclo di blocco si osservano i successivi scatti incrementali a quota 3 server (al secondo $t = 8.0$) e a quota 4 server (al secondo $t = 13.5$), azzerando la saturazione. Un'ultima azione isolata di espansione a 3 server si ripresenta correttamente al secondo $t = 39.0$ per mitigare l'ultimo burst, totalizzando l'esatto valore nominale di 4 azioni di *Scale Out* registrato nei log di report.
- Dinamica di Scale In e Draining:** Al secondo $t = 15.0$, l'immissione di job molto rapidi abbatte drasticamente il tempo medio di risposta sotto la soglia di 0.5 s. Non appena il *Cooldown* residuo si azzerava, l'analizzatore comanda una contrazione della flotta, riducendo i server da 4 a 3 (all'istante $t = 25.5$) e successivamente da 3 a 2 (all'istante $t = 29.5$), quantificando le 2 azioni di *Scale In* complessive presenti nel riepilogo statistico. La correttezza del meccanismo di *draining* trova riscontro diretto nell'andamento dei completamenti: il server rimosso cessa l'accettazione di nuovi task ma non interrompe le elaborazioni in corso, azzerando la sua coda interna in modo conservativo senza alcuna perdita di eventi.

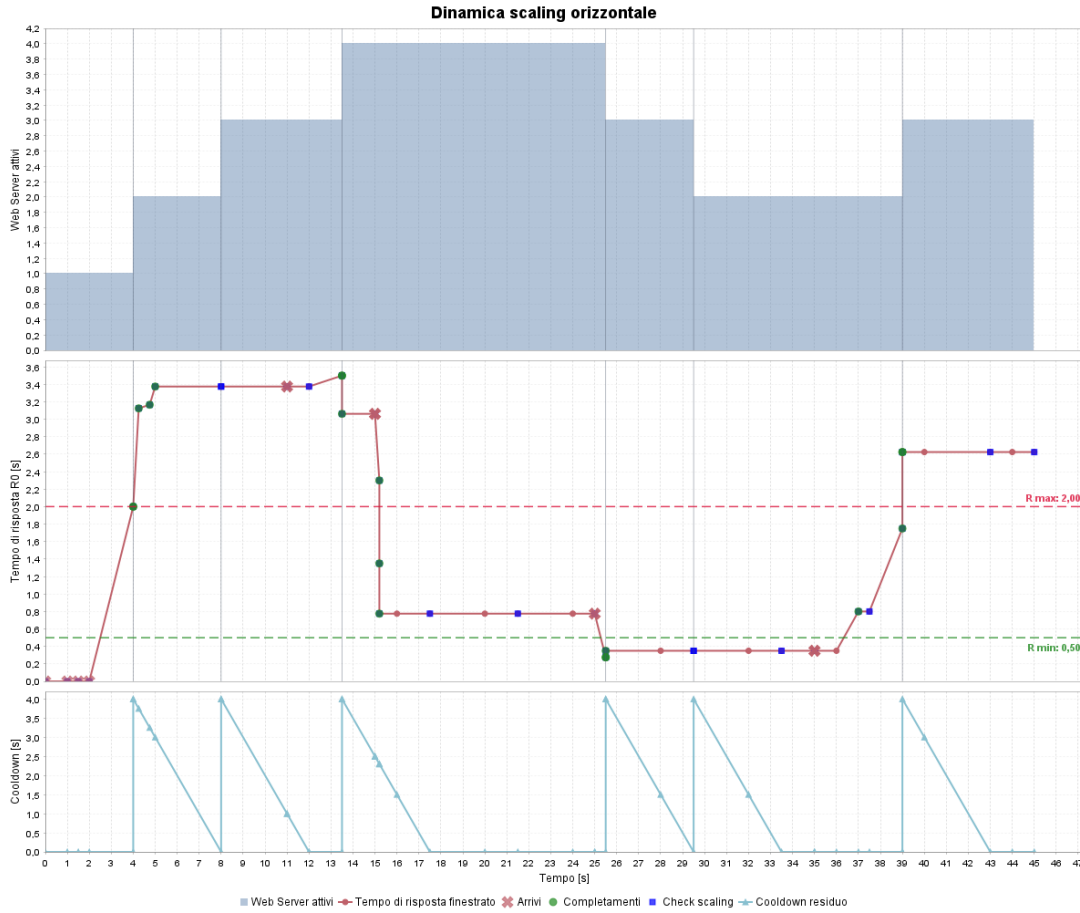


Figura 14: Evoluzione temporale dello Scaling Orizzontale: correlazione tra tempo di risposta finestrato, azioni di allocazione e vincolo di Cooldown.

Scaling Verticale: La validazione delle logiche di allocazione delle risorse hardware si è concentrata sulla riconfigurazione dinamica della capacità di calcolo dello Spike Server. In questo scenario, il sistema opera su un singolo server di linea e dirotta i job in eccesso sull'unità di picco ausiliaria. Le soglie di reazione per la modulazione della frequenza core sono state configurate a un valore limite superiore di 3 job concorrenti in coda e a una soglia inferiore di 1 job per il ripristino ordinario. Il timer di *Cooldown* interno del regolatore è stato ridotto a 0.3 secondi per garantire reattività immediata. La traccia di stress-test iniettata nel sistema prevede due flussi programmati distinti (mostrati in Figura 15).

I risultati della simulazione, illustrati graficamente nella Figura 16, dimostrano una perfetta coincidenza tra il comportamento atteso e le transizioni osservate. Al secondo $t = 2.0$, il cumulo di task all'interno dello Spike Server supera la soglia critica di 3 job attivi. Il controllore verticale interviene senza ritardi temporali registrando 1 azione di *Speed Up*, la quale innalza istantaneamente il fattore di capacità elaborativa dell'infrastruttura dal valore base di 4.0 a un livello di picco pari a 8.0. Grazie a questa accelerazione controllata, la coda viene smaltita rapidamente: al secondo $t = 4.4$, il numero di job residenti scende sotto la soglia minima di 1. Il software applica quindi l'esatta transizione duale di *Speed Down*, riportando il fattore di capacità al valore nominale di 4.0.

Il perfetto confinamento di queste transizioni all'interno della finestra di massimo carico giustifica il valore medio complessivo del fattore di capacità elaborativa registrato nel report finale, attestatosi a 4.3840. Il bilancio di fine simulazione conferma l'esecuzione di esattamente

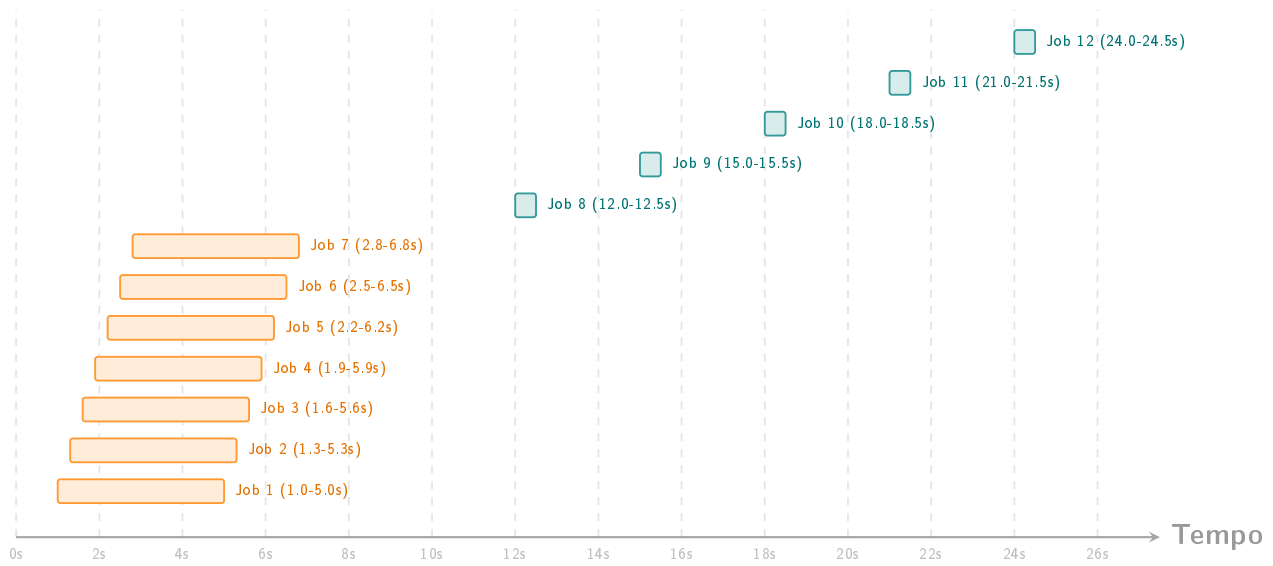


Figura 15: Rappresentazione temporale della traccia deterministica dei job in ingresso per la verifica del modulo di Scaling Verticale.

1 azione di Speed Up e 1 azione di Speed Down, validando la totale assenza di fenomeni di instabilità o derive computazionali nelle routine di controllo verticale.

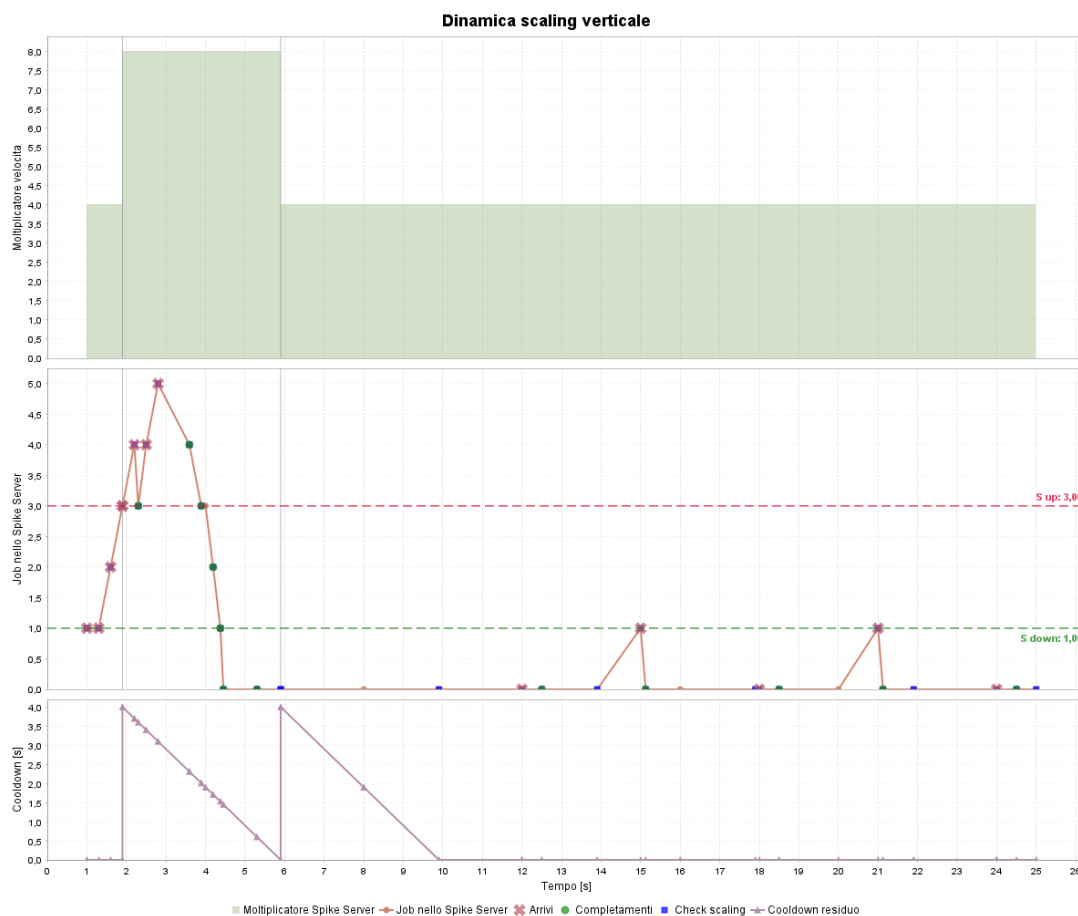


Figura 16: Dinamica dello Scaling Verticale: modulazione istantanea del fattore di capacità elaborativa in funzione del numero di job residenti nella coda dello Spike Server.

8. Validazione del Sistema

La validazione rappresenta il passaggio successivo alla verifica: non si limita ad accertare che il codice implementi correttamente le transizioni previste dal modello, ma controlla che le misure prodotte dal simulatore siano compatibili con quelle ottenute da un ambiente di riferimento indipendente. Per questo motivo i risultati principali sono stati confrontati con il simulatore fornito dal libro di testo, ovvero *Java Modelling Tools - JSIM* [3], riproducendo gli stessi scenari di carico, la stessa disciplina di servizio e le stesse ipotesi di assenza di autoscaling.

La validazione è stata condotta in orizzonte infinito, con misure stazionarie e intervalli di confidenza sulle metriche aggregate. Nel simulatore sviluppato, le stime sono state ottenute con il metodo dei *Batch Means*; in JSIM sono state invece usate le misure statistiche native del tool, configurate con seed fisso e criterio di arresto basato sul numero massimo di campioni analizzati. L'obiettivo del confronto non è dunque riprodurre la stessa sequenza di eventi elementari, ma verificare che due implementazioni indipendenti dello stesso sistema convergano agli stessi indici prestazionali entro la variabilità statistica attesa.

Per ciascun caso di validazione è stato quindi definito un comportamento atteso prima dell'esecuzione: sovrapposizione degli intervalli di confidenza per la stazione singola, e scarti contenuti per l'architettura completa, dove JSIM richiede una rappresentazione semplificata del router a soglia. Eventuali differenze residue non vengono interpretate come errore se restano coerenti con le diverse astrazioni modellistiche e con l'ampiezza degli intervalli statistici.

8.1 Validazione della Stazione Singola Iperesponenziale

Il primo scenario di validazione isola una singola stazione *Processor Sharing*, eliminando ogni effetto dovuto al routing, allo Spike Server e agli algoritmi di scaling. La rete JSIM di riferimento è composta da una sorgente aperta, una coda a capacità illimitata e un sink finale. Sia gli interarrivi sia i tempi di servizio seguono una distribuzione iperesponenziale a due rami, con tasso medio di arrivo $\lambda = 2.5$ job/s, tasso medio di servizio $\mu = 4.0$ job/s e coefficiente di variazione $C_V = 4.0$.

La parametrizzazione JSIM della distribuzione H_2 è stata impostata con probabilità di ramo $p = 0.030776$; per gli interarrivi sono stati usati $\lambda_1 = 0.15388$ e $\lambda_2 = 4.84612$, mentre per i servizi $\lambda_1 = 0.24621$ e $\lambda_2 = 7.75379$. Il modello JSIM è stato eseguito con seed 123456789, stop statistico disabilitato, 100000000 campioni massimi e intervalli di confidenza al 99% con errore relativo massimo pari a 0.03. Nel simulatore Java è stata usata una run a Batch Means da 512 batch di 16384 job, con warm-up nullo, coerentemente con la configurazione già impiegata nella verifica statistica.

Prima del confronto diretto, la dimensione dei batch è stata stimata tramite l'algoritmo di *Batch Means Estimator*. L'output, riportato in Tabella 8, mostra che il metodo dinamico raggiunge simultaneamente indipendenza e precisione relativa, mentre il metodo basato su ACF produce una struttura più ampia e conservativa.

Metodo	Dim. Batch	Num. Batch	Lag-1 ACF	Media Stimata	Half-Width
ACF	11970	417	0.0656	1.1970 s	0.0197
Dinamico	65536	32	0.0248	1.1939 s	0.0419

Tabella 8: Calibrazione del Batch Means Estimator per la validazione del modello $H_2/H_2/1$.



Figura 17: Risultati JSIM per la validazione della stazione singola iperesponenziale.

Il confronto numerico è riassunto nella Tabella 9. Il valore atteso, in questo scenario, è una coincidenza quasi diretta delle metriche, poiché entrambi i simulatori rappresentano la stessa singola stazione senza logiche di controllo aggiuntive. Le metriche ottenute dai due simulatori risultano infatti sostanzialmente coincidenti: il tempo medio di risposta differisce dello 0.80%, il numero medio di job dello 0.67%, il throughput dello 0.02% e l'utilizzazione dello 0.11%. Inoltre, gli intervalli di confidenza delle due campagne si sovrappongono per tutte le metriche osservate, confermando che la dinamica di servizio, l'integrazione temporale delle aree e il calcolo degli indici a regime sono coerenti con il simulatore di riferimento.

Metrica	Simulatore Java	CI Java 95%	JSIM	CI JSIM 99%
$E[T_s]$	1.1975 s	[1.1826, 1.2123]	1.2071 s	[1.2004, 1.2137]
$E[N_s]$	2.9979	[2.9561, 3.0398]	3.0182	[2.9956, 3.0408]
X	2.4991 job/s	[2.4921, 2.5062]	2.4997 job/s	[2.4966, 2.5028]
ρ	0.6240	[0.6215, 0.6265]	0.6247	[0.6232, 0.6262]

Tabella 9: Confronto di validazione tra simulatore Java e JSIM sul modello $H_2/H_2/1$.

8.2 Validazione dell'Architettura Web Server e Spike Server

Il secondo scenario estende la validazione alla struttura architetturale composta da un Web Server ordinario e da uno Spike Server di supporto. In questo caso l'obiettivo non è verificare una singola stazione, ma accertare che il meccanismo aggregato di instradamento verso il livello ausiliario produca lo stesso ordine di grandezza prestazionale del modello JSIM di riferimento.

Per isolare il solo comportamento del routing, entrambi gli autoscaler sono stati disabilitati: il numero di Web Server resta costante, il moltiplicatore dello Spike Server non varia nel tempo e non vengono eseguite azioni di *Scale Out*, *Scale In*, *Scale Up* o *Scale Down*. Il traffico in ingresso rimane iperesponenziale, con $\lambda = 2.5$ job/s e $C_V = 4.0$; i servizi delle stazioni sono modellati con lo stesso schema H_2 e tasso medio $\mu = 4.0$ job/s. In JSIM il comportamento a soglia del router è stato emulato tramite una rete ausiliaria di token e transizioni: la classe chiusa *maxReq_Link*, inizializzata con 10 token nella place *P_Tokens*, permette di riprodurre il vincolo di capacità del percorso ordinario e di inviare il traffico eccedente verso lo Spike Server.

Questa costruzione rappresenta una semplificazione esplicita rispetto alla *ThresholdSpike-ServerRoutingStrategy* implementata nel simulatore Java: JSIM non esegue lo stesso controllo oggetto-per-oggetto sul valore istantaneo di *SI* del Web Server, ma ne riproduce l'effetto macroscopico mediante abilitazione e inibizione delle transizioni. Il confronto va quindi interpretato come validazione delle metriche globali del sistema a due livelli, non come equivalenza perfetta delle singole decisioni di routing.

Anche per questo scenario è stata eseguita una stima preliminare dei Batch Means sul tempo di risposta globale. La Tabella 10 mostra che il metodo ACF suggerisce batch da 2360 job, mentre il metodo dinamico converge con batch da 512 job dopo avere soddisfatto il vincolo di autocorrelazione e quello di precisione relativa.

Metodo	Dim. Batch	Num. Batch	Lag-1 ACF	Media Stimata	Half-Width
ACF	2360	2118	0.0628	0.9161 s	0.0064
Dinamico	512	195	-0.0406	0.9420 s	0.0463

Tabella 10: Calibrazione del Batch Means Estimator per la validazione dell'architettura Web Server e Spike Server.

La campagna Java definitiva è stata configurata con 4096 batch da 4096 job. I risultati in Tabella 11 mostrano una buona aderenza tra le due implementazioni indipendenti. Il throughput, indice particolarmente sensibile alla corretta conservazione del flusso, differisce solo dello 0.34% e gli intervalli di confidenza si sovrappongono. Anche il numero medio di job nel sistema mantiene uno scarto contenuto, pari a circa 1.31%, con sovrapposizione tra gli intervalli. Il tempo di risposta medio del simulatore Java risulta leggermente più alto di quello JSIM (0.9157 s contro 0.9043 s), con uno scostamento relativo pari a circa 1.26%: tale differenza è coerente con la semplificazione adottata nel modello JSIM per approssimare il router a soglia e con la diversa logica di arresto statistico usata dai due strumenti.

Il report Java conferma inoltre che lo scenario è rimasto effettivamente statico rispetto agli autoscaler: tutte le azioni di *Scale Out*, *Scale In*, *Scale Up* e *Scale Down* hanno media nulla, mentre il numero medio di server attivi è pari a 1.0000. L'utilizzazione media dello Spike Server risulta pari a 0.0225, segnale che la risorsa ausiliaria viene attivata solo durante gli episodi di congestione locale. Questa evidenza è coerente con la finalità della validazione: il livello di picco non deve alterare il carico ordinario, ma deve intervenire come percorso di sfogo quando la soglia del Web Server viene raggiunta. Il leggero scostamento sul tempo di risposta non è

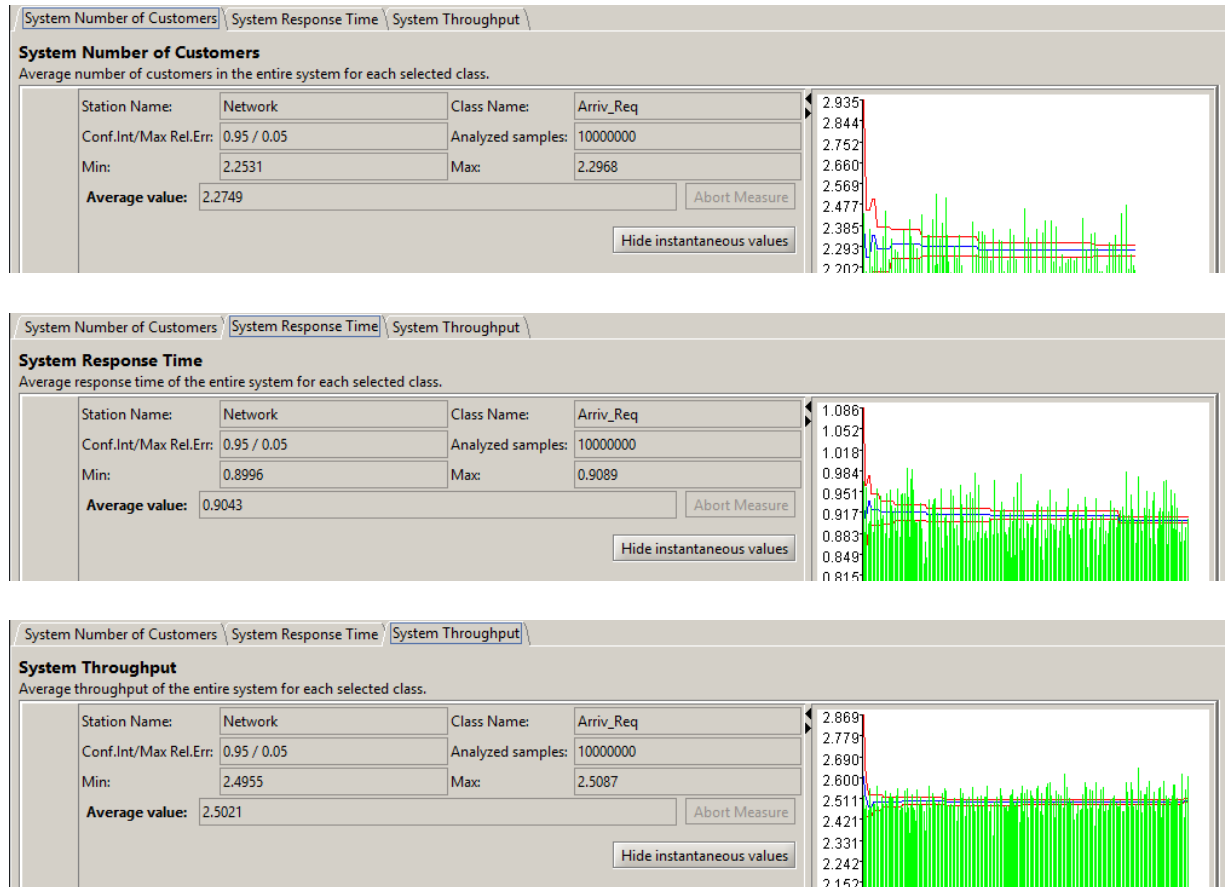


Figura 18: Risultati JSIM per la validazione dell'architettura Web Server e Spike Server.

Metrica	Simulatore Java	CI Java 95%	JSIM	CI JSIM 95%
$E[T_s]$	0.9157 s	[0.9121, 0.9194]	0.9043 s	[0.8996, 0.9089]
$E[N_s]$	2.3048	[2.2933, 2.3162]	2.2749	[2.2531, 2.2968]
X	2.5106 job/s	[2.5057, 2.5156]	2.5021 job/s	[2.4955, 2.5087]

Tabella 11: Confronto di validazione tra simulatore Java e JSIM sull'architettura Web Server e Spike Server.

quindi un risultato imprevisto, ma l'effetto atteso della modellazione approssimata della soglia in JSIM rispetto alla decisione evento-per-evento implementata nel simulatore Java.

Nel complesso, la validazione esterna mostra che il simulatore implementato replica correttamente sia il comportamento di una stazione singola ad alta variabilità, sia l'effetto prestazionale aggregato dell'architettura a due livelli. Le differenze residue sono contenute e spiegabili dalle diverse astrazioni richieste per rappresentare in JSIM la strategia di routing a soglia; pertanto il modello sviluppato può essere considerato adeguato per le successive campagne di analisi e dimensionamento.

9. Risultati e Analisi

In questo capitolo vengono presentati e discussi i risultati ottenuti dalle diverse campagne di simulazione, evidenziando il raggiungimento degli obiettivi prefissati e analizzando le prestazioni del sistema sotto diversi scenari operativi. L'esposizione segue lo stesso percorso sperimentale adottato nel progetto: prima vengono fissati i parametri di dimensionamento, poi viene valutata l'architettura completa in termini di costo, dinamica transitoria e robustezza rispetto alla variabilità del carico, infine viene discusso il miglioramento del meccanismo di dirottamento.

9.1 Dimensionamento

Ottimizzazione delle Politiche di Instradamento L'ottimizzazione delle politiche di instradamento è stata condotta tramite la classe `RoutingPolicyObjective`, isolando il solo effetto del bilanciamento del carico sul cluster ordinario. Lo scenario sperimentale è stato configurato con 4 Web Server statici, Spike Server abilitato, assenza completa di meccanismi di scaling orizzontale e verticale, carico iperesponenziale H_2/H_2 con $\lambda = 5.0$ job/s, $\mu = 4.0$ job/s per singolo server e coefficiente di variazione pari a $C_V = 4.0$ sia per gli interarrivi sia per i tempi di servizio. La stima preliminare dei Batch Means è stata effettuata su un'architettura rappresentativa dello stesso esperimento; il metodo ACF ha individuato un cutoff pari a 284, con batch da 2840 job, 1760 batch e lag-1 ACF pari a 0.0453, mentre il metodo dinamico ha raggiunto lag-1 ACF pari a -0.0394 con batch da 131072 job e 32 batch. Il candidato selezionato è stato quello derivato dal metodo ACF, poiché caratterizzato dal maggior numero di osservazioni utili e dalla minima *Half-Width* sul tempo di risposta. Come da criterio metodologico adottato, i parametri sono stati quindi arrotondati alla successiva potenza di 2, ottenendo la configurazione finale di $k = 4096$ batch da $b = 2048$ job ciascuno.

Il confronto ha coinvolto le quattro strategie *Round Robin*, *Random*, *Least Loaded* e *Power of Two Choices*. I risultati quantitativi, riportati in Tabella 12, mostrano una gerarchia netta e stabile. La politica **Least Loaded** risulta la migliore in assoluto, con un tempo medio di risposta pari a 0.2843 s, una *Half-Width* di appena 0.0011 s e una deviazione standard pari a 0.0243 s; rispetto a *Round Robin* riduce la latenza media di circa il 24.7%, mentre il guadagno sale a circa il 33.3% rispetto alla politica *Random*. La tecnica *Power of Two Choices* si colloca in seconda posizione con $R_0 = 0.3147$ s, confermando di essere un compromesso molto efficace tra qualità del bilanciamento e costo informativo, mentre *Random* risulta la politica meno efficiente a causa dell'assenza di qualunque meccanismo di correzione delle collisioni locali.

Il risultato osservato è coerente con l'aspettativa teorica. In uno scenario con server omogenei e carico altamente variabile, la politica che osserva direttamente il numero di job residenti deve ridurre più efficacemente la congestione locale; al contrario, una politica cieca come *Random* può produrre collisioni temporanee anche quando esiste capacità libera altrove. Non emergono quindi anomalie sperimentali: la classifica ottenuta riflette il diverso livello informativo disponibile al router.

L'evidenza grafica in Figura 19 conferma che il vantaggio di *Least Loaded* non si limita a un semplice abbassamento della media, ma si traduce anche in una sensibile contrazione della dispersione statistica. Questo punto è particolarmente rilevante alla luce della disciplina di scheduling adottata nei server, di tipo *Processor Sharing*: tale meccanismo tende infatti a ripartire equamente la capacità di servizio tra i job concorrenti, attenuando l'iniquità individuale rispetto a una coda FCFS, ma non elimina affatto il costo della congestione locale. Quando una politica di routing concentra troppi job sullo stesso nodo, ogni nuovo arrivo entra immediatamente in competizione con un numero maggiore di richieste già residenti e il rallentamento si propaga a tutti i task presenti su quel server. Per questa ragione, anche in presenza di *Processor*

Politica di Routing	R_0 Medio [s]	HW [s]	CI 95% R_0 [s]	StdDev [s]
Round Robin	0.3776	0.0019	[0.3757, 0.3795]	0.0441
Least Loaded	0.2843	0.0011	[0.2832, 0.2854]	0.0243
Random	0.4262	0.0020	[0.4242, 0.4282]	0.0465
Power of Two Choices	0.3146	0.0012	[0.3134, 0.3158]	0.0280

Tabella 12: Confronto prestazionale tra le politiche di instradamento sul cluster statico a 4 Web Server.

Sharing, la qualità dell'informazione usata dal router resta determinante: *Least Loaded* minimizza la concorrenza locale osservando lo stato istantaneo del sistema, *Power of Two Choices* ne cattura una versione approssimata ma già molto efficace, mentre *Round Robin* e soprattutto *Random* subiscono più facilmente sbilanciamenti transitori che si riflettono in una maggiore varianza del tempo di risposta.

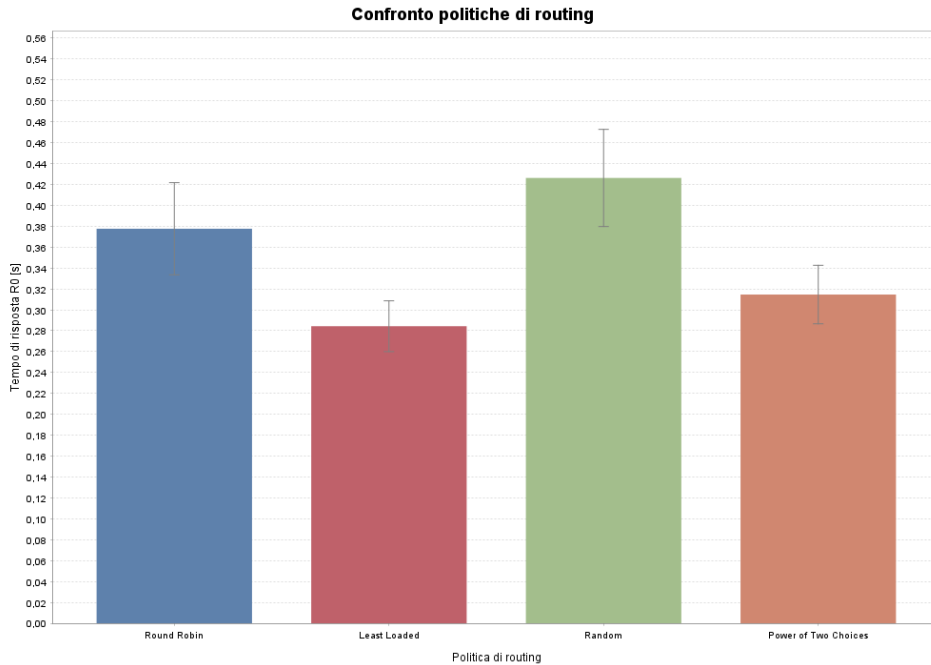


Figura 19: Confronto grafico delle prestazioni delle politiche di routing in termini di tempo medio di risposta e dispersione statistica.

Nel complesso, questa campagna sperimentale giustifica la scelta di *Least Loaded* come politica di riferimento per le successive analisi di dimensionamento: essa è la strategia che sfrutta meglio la parallelizzazione del cluster, riduce la probabilità di micro-congestioni sui singoli nodi e stabilizza maggiormente il comportamento del sistema sotto carico fortemente variabile. *Power of Two Choices* rimane tuttavia una valida alternativa qualora si voglia ridurre l'overhead di monitoraggio, accettando un degrado prestazionale contenuto.

Ottimizzazione della Soglia di Dirottamento SI_{max} La scelta della soglia di dirottamento verso lo Spike Server è stata condotta tramite la classe `SiMaxEstimationObjective`, con l'obiettivo di individuare il massimo valore di SI_{max} compatibile con il vincolo prestazionale imposto dallo SLA. Lo scenario di prova è stato mantenuto volutamente stazionario, con un singolo Web Server ordinario, Spike Server attivo, scaling disabilitato e carico iperesponenziale

H_2/H_2 ad alta variabilità. La campagna *what-if* ha esplorato valori di SI_{max} compresi tra 10 e 150, osservando congiuntamente il tempo medio di risposta R_0 , la quota di job dirottati e l'utilizzazione media dello Spike Server.

Anche per questa campagna è stata condotta una stima preliminare dei Batch Means. Il metodo ACF ha rilevato un cutoff pari a 60, con batch da 600 job, 8333 batch e lag-1 ACF pari a 0.0683; il metodo dinamico ha invece prodotto batch da 2048 job, 48 batch e lag-1 ACF pari a 0.0020. La configurazione finale è stata scelta in modo conservativo, privilegiando la stabilità degli intervalli di confidenza nella regione prossima alla soglia SLA.

I risultati numerici, sintetizzati nella Tabella 13, mostrano un comportamento molto netto. All'aumentare di SI_{max} , il router tollera livelli di congestione locale sempre più elevati prima di attivare il percorso ausiliario; di conseguenza la percentuale di job deviati e l'utilizzazione dello Spike Server decrescono rapidamente nella prima parte della curva, mentre il tempo medio di risposta cresce in modo monotono. Questo trade-off è atteso: soglie più basse proteggono in maniera aggressiva il server ordinario, ma trasferiscono una porzione maggiore del carico alla risorsa di picco; soglie più alte, al contrario, riducono il ricorso allo Spike Server ma espongono il nodo principale a fenomeni di saturazione prolungata, che in presenza di scheduling *Processor Sharing* penalizzano simultaneamente tutti i job residenti.

SI_{max}	R_0	Medio [s]	HW [s]	CI 95% R_0 [s]	Job Deviatl [%]	Util. Spike [%]
10	1.4130	0.0050	[1.4080, 1.4181]	30.06	38.73	
20	2.5786	0.0079	[2.5707, 2.5865]	23.65	30.80	
30	3.9709	0.0124	[3.9585, 3.9833]	21.34	27.90	
40	5.5279	0.0172	[5.5107, 5.5451]	20.38	26.74	
50	7.2009	0.0216	[7.1793, 7.2225]	19.98	26.29	
60	8.9588	0.0259	[8.9329, 8.9847]	19.85	26.13	
70	10.7680	0.0303	[10.7377, 10.7982]	19.83	26.06	
80	12.6091	0.0346	[12.5745, 12.6437]	19.83	26.05	
100	16.3577	0.0435	[16.3142, 16.4011]	19.85	26.05	
150	25.9110	0.0676	[25.8434, 25.9786]	19.86	26.10	

Tabella 13: Analisi di sensitività della soglia SI_{max} in regime stazionario.

La Figura 20 rende ancora più evidente la struttura del compromesso. Il comportamento atteso era una crescita monotona del tempo di risposta al crescere della soglia, accompagnata da una riduzione progressiva del ricorso allo Spike Server; i risultati confermano questa lettura. Il tempo medio di risposta cresce in modo pressoché lineare fino ad avvicinarsi alla frontiera dello SLA, mentre la curva di utilizzazione dello Spike Server subisce una rapida discesa iniziale e poi si assesta su un plateau quasi stazionario attorno al 25%. Questo asintoto è importante dal punto di vista progettuale: oltre un certo valore della soglia, il sistema non ottiene più un risparmio significativo nell'uso della risorsa di picco, ma continua invece a pagare un costo crescente in termini di latenza.

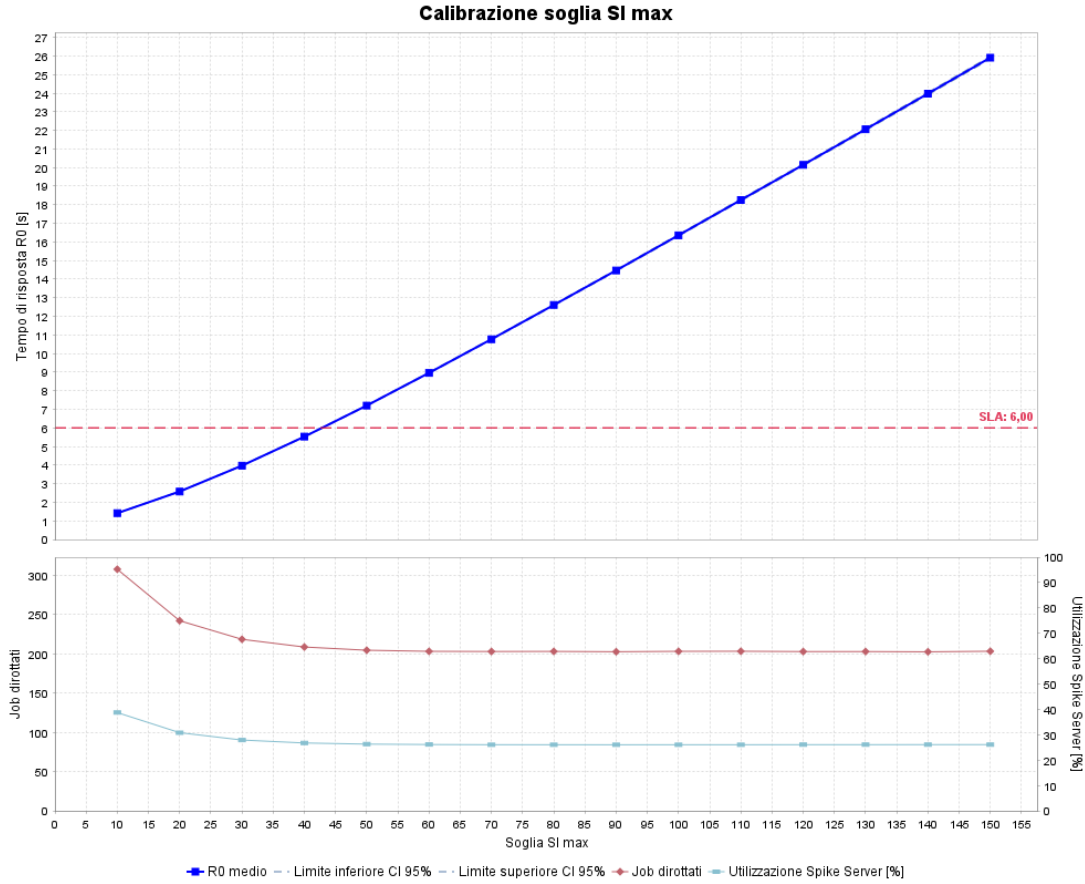


Figura 20: Trade-off tra tempo di risposta, quota di job dirottati e utilizzazione dello Spike Server al variare di SI_{max} .

Alla luce di queste evidenze, la scelta finale ricade su $SI_{max} = 40$. Tale valore rappresenta infatti il punto limite immediatamente precedente al superamento della soglia di 6 s fissata dal nostro SLA: con $SI_{max} = 40$ il sistema mantiene ancora $R_0 = 5.5279$ s e un intervallo di confidenza $[5.5107, 5.5451]$ s interamente al di sotto del vincolo, mentre già al passo successivo, $SI_{max} = 50$, il tempo medio di risposta sale a 7.2009 s violando il requisito di servizio. Inoltre, $SI_{max} = 40$ si colloca nella regione in cui la curva di utilizzazione dello Spike Server si è ormai stabilizzata attorno al 26%, rendendo non conveniente spostare ulteriormente in alto la soglia. Questa configurazione costituisce quindi il miglior punto di equilibrio tra protezione del Web Server principale, contenimento dell'uso dello Spike Server e rispetto rigoroso del vincolo SLA a regime.

Scaling Verticale e Sizing dell'Incremento Il dimensionamento dell'incremento verticale di capacità dello Spike Server è stato analizzato tramite la classe `VerticalStepSizingObjective`. L'obiettivo dell'esperimento era verificare se un incremento additivo più aggressivo della capacità elaborativa della risorsa ausiliaria producesse un beneficio misurabile sul tempo medio di risposta, una volta fissata la soglia di dirottamento a $SI_{max} = 40$. Lo scenario è stato mantenuto coerente con la precedente fase di dimensionamento: singolo Web Server ordinario, Spike Server abilitato, scaling orizzontale disabilitato, scaling verticale attivo e carico iperesponenziale ad alta variabilità. La campagna è stata eseguita con 16384 batch da 1024 job, scartando una fase iniziale di warm-up pari a 2048 job.

La calibrazione statistica preliminare ha rilevato, con il metodo ACF, un cutoff pari a 486, batch da 4860 job, 1028 batch e lag-1 ACF pari a 0.0252. Il metodo dinamico ha invece

raggiunto lag-1 ACF pari a 0.0059, ma con intervalli di confidenza più ampi sulla media. Anche in questo caso la scelta finale dei parametri privilegia quindi una stima più regolare del regime stazionario.

I risultati riportati in Tabella 14 mostrano che la variazione dell'incremento additivo di capacità non introduce alcun miglioramento prestazionale sostanziale. L'aspettativa iniziale era osservare almeno una riduzione moderata di R_0 per incrementi più elevati, nel caso in cui lo Spike Server fosse il collo di bottiglia dominante; l'esito sperimentale mostra invece che tale ipotesi non è verificata. Il tempo medio di risposta resta infatti compreso in un intervallo estremamente ristretto, tra 5.5103 s e 5.5162 s, e tutti gli intervalli di confidenza al 95% risultano completamente sovrapposti. Anche il grafico in Figura 21 conferma l'andamento sostanzialmente piatto della curva di R_0 : l'aumento della capacità verticale disponibile non modifica in modo apprezzabile il comportamento globale del sistema, perché il collo di bottiglia dominante non è la capacità istantanea dello Spike Server, ma il momento in cui il traffico viene effettivamente dirottato dal Web Server ordinario.

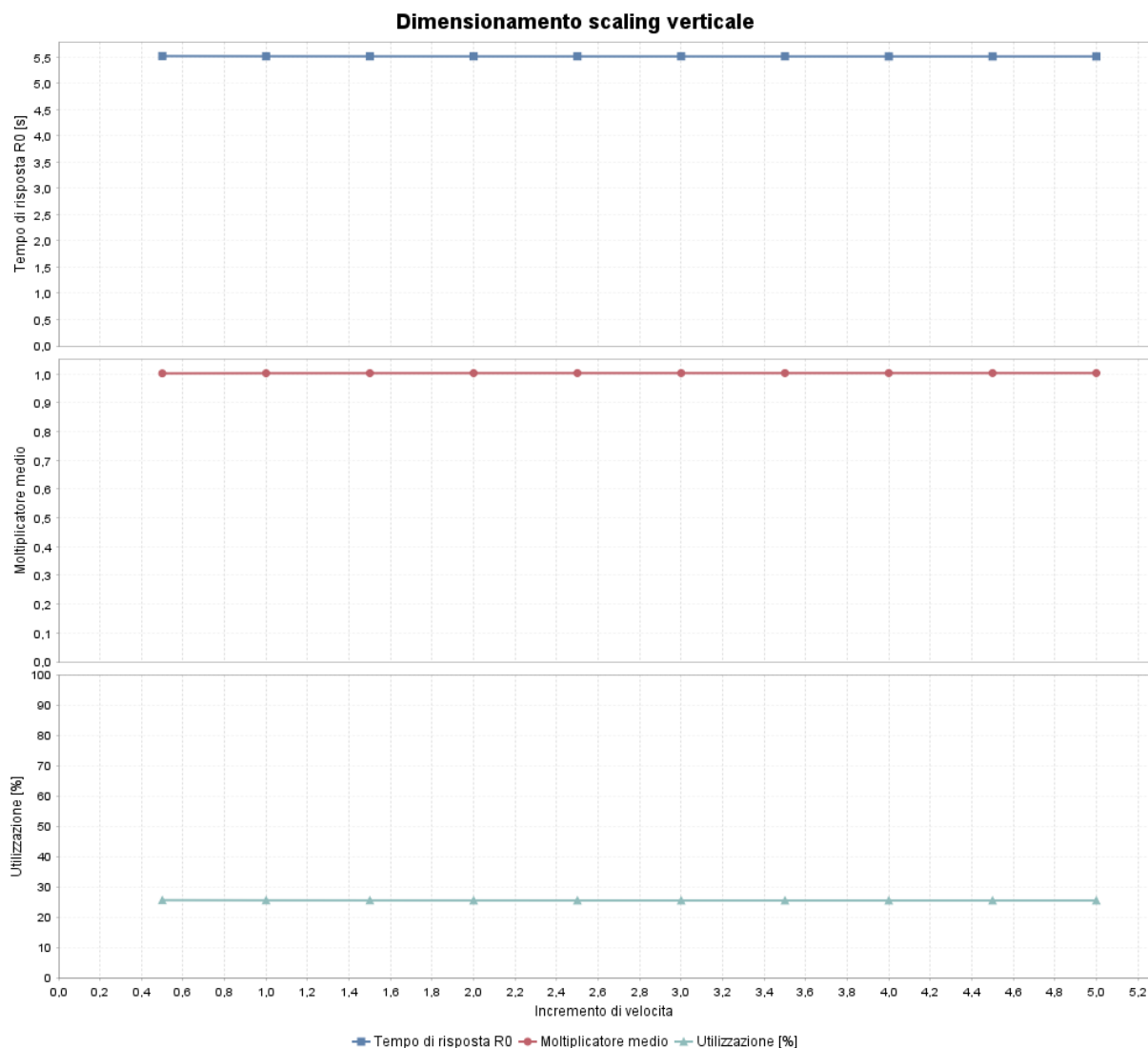


Figura 21: Effetto dell'incremento additivo di capacità su tempo di risposta, capacità media e utilizzazione dello Spike Server.

Questa evidenza porta a una conclusione progettuale precisa: aumentare ulteriormente l'incremento verticale di capacità non è conveniente. Dal punto di vista economico, una capacità

Incremento Velocità	R_0 Medio [s]	Half-Width [s]	CI 95% R_0 [s]
0.50	5.5162	0.0193	[5.4969, 5.5355]
1.00	5.5132	0.0193	[5.4939, 5.5325]
1.50	5.5120	0.0192	[5.4928, 5.5312]
2.00	5.5113	0.0192	[5.4921, 5.5305]
3.00	5.5107	0.0192	[5.4915, 5.5299]
4.00	5.5104	0.0192	[5.4912, 5.5296]
5.00	5.5103	0.0192	[5.4911, 5.5295]

Tabella 14: Analisi dell’impatto dell’incremento additivo di capacità sul tempo medio di risposta.

di picco più elevata comporterebbe un costo potenziale maggiore senza produrre una riduzione statisticamente significativa della latenza media. Per questo motivo si è scelto di adottare un incremento additivo di capacità pari a **1.0**; poiché il valore di base impiegato nell’esperimento è anch’esso pari a 1.0, lo stato scalato raggiunge $1.0 + 1.0 = 2.0$, corrispondente in questo caso a un raddoppio della potenza disponibile. Tale configurazione rappresenta la scelta più sobria e difendibile: garantisce una risposta sufficiente nelle fasi di picco, evita sovradimensionamenti inutili e mantiene il costo operativo dello Spike Server sotto controllo.

Scaling Orizzontale e Identificazione delle Soglie La stima dei parametri per lo scaling orizzontale è stata condotta tramite la classe `HorizontalScalingEstimationObjective`. Il parametro superiore di intervento, cioè la soglia di *Scale Out*, è stato fissato direttamente al valore del vincolo SLA, pari a $R^{max} = 6.0$ s. Questa scelta è coerente con una politica di controllo parsimoniosa: l’aumento del numero di Web Server viene attivato solo quando il tempo medio di risposta supera effettivamente il limite prestazionale concordato, evitando espansioni inutili quando il sistema si trova ancora all’interno dello SLA. Qualora si desiderasse un comportamento più reattivo e prudentiale, lo stesso parametro potrebbe essere abbassato leggermente, anticipando lo *Scale Out* prima del raggiungimento formale della soglia critica e aumentando così il margine di sicurezza rispetto a variazioni improvvise del carico.

L’obiettivo dell’esperimento non è stato quello di osservare direttamente la dinamica del controller, bensì di ricostruire la frontiera prestazionale delle configurazioni statiche da 1 a 5 Web Server al variare del tasso di arrivo λ . Per ogni coppia (λ, N) è stata preliminarmente eseguita una verifica di convergenza a orizzonte finito tramite repliche indipendenti; solo le configurazioni classificate come convergenti sono state poi analizzate a regime con Batch Means, usando il numero di job di convergenza come *warm-up*. Le configurazioni non convergenti o inconclusive sono state escluse dalla stima steady-state e riportate esplicitamente nel CSV e nel grafico.

I risultati, sintetizzati nella Tabella 15, evidenziano che con SLA pari a 6 s la regione ammissibile rimane coerente con il carico nominale del progetto, ma diventa più selettiva nei regimi ad alta intensità. Il comportamento atteso è quello tipico di una frontiera di capacità: a parità di λ , l’aumento del numero di Web Server deve ridurre il tempo di risposta, mentre a parità di server l’incremento del carico deve avvicinare progressivamente il sistema alla saturazione. I risultati seguono questa struttura. Per carichi prossimi a $\lambda = 5$, che rappresentano il centro dello scenario di riferimento, un singolo Web Server è ancora sostenibile, con $R_0 = 5.5212$ s e intervallo di confidenza $[5.4926, 5.5498]$ s interamente sotto soglia. Anche a $\lambda = 6.5$ la configurazione a un solo server resta formalmente ammissibile, con $R_0 = 5.8807$ s, ma il margine

rispetto allo SLA diventa molto ridotto. L'aggiunta di un secondo server riduce drasticamente la latenza, portandola a 0.7296 s per $\lambda = 5$ e a 1.5517 s per $\lambda = 6.5$, mostrando che in questa regione due nodi sono prestazionalmente conservativi.

λ	1 WS	2 WS	3 WS	5 WS
5.00	5.5130 ± 0.0283	0.7316 ± 0.0077	0.3563 ± 0.0012	0.2614 ± 0.0008
6.50	5.8808 ± 0.0334	1.5519 ± 0.0309	0.4608 ± 0.0022	0.2757 ± 0.0010
8.00	<i>INC.</i>	4.4585 ± 0.0878	0.6542 ± 0.0046	0.2978 ± 0.0012
9.50	<i>Skip</i>	6.5519 ± 0.0418	1.0545 ± 0.0117	0.3303 ± 0.0015
11.00	<i>Skip</i>	7.7255 ± 0.1140	<i>INC.</i>	0.3776 ± 0.0020

Tabella 15: Sintesi dei risultati di dimensionamento orizzontale con SLA pari a 6 s. I valori riportano $R_0 \pm$ Half-Width; *INC.* indica configurazione inconclusiva nella verifica di convergenza, mentre *Skip* indica configurazioni saltate per monotonicità del carico.

La soglia di *Scale In* deve quindi essere scelta in modo da favorire il ritorno a una sola unità quando il carico nominale è prossimo a $\lambda = 5$, ma senza autorizzare contrazioni pericolose nella regione in cui un singolo Web Server non è più sostenibile. Poiché $R^{max} = 6.0$ s rappresenta già la soglia superiore di violazione dello SLA, non sarebbe corretto usare lo stesso valore anche come soglia di riduzione: verrebbe infatti annullato il margine tra espansione e contrazione, rendendo il controller troppo incline a rimuovere risorse appena il sistema rientra formalmente sotto il limite. Alla luce dei risultati aggiornati, una scelta più coerente per lo scenario principale è $R^{min} = 3.5$ s. Questo valore mantiene un margine sufficiente rispetto allo SLA, evita oscillazioni eccessive vicino alla soglia critica e consente comunque il ritorno verso configurazioni più economiche quando il carico si trova nella regione ordinaria del progetto.

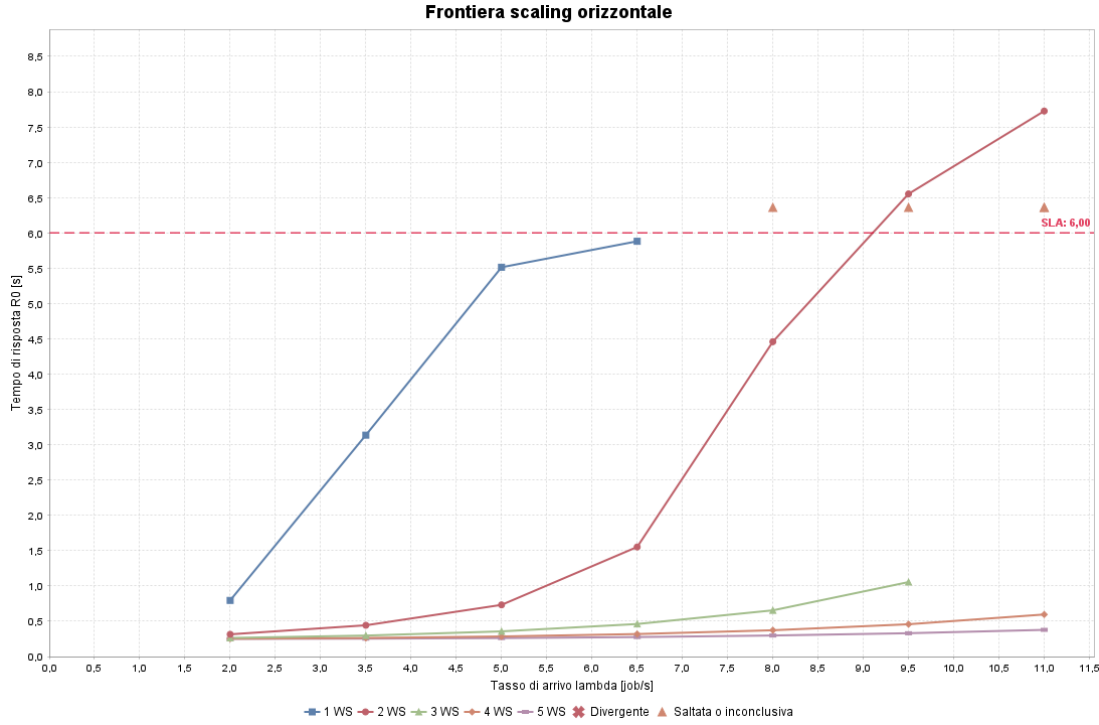


Figura 22: Frontiera prestazionale delle configurazioni statiche da 1 a 5 Web Server al variare di λ , con SLA pari a 6 s. I marker sopra la soglia indicano configurazioni non simulate a regime poiché non convergenti o saltate per monotonicità.

Per scenari di carico più elevato, prossimi a $\lambda = 9.5$ o $\lambda = 11$, la soglia a 6 s modifica sensibilmente l'interpretazione. Due server non garantiscono più il rispetto del vincolo, poiché producono rispettivamente $R_0 = 6.5519$ s e $R_0 = 7.7255$ s; al contrario, cinque server mantengono la latenza entro margini molto ampi, con $R_0 = 0.3303$ s a $\lambda = 9.5$ e $R_0 = 0.3776$ s a $\lambda = 11$. La configurazione a tre server resta valida fino a $\lambda = 9.5$, ma nello scenario più intenso risulta inconclusiva nella verifica di convergenza; per questo motivo, in presenza di workload persistentemente elevati, una soglia di *Scale In* tra 1.5 s e 2.0 s rimarrebbe la scelta più conservativa, perché manterrebbe più facilmente tre o più server attivi e offrirebbe maggiore margine rispetto a burst improvvisi. Il valore $R^{min} = 3.5$ s va invece interpretato come una scelta coerente con un profilo di carico centrato su $\lambda \simeq 5$, non come soglia ottimale per workload persistentemente elevati.

In conclusione, il dimensionamento dello scaling orizzontale dipende dal carico nominale atteso. Nel caso di studio principale, centrato su valori di λ prossimi a 5 e con SLA pari a 6 s, la scelta $R^{min} = 3.5$ s è coerente con una politica orientata al contenimento dei costi: preserva il risparmio economico nei regimi sostenibili da un solo Web Server, ma conserva un margine rispetto a R^{max} sufficiente a evitare rimozioni premature. Per workload persistentemente più aggressivi, invece, la soglia deve essere abbassata ulteriormente, indicativamente nell'intervallo 1.5–2.0 s, accettando un costo infrastrutturale maggiore in cambio di una maggiore robustezza operativa. Le verifiche di convergenza hanno mostrato warm-up molto variabili: le configurazioni stabili convergono tipicamente tra 60000 e 240000 job, mentre i casi prossimi alla saturazione possono arrivare a 940000 job senza classificazione conclusiva. Questo risultato conferma la necessità di usare il job di convergenza stimato come warm-up della successiva analisi Batch Means.

9.2 Analisi

Analisi Economica e Trade-off dei Costi L'analisi dei costi operativi è stata implementata nella classe `CostOptimizationObjective`, mantenendo fisso il limite superiore $R^{max} = 6.0$ s e valutando congiuntamente diverse soglie di *Scale In* e diversi tempi di *Cooldown*. La campagna è stata costruita sulla configurazione completa del sistema: politica *Least Loaded*, $SI_{max} = 40$, routing verso lo Spike Server basato sulla sola soglia superiore, scaling orizzontale e verticale abilitati, *Window Size* pari a 100 s e *Cooldown* variabile tra 5, 10, 50 e 100 s.

Per questa campagna sperimentale la calibrazione dei Batch Means è stata effettuata tramite l'approccio dinamico, poiché il metodo ACF richiedeva run molto lunghe e costose. La stima preliminare ha individuato un cutoff ACF pari a 1221, con 409 batch da 12210 job e lag-1 ACF pari a -0.0531 ; il metodo dinamico ha invece raggiunto lag-1 ACF pari a 0.0050 con 34 batch da 4096 job e precisione relativa pari a 0.0293 . L'esperimento finale usa quindi un arrotondamento superiore, pari a 64 batch da 8192 job, e un warm-up prudenziale pari a 350000 job. La stima di convergenza a orizzonte finito aveva indicato stabilizzazione già a 60000 job, con $R_0 = 3.3309$ s e HW pari a 0.0343 s; la scelta di amplificare il warm-up riduce ulteriormente il rischio di includere transitori residui.

Il modello economico adottato considera un costo fisso per ogni Web Server attivo e un costo variabile per lo Spike Server proporzionale sia alla sua utilizzazione sia al fattore medio di capacità allocata:

$$C_{tot} = \bar{N} C_{WS} + \bar{U}_{spike} \bar{S}_{spike} C_{CPU}.$$

In questo modo il costo della capacità orizzontale viene pagato ogni volta che un Web Server resta mediamente acceso, mentre il costo della risorsa di picco cresce solo quando lo Spike Server viene effettivamente utilizzato e accelerato.

I risultati, sintetizzati nella Tabella 16 e nelle Figure 23–26, confermano il trade-off atteso. Soglie di *Scale In* basse, come $R^{min} = 1.5$ s o $R^{min} = 2.0$ s, mantengono più a lungo server aggiuntivi nel cluster: questo abbassa il tempo medio di risposta, soprattutto ad alto carico, ma aumenta sensibilmente il costo medio a causa del maggior numero di Web Server attivi. Al contrario, soglie più permissive, come $R^{min} = 3.5$ s o $R^{min} = 5.0$ s, riducono il costo operativo perché favoriscono una contrazione più rapida del cluster; il prezzo pagato è un tempo di risposta più elevato, pur rimanendo entro il vincolo SLA in tutti gli scenari simulati.

L'esito è quindi in linea con il comportamento atteso del controller: abbassare la soglia di *Scale In* rende il sistema più prudente e prestazionalmente robusto, ma più costoso; alzarla spinge invece verso configurazioni più economiche, accettando una latenza media maggiore. La campagna serve proprio a quantificare questo compromesso, non a individuare un minimo assoluto valido per qualunque profilo di traffico.

R^{min}	Cooldown [s]	$\bar{C}_{tot}$	$\bar{R}_0$ [s]
1.5	10	5.282	2.997
2.0	10	5.040	3.106
3.5	10	4.496	3.338
5.0	10	4.302	3.470

Tabella 16: Sintesi del compromesso costo-prestazioni con *Cooldown* fissato a 10 s. Le medie sono calcolate sui valori di λ simulati.

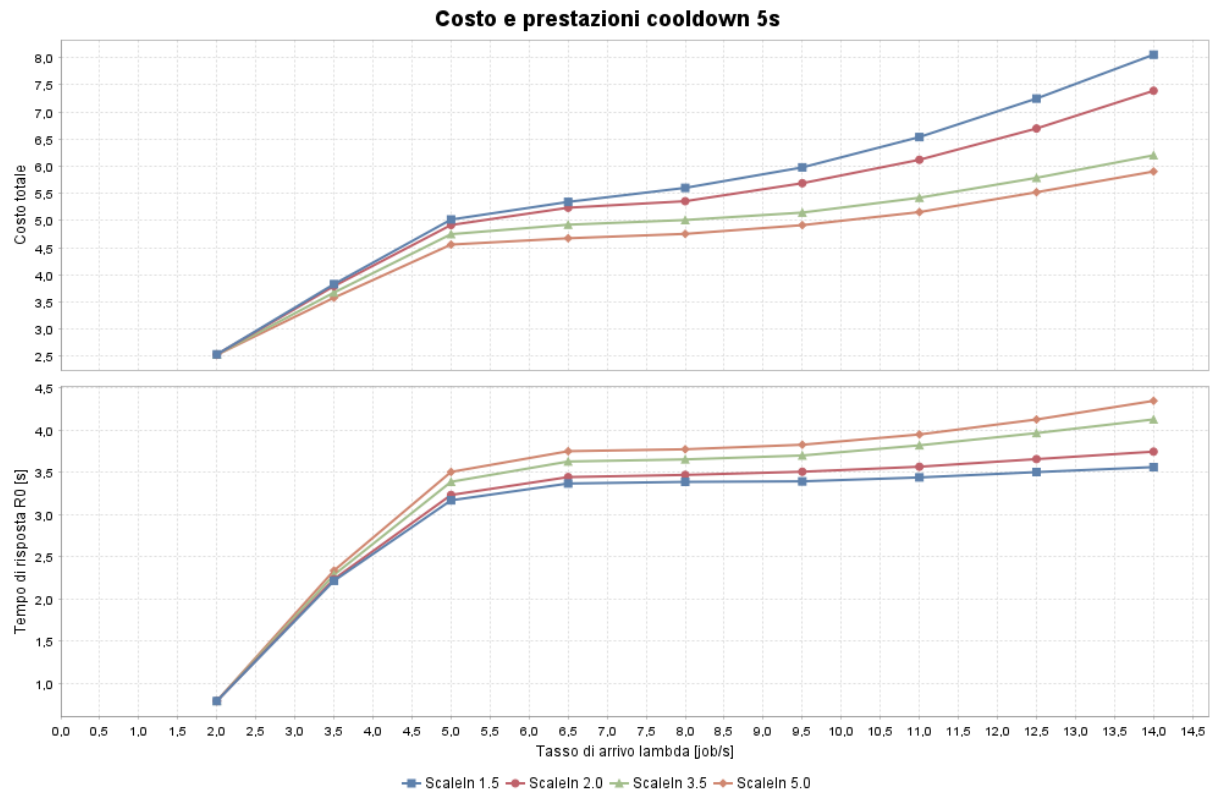


Figura 23: Andamento del costo totale e del tempo medio di risposta con *Cooldown* pari a 5 s.

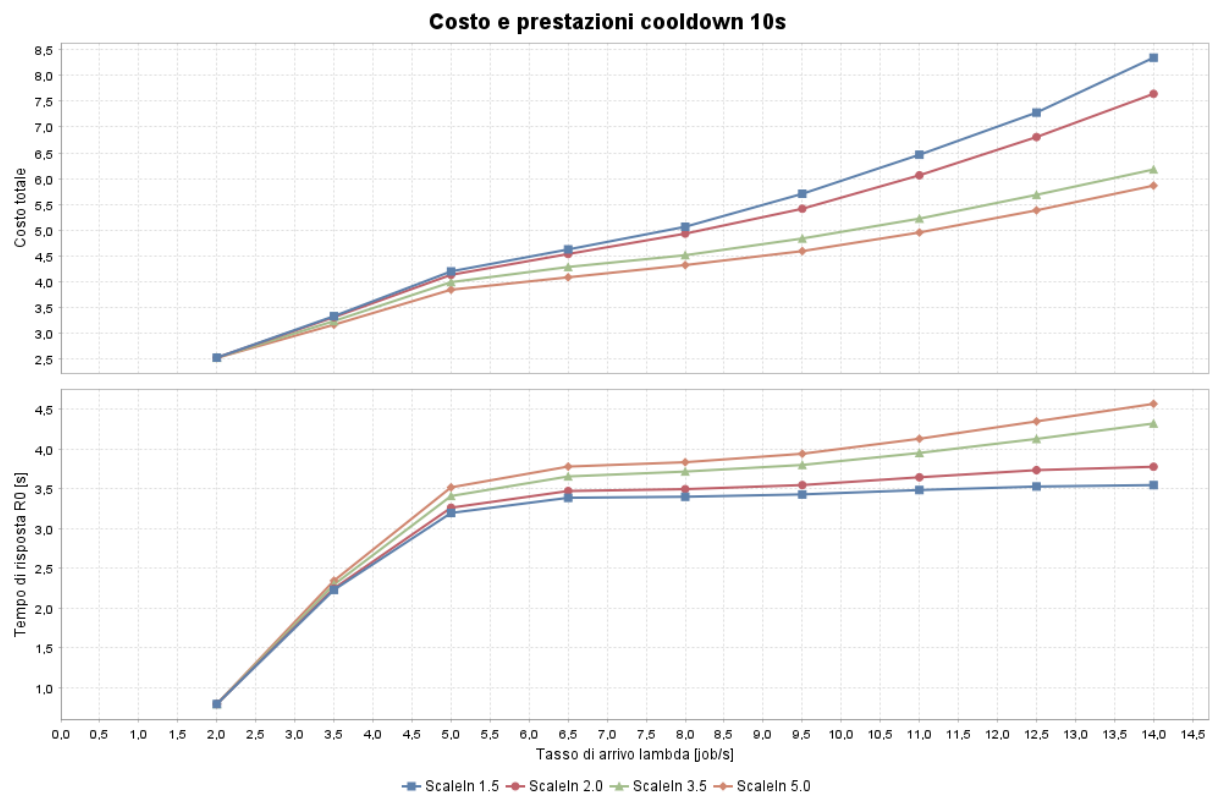


Figura 24: Andamento del costo totale e del tempo medio di risposta con *Cooldown* pari a 10 s.

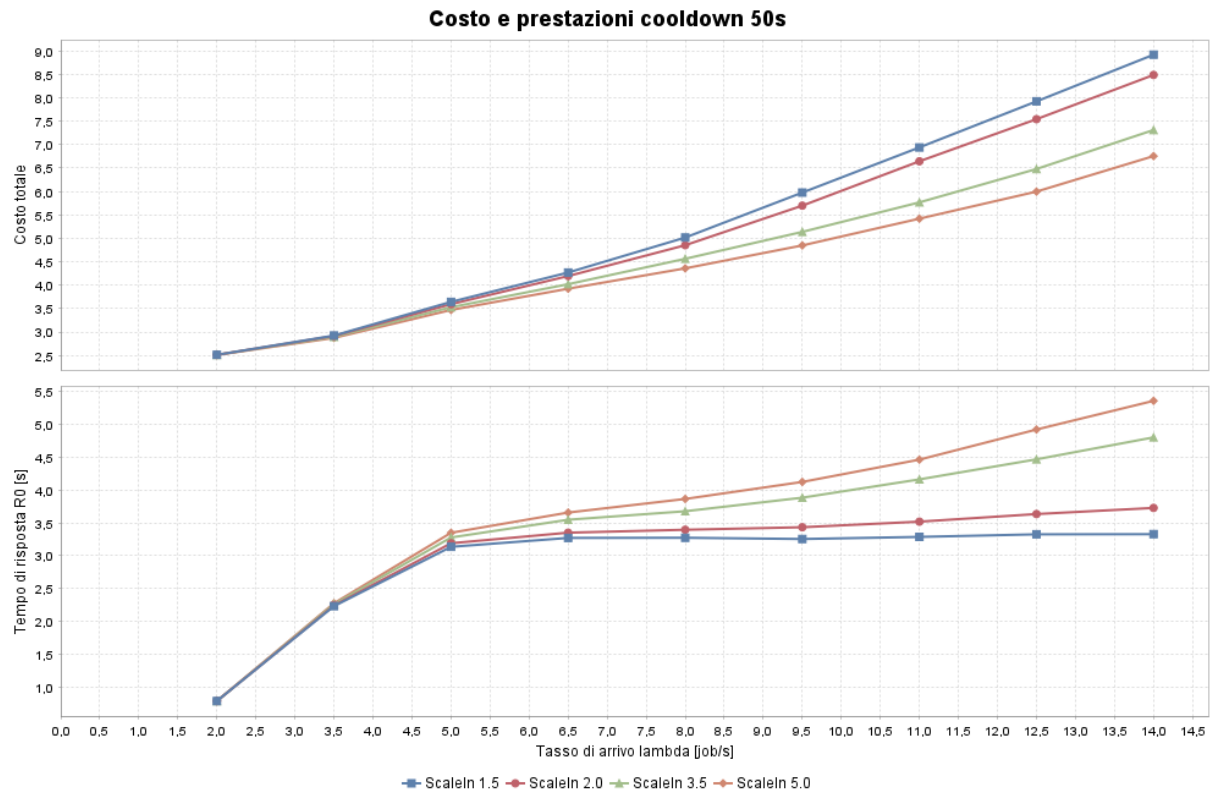


Figura 25: Andamento del costo totale e del tempo medio di risposta con *Cooldown* pari a 50 s.

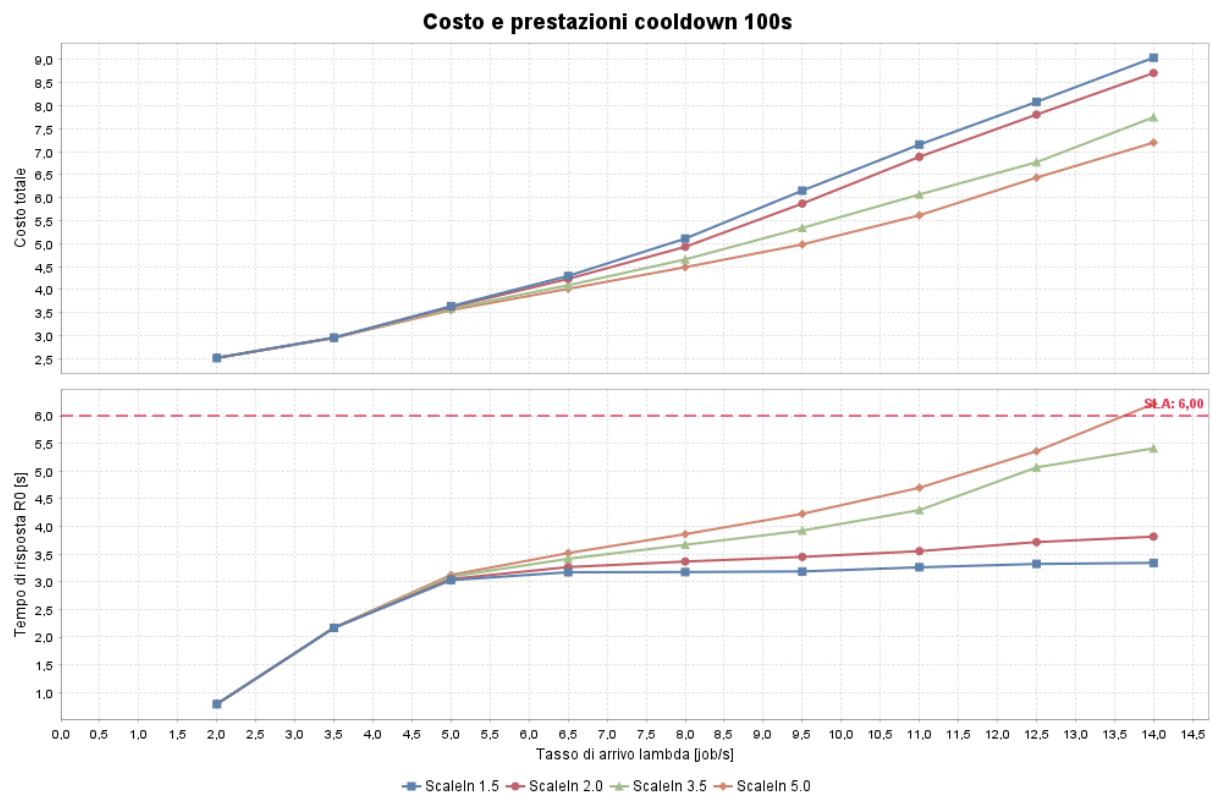


Figura 26: Andamento del costo totale e del tempo medio di risposta con *Cooldown* pari a 100 s.

Il *Cooldown* ha un ruolo altrettanto rilevante. Un valore troppo breve, pari a 5 s, rende il controller molto reattivo ma tende ad aumentare il costo medio: con $R^{min} = 5.0$ s il costo medio è 4.622, contro 4.302 ottenuto con 10 s; con $R^{min} = 3.5$ s il costo passa analogamente da 4.828 a 4.496. Valori più lunghi riducono la frequenza delle riconfigurazioni, ma possono ritardare l'adattamento del cluster nei regimi più intensi: con $R^{min} = 5.0$ s e $\lambda = 11$, il tempo medio di risposta passa da 4.1262 s con *Cooldown* pari a 10 s a 4.4611 s con 50 s e 4.6981 s con 100 s. D'altra parte, per soglie di *Scale In* molto basse, un *Cooldown* più lungo può anche ridurre leggermente la latenza media, perché mantiene la capacità orizzontale allocata più a lungo dopo un episodio di carico elevato.

Alla luce di queste evidenze, la configurazione economicamente più difendibile dipende dal profilo di carico atteso. Se si privilegia la robustezza prestazionale, $R^{min} = 1.5$ s o $R^{min} = 2.0$ s mantengono tempi di risposta più bassi, ma accettano un costo medio più elevato, rispettivamente pari a 5.282 e 5.040 con *Cooldown* di 10 s. Se invece l'obiettivo principale è contenere la spesa mantenendo il rispetto dello SLA, le configurazioni con $R^{min} = 3.5$ s–5.0 s sono più convenienti: il costo medio scende a 4.496 e 4.302, mentre a $\lambda = 11$ il tempo medio di risposta rimane pari a 3.9478 s e 4.1262 s. Per le analisi successive si adotta quindi ***Cooldown* pari a 10.0 s**: tale valore preserva una buona reattività nei regimi ad alto carico, mantiene R_0 ben al di sotto dello SLA anche per $\lambda = 11$ e riduce sensibilmente il costo rispetto al *Cooldown* di 5 s.

Analisi Transitoria a Orizzonte Finito L'analisi transitoria è stata condotta tramite la classe `TransientAnalysisObjective`. A differenza delle campagne steady-state, qui il sistema viene osservato a partire da uno stato iniziale vuoto, usando repliche indipendenti e salvando l'evoluzione temporale del tempo medio di risposta. Sono stati considerati sei scenari sulla sola architettura completa: due a basso carico, due a carico medio e due ad alto carico, variando sia λ sia il coefficiente di variazione degli interarrivi. Ogni scenario è stato eseguito con 10 repliche indipendenti; gli scenari low e medium sono stati simulati fino a circa 1008000 s, mentre gli scenari high fino a 504000 s, con scala temporale dei grafici convertita in ore per renderli leggibili.

Scenario	λ	C_V	R_0 [s]	HW [s]	$\bar{N}$	Util. Spike [%]
Low CV2	2.5	2.0	0.8102	0.0020	1.0055	0.00
Low CV4	2.5	4.0	1.1721	0.0049	1.0466	0.00
Medium CV4	5.0	4.0	3.4218	0.0041	1.5707	9.68
Medium CV6	5.0	6.0	3.3204	0.0062	1.6710	14.34
High CV4	8.0	4.0	3.7118	0.0031	1.6274	39.59
High CV6	8.0	6.0	3.4832	0.0047	1.6581	42.28

Tabella 17: Risultati aggregati dell'analisi transitoria a orizzonte finito sulla configurazione completa.

La Tabella 17 mostra che il sistema rimane ampiamente sotto lo SLA in tutti gli scenari. L'obiettivo era verificare che, partendo da sistema vuoto, la configurazione completa non producesse transitori lunghi o derive iniziali capaci di falsare le conclusioni steady-state. Il risultato ottenuto è positivo: nei casi low load lo Spike Server rimane praticamente inutilizzato, con $\lambda = 2.5$ e $C_V = 2.0$ la media dei job devianti è prossima a zero e la latenza converge attorno a 0.81 s; aumentando la variabilità a $C_V = 4.0$ il tempo medio sale a 1.17 s, ma l'architettura non richiede ancora un uso strutturale della risorsa di picco. Nei casi medium e high emerge invece la funzione gerarchica del sistema: il numero medio di server attivi cresce tra 1.57 e 1.67,

mentre lo Spike Server assorbe una quota crescente dei burst, arrivando a utilizzazioni medie superiori al 40% negli scenari high.

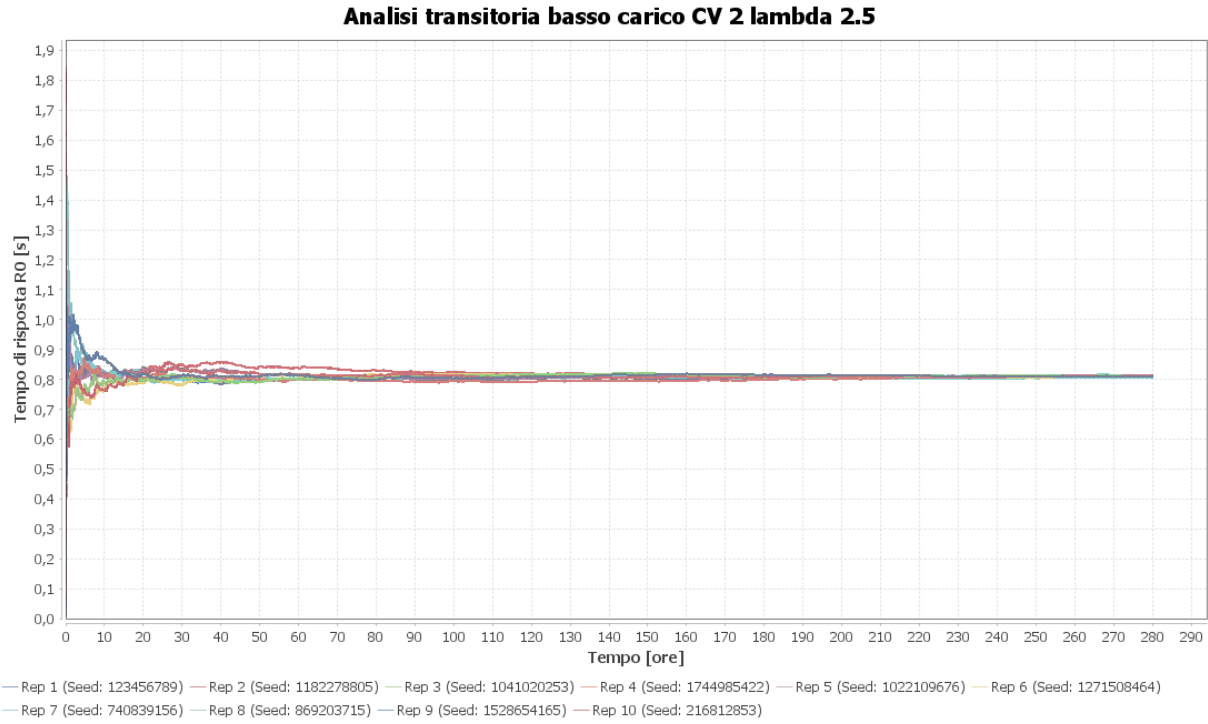


Figura 27: Analisi transitoria dello scenario Low Load con $C_V = 2.0$ e $\lambda = 2.5$.

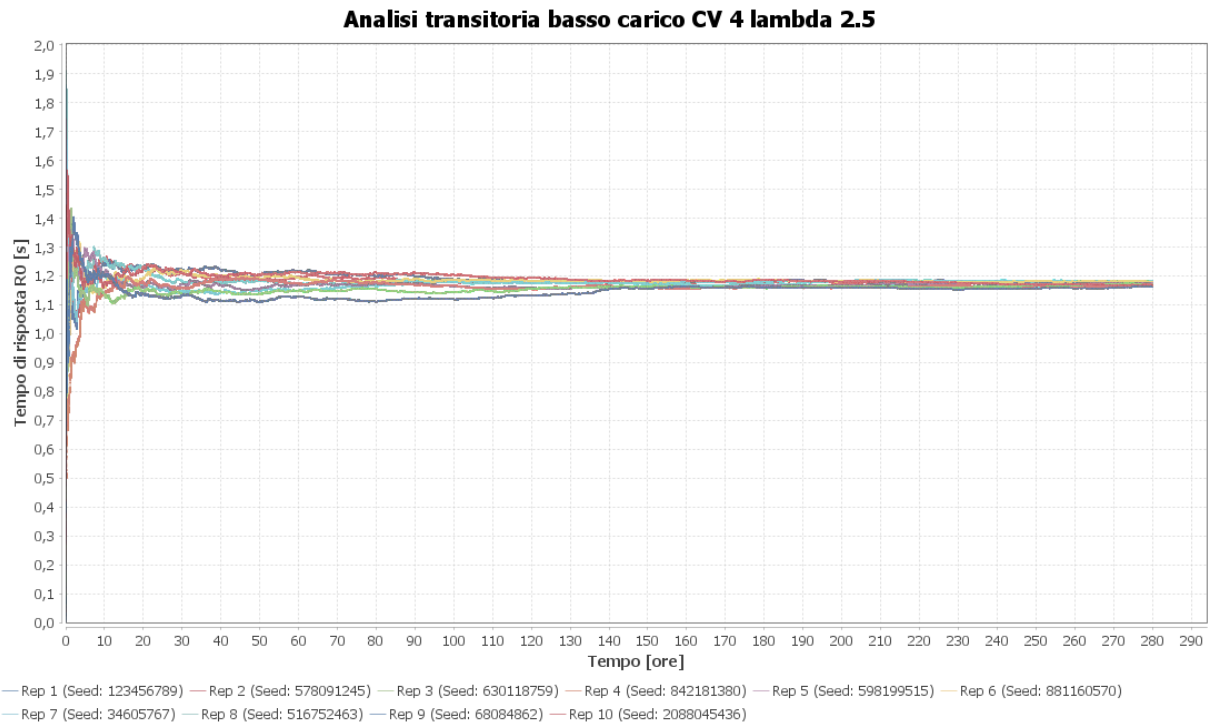


Figura 28: Analisi transitoria dello scenario Low Load con $C_V = 4.0$ e $\lambda = 2.5$

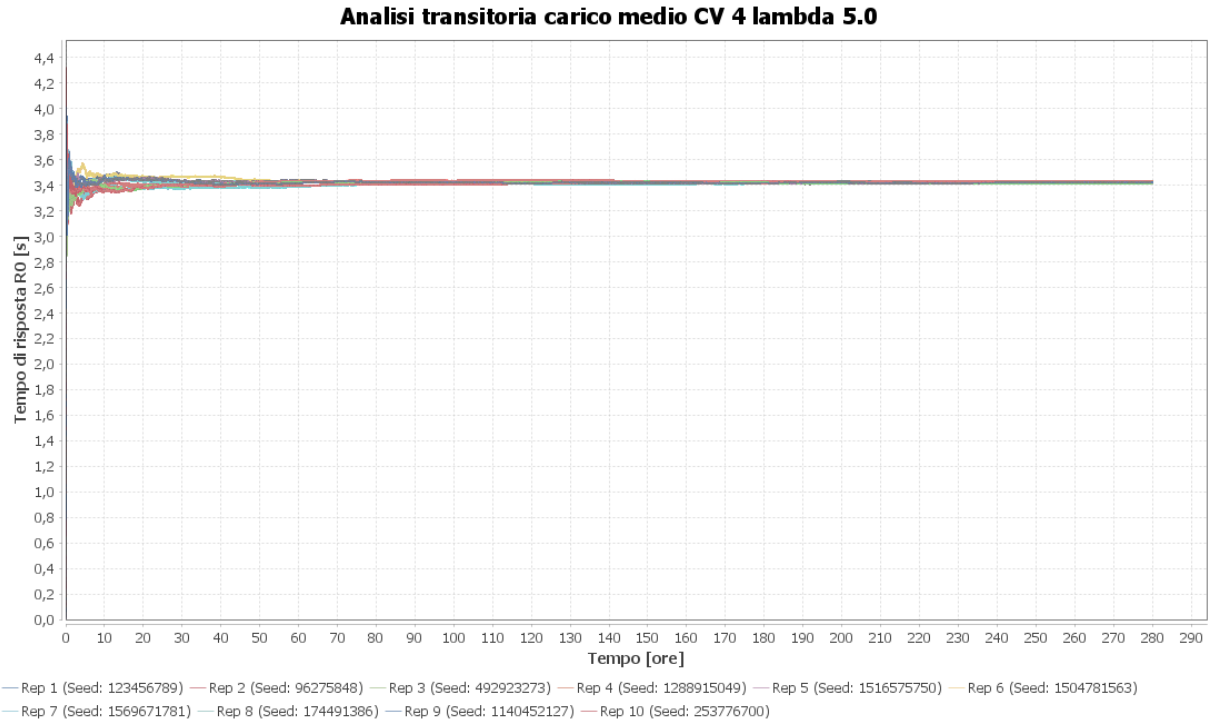


Figura 29: Analisi transitoria dello scenario Medium Load con $C_V = 4.0$ e $\lambda = 5.0$

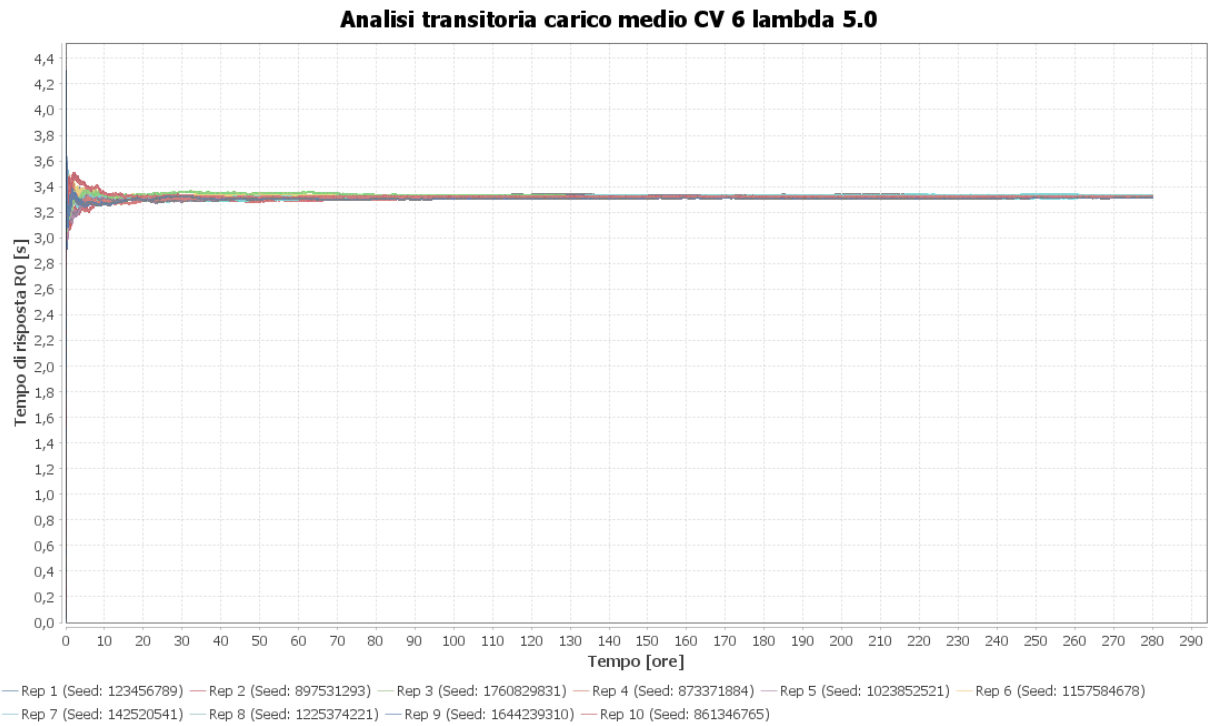


Figura 30: Analisi transitoria dello scenario Medium Load con $C_V = 6.0$ e $\lambda = 5.0$

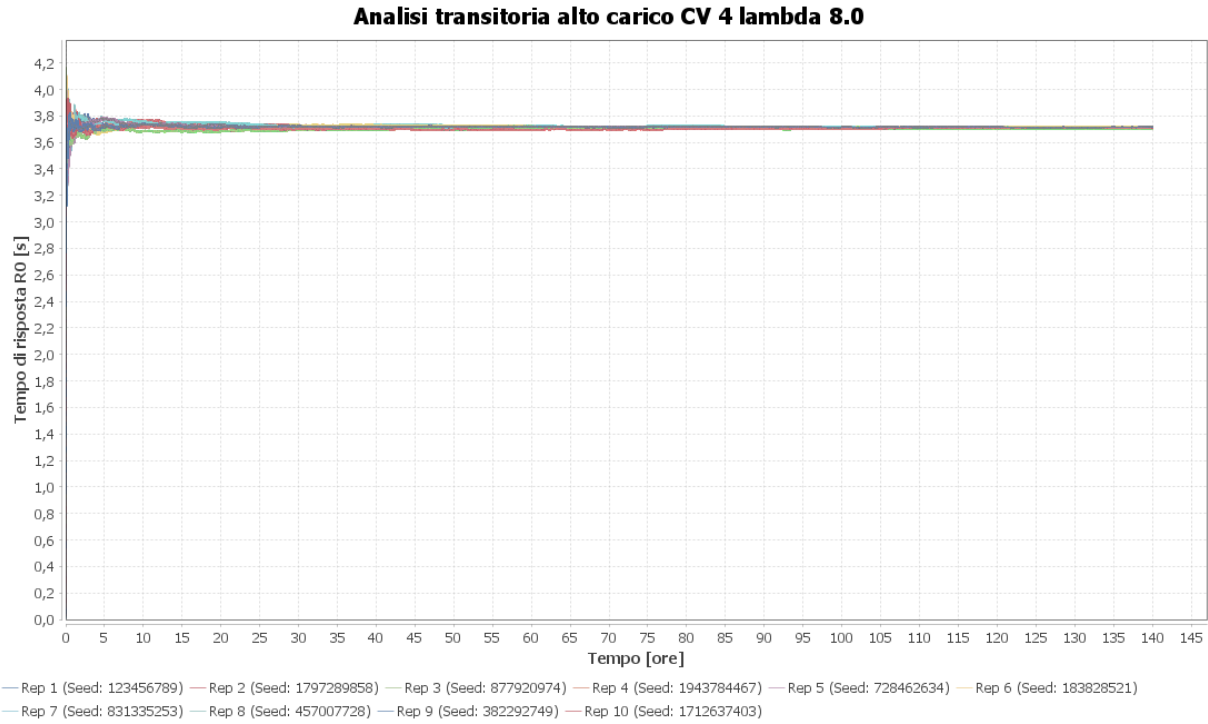


Figura 31: Analisi transitoria dello scenario High Load con $C_V = 4.0$ e $\lambda = 8.0$

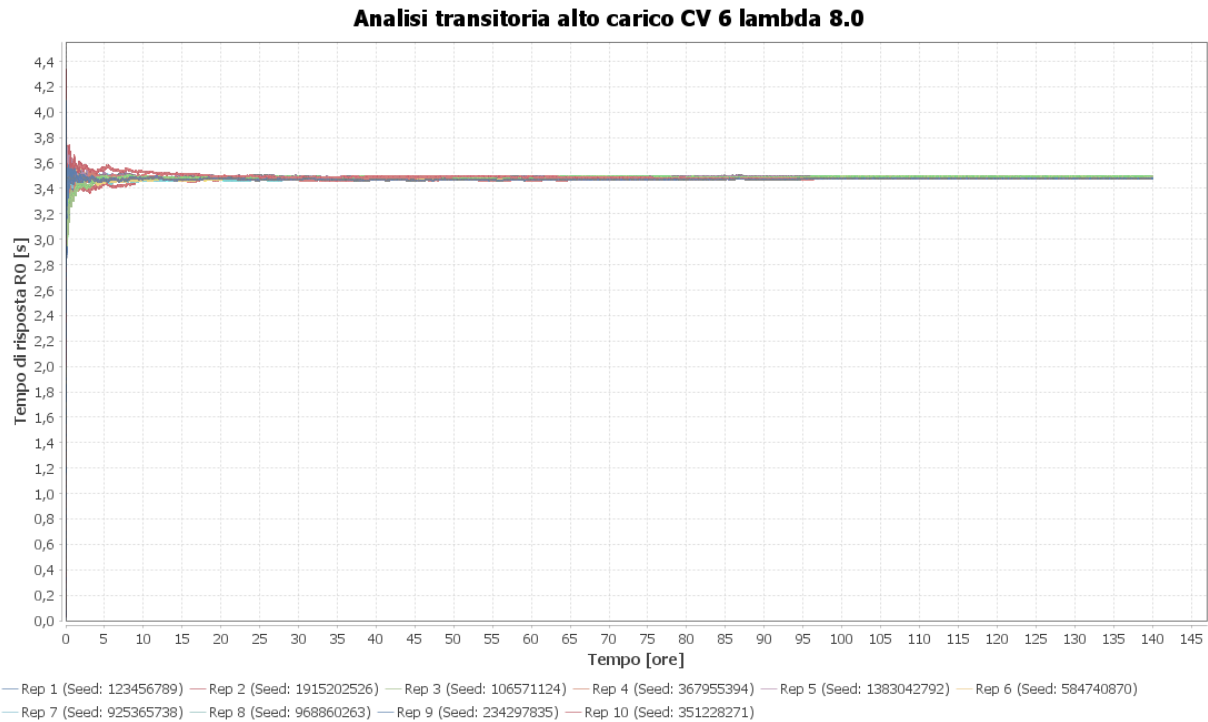


Figura 32: Analisi transitoria dello scenario High Load con $C_V = 6.0$ e $\lambda = 8.0$

I grafici confermano che la fase iniziale non presenta derive instabili: dopo l'assestamento iniziale, le repliche oscillano attorno a livelli coerenti con le stime aggregate. L'aumento del coefficiente di variazione non produce necessariamente un peggioramento monotono della media finale, perché il sistema reagisce in modo diverso alla burstiness: quando i burst diventano più concentrati, una quota maggiore di traffico viene intercettata dal meccanismo di dirottamento

C_V	λ	R_0 [s]	HW [s]	JIS	$\bar{N}$	Util.	Spike [%]
1.0	5.0	3.4204	0.0214	17.12	1.61		2.34
1.0	11.0	4.0110	0.0167	44.15	1.68		67.34
4.0	5.0	3.4064	0.0167	17.14	1.56		9.52
4.0	11.0	3.9478	0.0182	43.67	1.76		62.44
8.0	5.0	3.2196	0.0163	16.42	1.83		17.91
8.0	11.0	3.7171	0.0227	41.79	2.03		61.20
12.0	5.0	3.1388	0.0195	16.42	2.33		22.26
12.0	11.0	3.6265	0.0312	42.03	2.57		58.35

Tabella 18: Sintesi della robustezza alla burstiness per alcuni punti rappresentativi.

e dallo scaling verticale, riducendo il carico residuo sui Web Server ordinari. Questo comportamento è particolarmente visibile negli scenari medium e high, dove l'aumento di C_V fa crescere il numero di eventi di *Scale Up* e l'utilizzazione dello Spike Server, ma mantiene R_0 in una regione sicura.

Robustezza alla Burstiness La robustezza del sistema rispetto alla variabilità degli interarrivi è stata analizzata tramite la classe `BurstinessRobustnessObjective`. La campagna usa la configurazione completa scelta nelle fasi precedenti e osserva, al variare di λ e di C_V , non solo il tempo medio di risposta, ma anche job nel sistema, utilizzazione, throughput, job deviati, numero medio di server attivi e numero di azioni di scaling. La stima è stata condotta con 1024 batch da 4096 job, riportando per ogni metrica media, deviazione standard, *Half-Width* e intervallo di confidenza al 95%. Il comportamento atteso è una maggiore attivazione delle risorse elastiche al crescere della burstiness, con possibile aumento della variabilità delle misure ma senza perdita di stabilità del tempo medio di risposta.

La Tabella 18 evidenzia un risultato importante: l'incremento della burstiness non porta il sistema fuori controllo. Al contrario, a parità di λ , l'aumento di C_V tende a far crescere l'uso delle risorse elastiche, ma mantiene il tempo medio di risposta in una regione più che accettabile rispetto allo SLA. Per $\lambda = 11$, ad esempio, R_0 passa da 4.0110 s con $C_V = 1.0$ a 3.6265 s con $C_V = 12.0$; questo non significa che un traffico più bursty sia intrinsecamente più semplice, ma che il meccanismo di protezione intercetta più frequentemente i picchi e sposta lavoro verso lo Spike Server e verso configurazioni con più Web Server attivi.

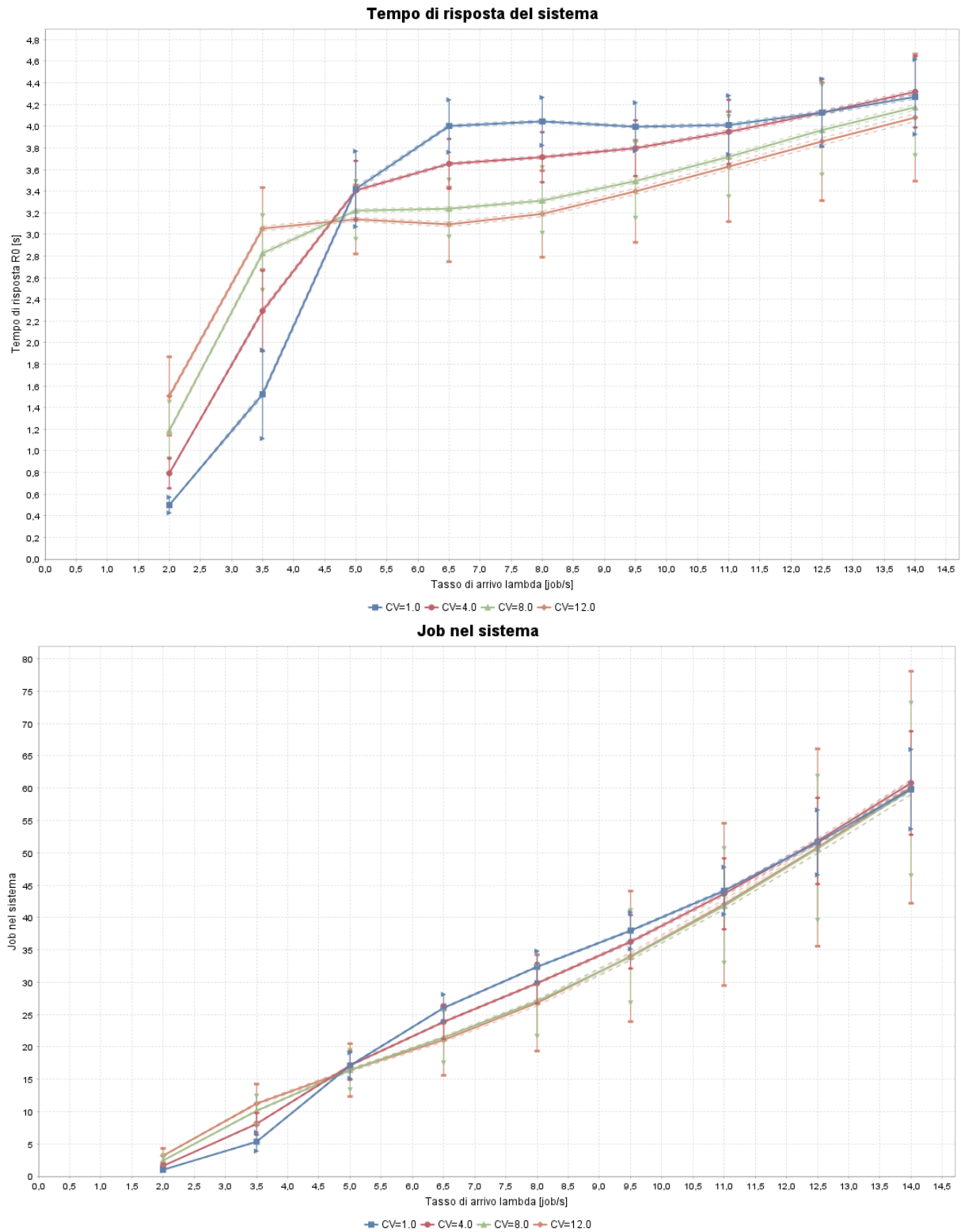


Figura 33: Robustezza alla burstiness: tempo medio di risposta e numero medio di job nel sistema.

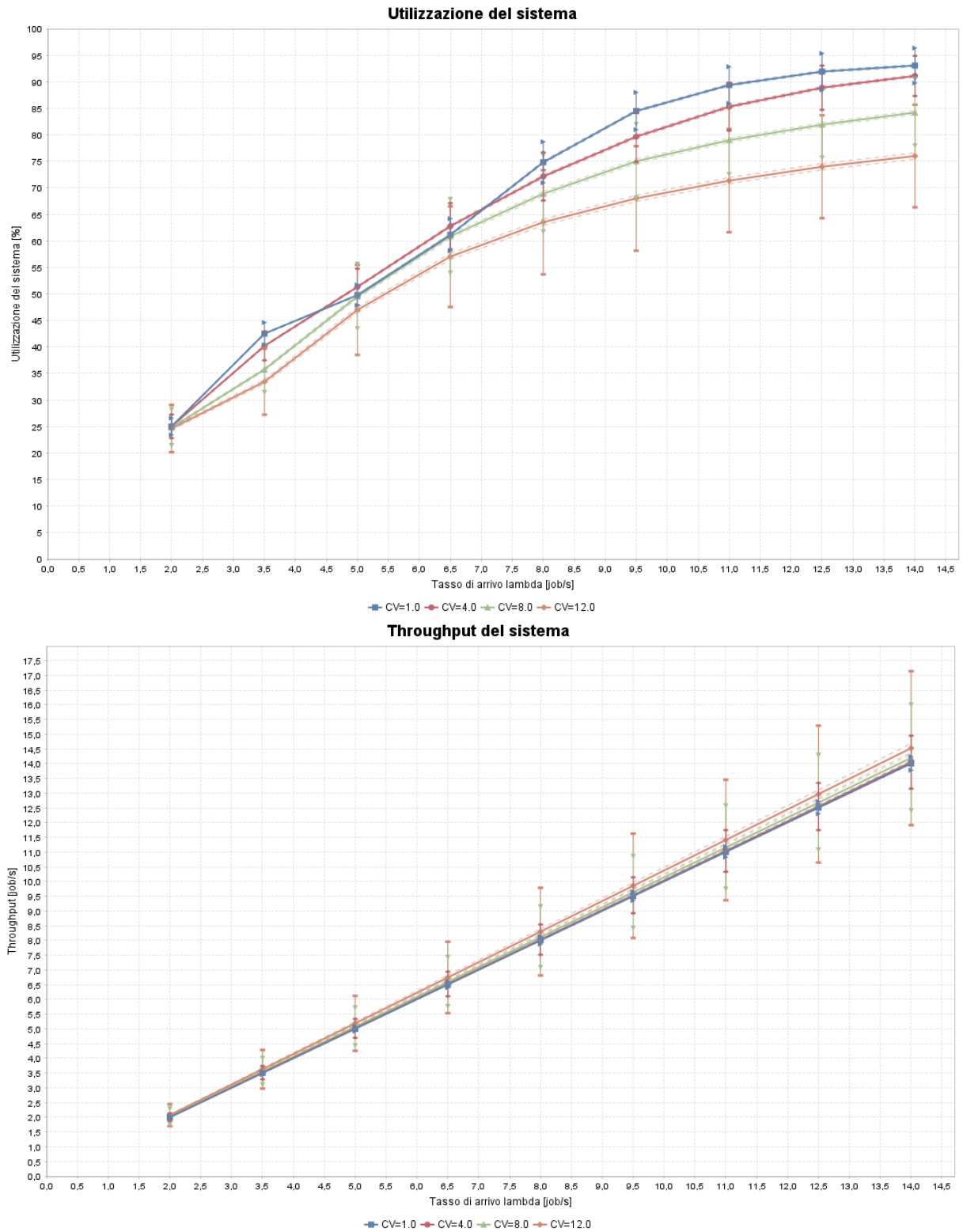


Figura 34: Robustezza alla burstiness: utilizzazione complessiva del sistema e throughput.

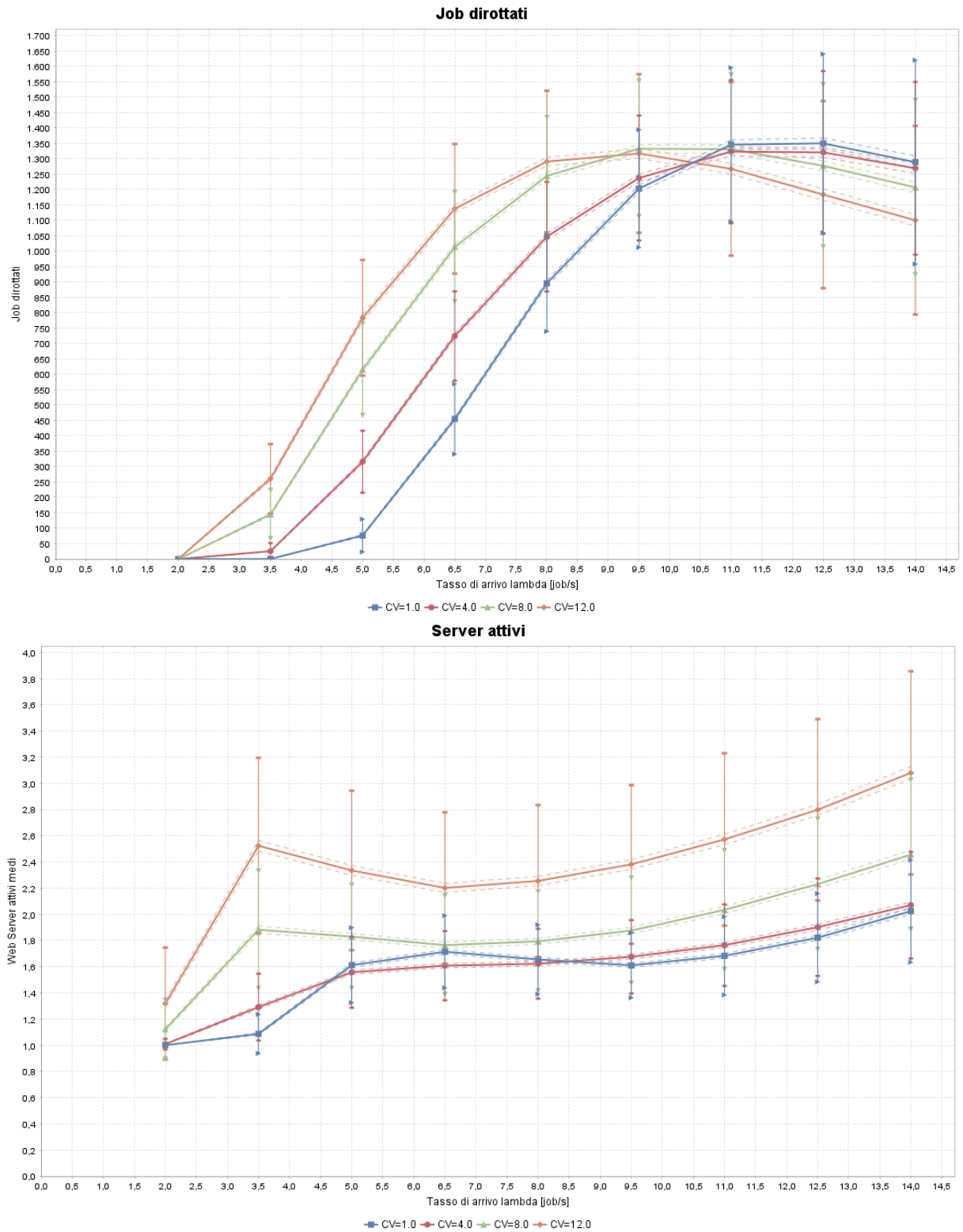


Figura 35: Robustezza alla burstiness: job deviati e numero medio di Web Server attivi.

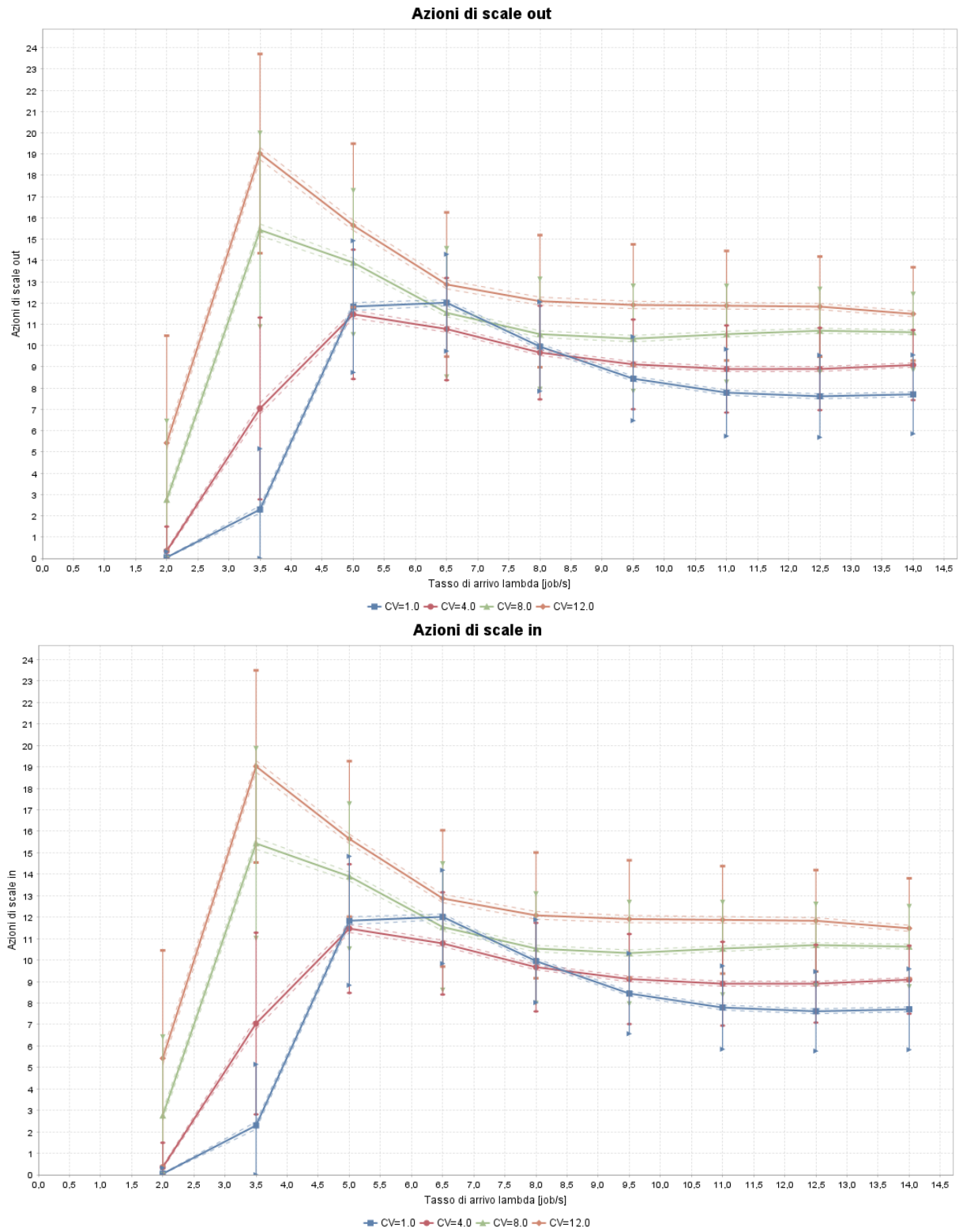


Figura 36: Robustezza alla burstiness: azioni di *Scale Out* e *Scale In*.

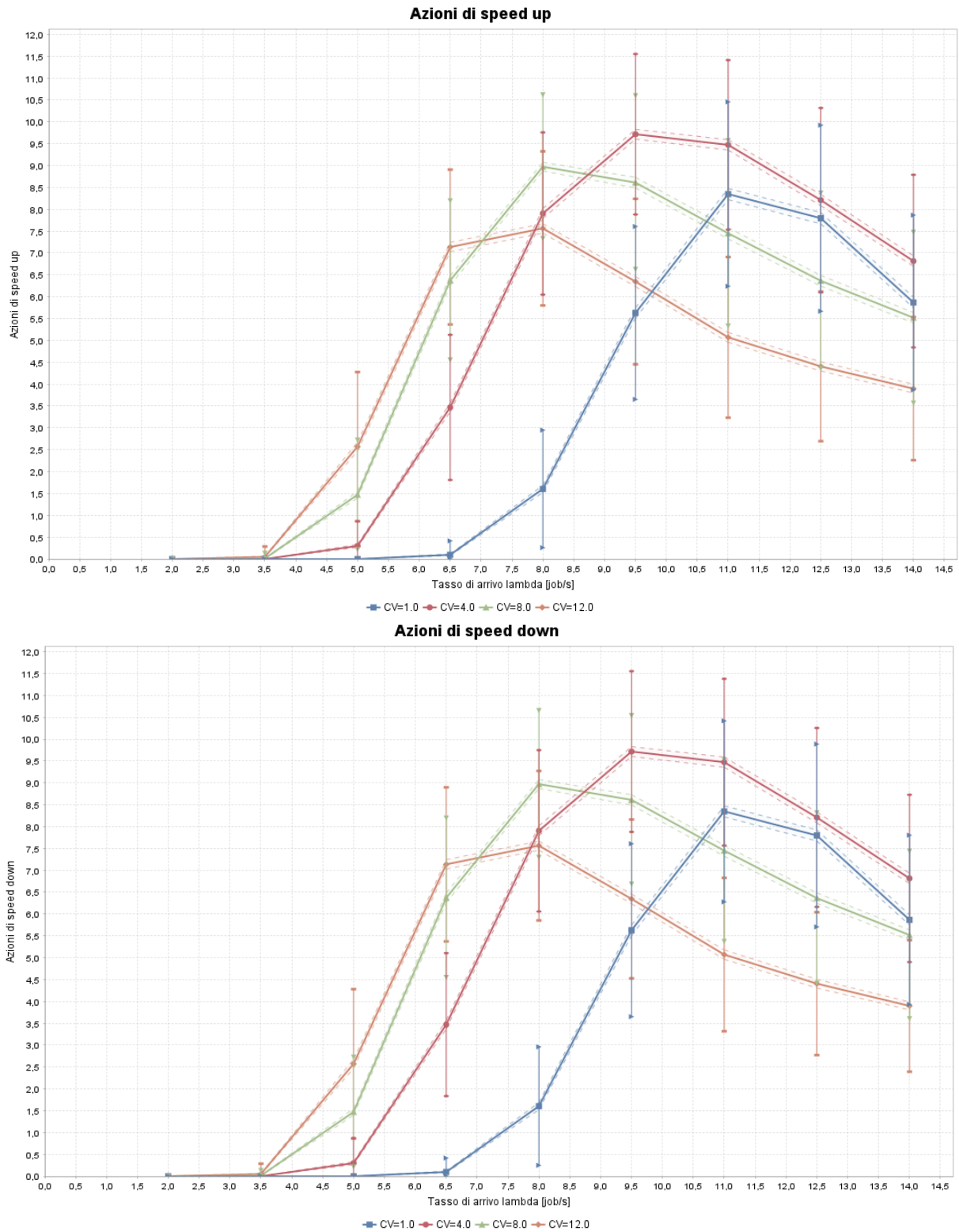


Figura 37: Robustezza alla burstiness: azioni di *Scale Up* e *Scale Down*.

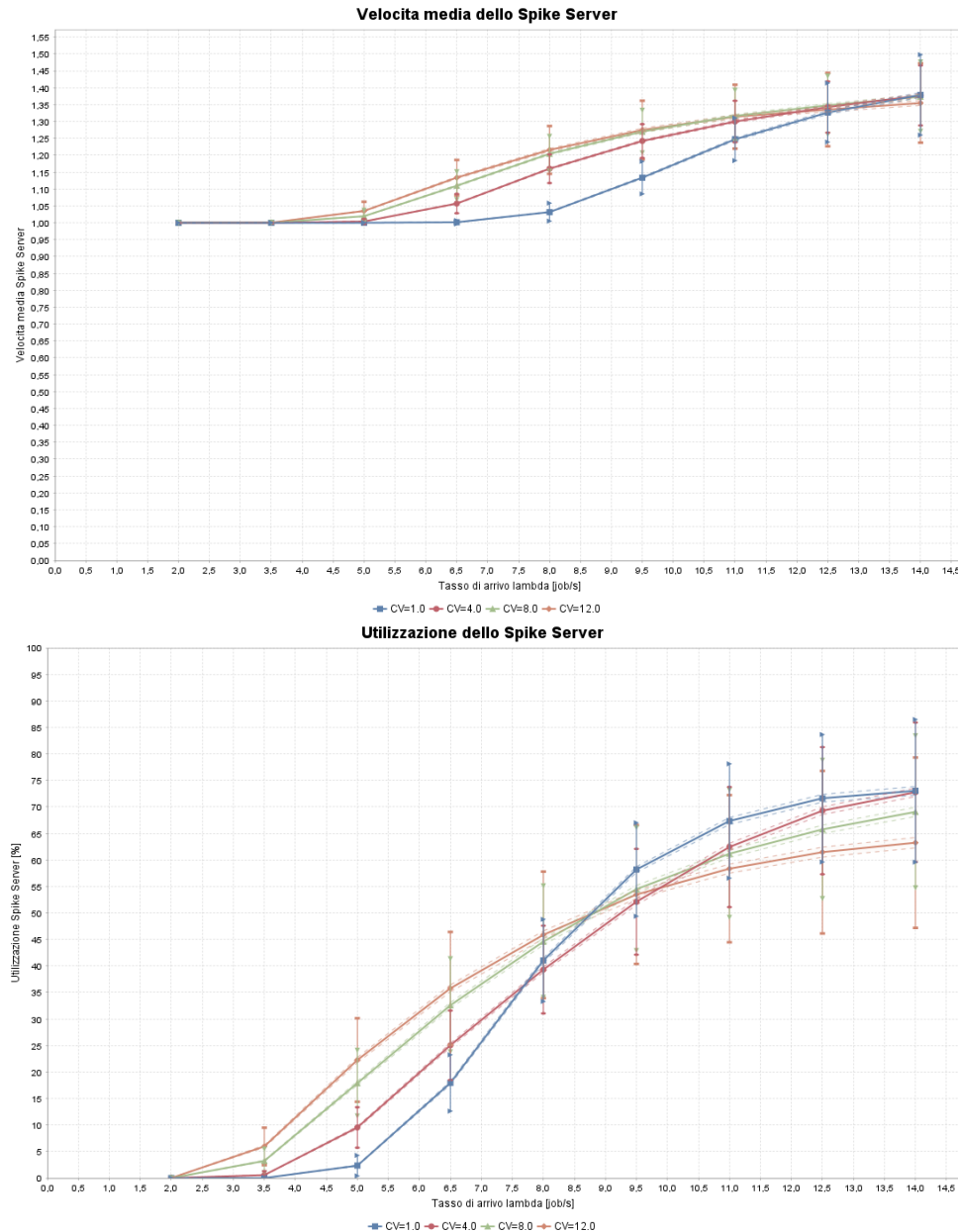


Figura 38: Robustezza alla burstiness: capacità elaborativa media e utilizzazione dello Spike Server.

Le figure confermano la natura adattiva dell'architettura. Il throughput segue correttamente il tasso di arrivo, segnalando assenza di perdita strutturale di capacità; il numero di job nel sistema cresce con λ , ma resta compatibile con tempi medi di risposta sotto soglia; le azioni di scaling aumentano nelle regioni più intense, ma non producono divergenze. La presenza delle barre di deviazione standard e del contorno dell'intervallo di confidenza permette inoltre di distinguere il rumore statistico dalle tendenze effettive: le curve restano separate soprattutto per le metriche operative, mentre il tempo di risposta mostra intervalli contenuti rispetto al margine disponibile sullo SLA.

10. Obiettivo di Miglioramento

L'ultimo obiettivo del progetto introduce una modifica al meccanismo di dirottamento verso lo Spike Server. A differenza delle analisi precedenti, che calibrano i parametri dell'architettura di base, questa sezione studia una variazione del modello: la sostituzione della soglia rigida su SI con un **meccanismo a bande**. Per mantenere lo stesso percorso metodologico usato nel resto del lavoro, la sezione presenta prima la modifica concettuale, poi le specifiche operative, quindi l'implementazione computazionale e infine i risultati sperimentali. Le parti di verifica e validazione del meccanismo migliorativo sono lasciate come attività da completare.

10.1 Modifica al Modello Concettuale

Nel modello base il router decide il dirottamento osservando il solo valore corrente dello *Spike Indicator* del Web Server candidato. Se $SI_i(t) \geq SI_{max}$, il job viene inviato allo Spike Server; non appena un successivo arrivo osserva un valore inferiore alla soglia, il traffico può tornare immediatamente sul Web Server ordinario. Questa logica è semplice, ma può produrre instabilità decisionale quando $SI_i(t)$ oscilla vicino alla soglia.

Il miglioramento introduce una memoria locale dello stato di routing per ciascun Web Server. Ogni server può trovarsi in modalità ordinaria oppure in modalità spike. Il passaggio alla modalità spike avviene al superamento della soglia superiore SI_{max} ; il ritorno alla modalità ordinaria non avviene però al primo valore sotto soglia, ma solo quando lo *Spike Indicator* scende al di sotto di una soglia inferiore SI_{low} . Si definisce quindi una banda:

$$B = SI_{max} - SI_{low}. \quad (35)$$

Il caso $B = 0$ coincide con il modello base, mentre valori positivi introducono una zona di transizione controllata. L'obiettivo non è ridurre indiscriminatamente l'uso dello Spike Server, ma rendere più regolare il suo impiego. Con una soglia rigida, infatti, il sistema tende ad alternare rapidamente fasi di dirottamento e fasi di ritorno al Web Server ordinario, specialmente quando il numero di job residenti oscilla vicino a SI_{max} . Questa dinamica può generare *chattering*: molte decisioni ravvicinate, poco informative e potenzialmente costose. Una banda più ampia evita che il router cambi stato per oscillazioni marginali, accettando una maggiore permanenza del traffico sullo Spike Server in cambio di una risposta più stabile.

10.2 Modello delle Specifiche

La specifica del meccanismo a banda è formalizzata tramite una variabile di stato booleana $M_i(t)$ associata a ciascun Web Server i , dove $M_i(t) = 0$ indica routing ordinario e $M_i(t) = 1$ indica modalità spike attiva. La legge di transizione è:

$$M_i(t^+) = \begin{cases} 1 & \text{se } M_i(t^-) = 0 \text{ e } SI_i(t) \geq SI_{max}, \\ 0 & \text{se } M_i(t^-) = 1 \text{ e } SI_i(t) \leq SI_{low}, \\ M_i(t^-) & \text{altrimenti.} \end{cases} \quad (36)$$

Un job candidato per il server i viene quindi instradato allo Spike Server se e solo se $M_i(t^+) = 1$. In termini operazionali, l'introduzione della banda tende ad aumentare la probabilità di dirottamento q_i rispetto al caso a soglia rigida, poiché il server resta più a lungo in modalità spike dopo l'attivazione. Il comportamento atteso è quindi duplice: da un lato una riduzione del tempo medio di risposta, poiché il Web Server ordinario viene protetto più a lungo; dall'altro un aumento dell'utilizzazione dello Spike Server e del relativo costo operativo.

10.3 Modello Computazionale

La modifica è implementata nella classe `ThresholdSpikeServerRoutingStrategy`. La classe mantiene una mappa `spikeModeMap`, indicizzata per identificativo del Web Server, che conserva lo stato corrente del routing. Il parametro `siLow` rappresenta la soglia inferiore; quando non è specificato, assume lo stesso valore di SI_{max} e il comportamento coincide con il modello base.

```

1  int currentSiLow = (siLow == -1) ? siMax : siLow;
2  boolean currentlyInSpike =
3      spikeModeMap.getOrDefault(targetServer.getId(), false);
4
5  boolean nextState;
6  if (!currentlyInSpike) {
7      nextState = targetServer.getSpikeIndicator() >= siMax;
8  } else {
9      nextState = targetServer.getSpikeIndicator() > currentSiLow;
10 }
11
12 if (nextState != currentlyInSpike) {
13     spikeModeMap.put(targetServer.getId(), nextState);
14     stateChanges++;
15 }

```

La campagna sperimentale è realizzata dalla classe `VerticalScalerStabilityObjective`, che confronta quattro ampiezze di banda, pari a 0, 5, 10 e 20, mantenendo fisso $SI_{max} = 40$ e variando λ e coefficiente di variazione degli interarrivi. Le metriche osservate sono tempo medio di risposta, utilizzazione complessiva del sistema e utilizzazione dello Spike Server, riportate con deviazione standard e intervallo di confidenza.

10.4 Verifica e Validazione

TODO. La verifica funzionale del meccanismo a banda dovrà controllare esplicitamente le transizioni di stato del router, accertando che l'attivazione avvenga solo per $SI \geq SI_{max}$ e che il rilascio richieda il superamento verso il basso della soglia SI_{low} . La validazione dovrà invece confrontare il comportamento aggregato del meccanismo migliorativo con un modello indipendente o con casi limite controllati, verificando che il meccanismo a bande produca effettivamente una riduzione del *chattering* senza alterare la conservazione del flusso.

10.5 Risultati e Considerazioni

La classe `VerticalScalerStabilityObjective` confronta quattro ampiezze di banda, pari a 0, 5, 10 e 20, mantenendo fisso $SI_{max} = 40$ e variando λ e il coefficiente di variazione degli interarrivi. Una banda nulla corrisponde alla soglia rigida: il traffico viene dirottato solo quando il server supera la soglia superiore e il rilascio avviene immediatamente appena il valore torna sotto soglia. Le bande positive introducono invece una soglia inferiore SI_{low} , stabilizzando la decisione del router. L'attesa progettuale è che bande più ampie riducano l'intermittenza del controllo, migliorando la regolarità del tempo di risposta, ma aumentando l'utilizzo medio dello Spike Server.

Banda	R_0 a $\lambda = 5, C_V = 4$	Util. Spike [%]	R_0 a $\lambda = 11, C_V = 4$	Util. Spike [%]
0	3.406	9.5	3.948	62.4
5	3.360	10.6	3.899	65.5
10	3.312	11.7	3.846	68.2
20	3.177	13.5	3.758	74.5

Tabella 19: Effetto della banda sul tempo di risposta e sull'utilizzazione dello Spike Server per $C_V = 4.0$.

I risultati in Tabella 19 mostrano un miglioramento progressivo del tempo medio di risposta al crescere della banda. Per $C_V = 4.0$ e $\lambda = 5$, R_0 passa da 3.406 s con banda nulla a 3.177 s con banda pari a 20, mentre per $\lambda = 11$ passa da 3.948 s a 3.758 s. Il prezzo pagato è un aumento dell'utilizzazione dello Spike Server, che nello scenario a $\lambda = 11$ sale dal 62.4% al 74.5%. Il comportamento è quindi coerente con l'interpretazione di progetto: la banda non elimina il costo della protezione, ma lo trasforma in un uso più continuo e meno intermittente della risorsa ausiliaria.

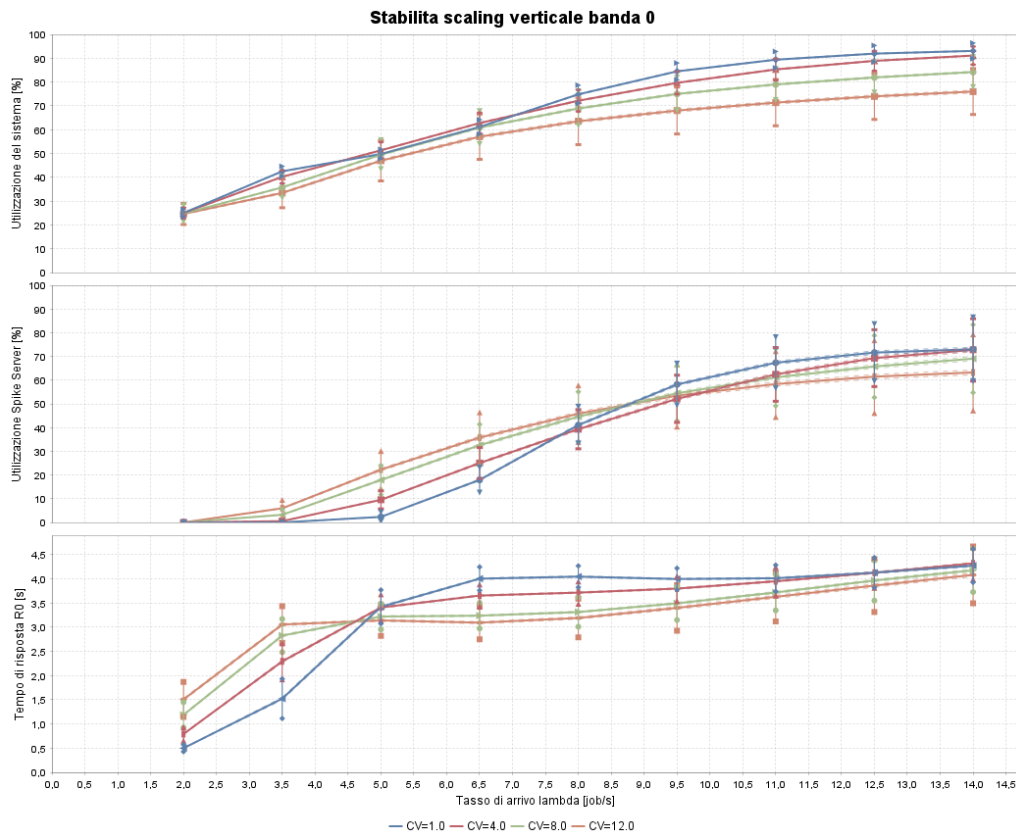


Figura 39: Stabilità dello scaler verticale e prestazioni con banda pari a 0.

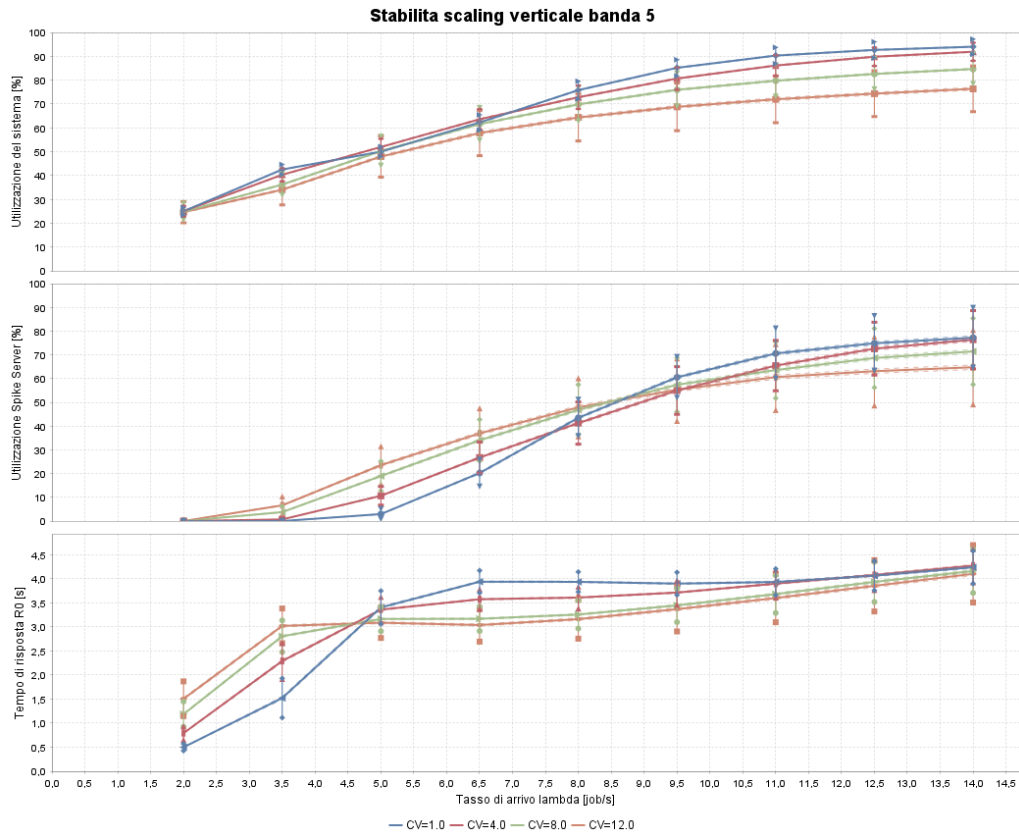


Figura 40: Stabilità dello scaler verticale e prestazioni con banda pari a 5.

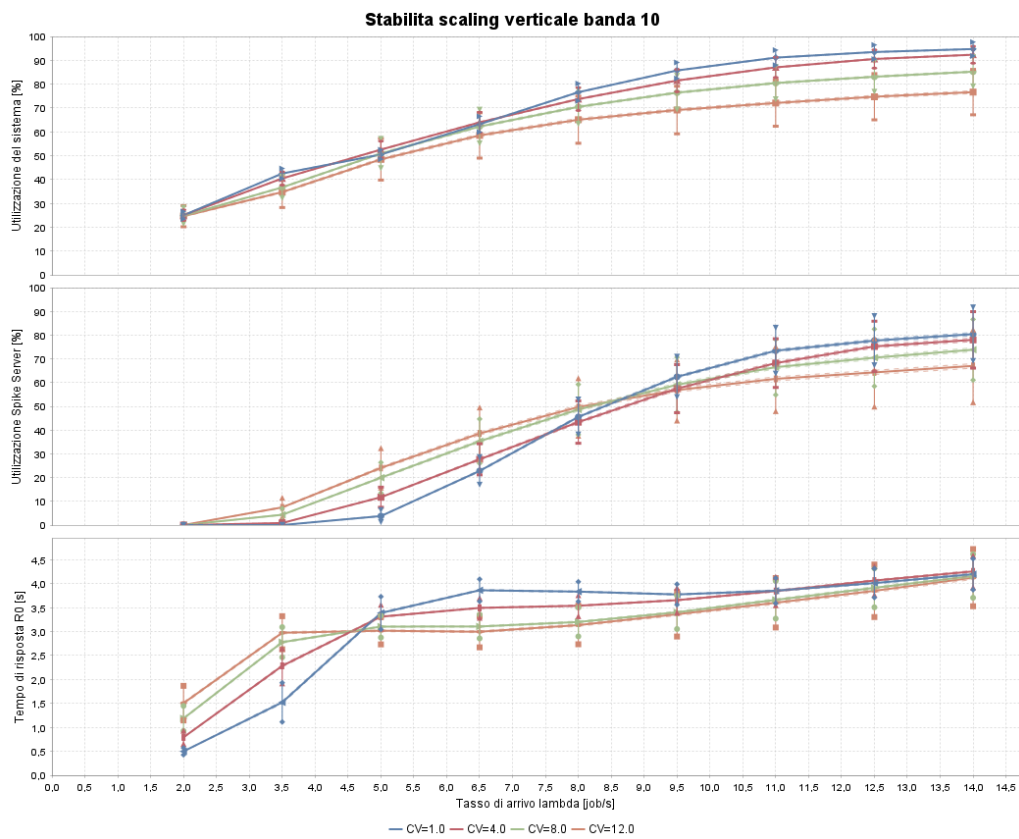


Figura 41: Stabilità dello scaler verticale e prestazioni con banda pari a 10.

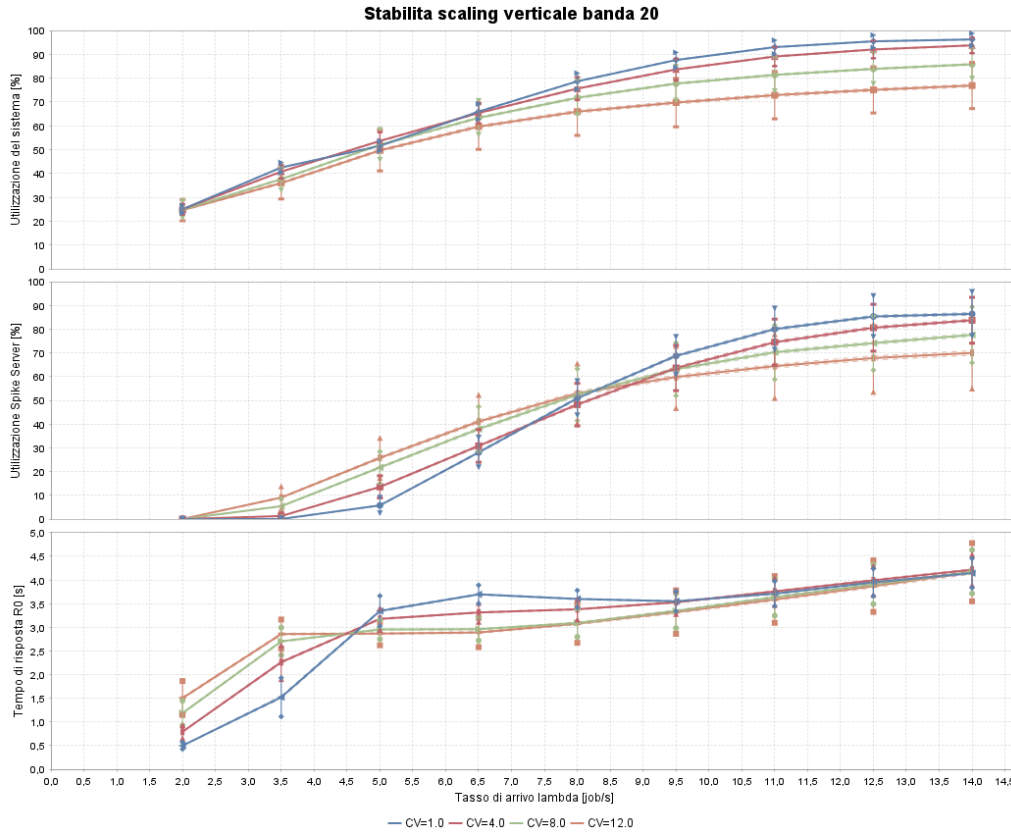


Figura 42: Stabilità dello scaler verticale e prestazioni con banda pari a 20.

Le Figure 39–42 completano l’analisi mostrando tre curve per ciascuna banda: utilizzazione complessiva del sistema, utilizzazione dello Spike Server e tempo medio di risposta. L’aumento della banda rende più alta la curva di utilizzazione dello Spike Server, soprattutto nei regimi ad alta intensità, ma al tempo stesso abbassa la curva di R_0 e ne rende più regolare l’andamento. La banda pari a 20 è la più aggressiva e fornisce i tempi di risposta più bassi, ma sposta una quota maggiore di costo operativo sulla risorsa di picco; una banda intermedia, pari a 10, rappresenta invece un compromesso molto equilibrato, perché produce una riduzione visibile della latenza rispetto alla soglia rigida senza saturare eccessivamente lo Spike Server.

11. Conclusioni

Il lavoro svolto ha permesso di costruire, verificare e utilizzare un simulatore a eventi discreti per lo studio di un’infrastruttura cloud con autoscaling gerarchico. La verifica interna ha mostrato la coerenza del simulatore rispetto ai casi analitici noti, mentre la validazione esterna con JSIM ha confermato che il modello implementato riproduce correttamente sia il comportamento di una singola stazione ad alta variabilità sia l’effetto aggregato dell’architettura Web Server–Spike Server. Le differenze residue osservate nella validazione sono rimaste contenute e spiegabili dalle diverse astrazioni necessarie per rappresentare il routing a soglia nel simulatore del libro.

Dal punto di vista del dimensionamento, il risultato più netto riguarda il ruolo della politica di routing. La strategia *Least Loaded* è risultata la più efficace, con $R_0 = 0.2843$ s nello scenario statico a quattro Web Server, superando sia *Round Robin* sia *Power of Two Choices*. Questo conferma che, anche con disciplina *Processor Sharing*, la distribuzione istantanea dei job tra i server resta decisiva: ridurre la congestione locale significa migliorare simultaneamente media e dispersione dei tempi di risposta.

La soglia di dirottamento SI_{max} si è rivelata il parametro centrale per bilanciare protezione prestazionale e costo operativo della risorsa ausiliaria. La scelta $SI_{max} = 40$ è risultata il miglior punto prima della violazione dello SLA: a tale valore il sistema mantiene $R_0 = 5.5279$ s con intervallo di confidenza interamente sotto 6 s, mentre a $SI_{max} = 50$ la latenza supera già il vincolo. L'analisi dell'incremento verticale di capacità ha invece mostrato che incrementi additivi più aggressivi non portano benefici apprezzabili; per questo motivo è stato scelto $\Delta v_s = 1.0$, che, sommato al valore base $v_{base} = 1.0$, porta la capacità a 2.0 nel momento di attivazione.

Lo scaling orizzontale completa il controllo sul lungo periodo. La soglia superiore $R^{max} = 6.0$ s è stata mantenuta coincidente con il vincolo SLA, mentre $R^{min} = 3.5$ s è risultata coerente con uno scenario nominale centrato su $\lambda \simeq 5$. Le analisi di frontiera mostrano però che questa scelta non è universale: in presenza di carichi persistentemente elevati, prossimi a $\lambda = 9.5$ o $\lambda = 11$, è preferibile abbassare la soglia di *Scale In* nell'intervallo 1.5–2.0 s, accettando un maggior costo infrastrutturale in cambio di più margine operativo.

L'analisi economica ha quantificato in modo esplicito il compromesso tra costo e prestazioni. Con *Cooldown* pari a 10 s, le soglie di *Scale In* più permissive, come 3.5 e 5.0 s, riducono il costo medio rispettivamente a 4.496 e 4.302, mantenendo comunque il tempo medio di risposta sotto SLA anche a $\lambda = 11$. Soglie più aggressive, come 1.5 e 2.0 s, garantiscono latenze inferiori ma richiedono un numero medio maggiore di server attivi, con costi medi pari a 5.282 e 5.040. Il *Cooldown* pari a 10 s è emerso come scelta ragionevole perché evita sia l'iper-reattività del caso a 5 s sia l'eccessivo ritardo di adattamento osservabile nei casi a 50 e 100 s.

Le analisi transitorie e di burstiness mostrano la robustezza dell'architettura completa. Partendo da sistema vuoto, tutti i sei scenari a orizzonte finito convergono a tempi medi di risposta ampiamente inferiori allo SLA, anche quando il coefficiente di variazione degli interarrivi aumenta. In regime stazionario, la burstiness non produce instabilità: il sistema reagisce aumentando il ricorso allo Spike Server e alle azioni di scaling, mantenendo però R_0 in una regione controllata. Questo comportamento conferma l'efficacia della gerarchia: lo scaling orizzontale assorbe le variazioni strutturali del carico, mentre lo Spike Server protegge i Web Server ordinari dai burst locali che si sviluppano su scala temporale più rapida.

Infine, il meccanismo a bande rappresenta il miglioramento più interessante dal punto di vista del controllo. La soglia rigida funziona, ma può generare decisioni intermittenti vicino al limite. L'introduzione di una banda riduce il *chattering*, rende più continuo l'uso dello Spike Server e abbassa il tempo medio di risposta. Una banda pari a 20 massimizza il beneficio prestazionale, ma aumenta sensibilmente l'utilizzazione della risorsa di picco; una banda pari a 10 appare quindi come compromesso progettuale più equilibrato, perché migliora la stabilità senza spostare eccessivamente il costo sulla componente verticale.

In sintesi, l'architettura gerarchica proposta risulta efficace perché separa correttamente le scale temporali del problema: routing e Spike Server agiscono come protezione immediata contro i picchi, mentre lo scaling orizzontale governa il dimensionamento strutturale del cluster. Il simulatore sviluppato consente di quantificare questo equilibrio e fornisce linee guida operative precise: *Least Loaded* come politica di routing, $SI_{max} = 40$, incremento verticale pari a 1.0, $R^{max} = 6.0$ s, $R^{min} = 3.5$ s per il carico nominale, *Cooldown* pari a 10 s e banda preferibilmente intermedia. Queste scelte permettono di rispettare lo SLA con margine, contenendo al tempo stesso il costo operativo e mantenendo stabile il comportamento dinamico del sistema.

Riferimenti bibliografici

- [1] L. M. Leemis and S. K. Park, *Discrete-Event Simulation: A First Course*, Pearson Prentice Hall, 2006.
- [2] G. Serazzi, *Performance Engineering: Learning Through Applications Using JMT*, Springer Nature Switzerland AG, 2024.
- [3] G. Serazzi, M. Gribaudo, G. Casale, et al., “Java Modelling Tools - JMT”, open-source suite, <http://jmt.sourceforge.net/>.
- [4] M. Harchol-Balter, *Performance Modeling and Design of Computer Systems: Queueing Theory in Action*, Cambridge University Press, 2013.
- [5] S. K. Park and K. W. Miller, “Random Number Generators: Good Ones Are Hard To Find”, *Communications of the ACM*, Vol. 31, No. 10, pp. 1192-1201, 1988.
- [6] Dispense del corso, *Interval Estimation*, materiale didattico di simulazione, 2025/2026.
- [7] F. Masci, *Load Manager Simulator*, repository GitHub, <https://github.com/F-masci/load-manager-simulator>.